

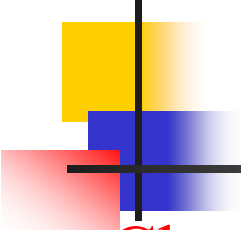
**Trường Đại học Bách khoa
Khoa Công Nghệ Thông Tin**

**BÀI GIẢNG MÔN HỌC
LÝ THUYẾT ÔTÔMÁT & NNHT**

Giảng Viên: Hồ Văn Quân

E-mail: hcquan@dit.hcmut.edu.vn

Web site: <http://www.dit.hcmut.edu.vn/~hcquan/student.htm>



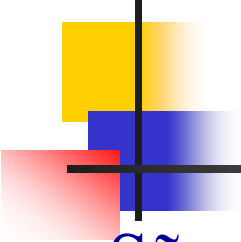
NỘI DUNG MÔN HỌC

- Chương 1 Giới thiệu về lý thuyết tính toán
- Chương 2 Ôtômát hữu hạn
- Chương 3 Ngôn ngữ chính qui và văn phạm chính qui
- Chương 4 Các tính chất của ngôn ngữ chính qui
- Chương 5 Ngôn ngữ phi ngữ cảnh
- Chương 6 Đơn giản hóa văn phạm phi ngữ cảnh và các dạng chuẩn
- Chương 7 Ôtômát đẩy xuống
- Chương 8 Các tính chất của ngôn ngữ phi ngữ cảnh
- Chương 9 Máy Turing



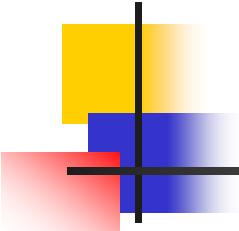
TÀI LIỆU THAM KHẢO

1. Bài giảng lý thuyết Ngôn ngữ Hình thức và Automat - *Hồ Văn Quân* [2002].
2. An Introduction to Formal Languages and Automata - *Peter Linz* [1990].



HÌNH THỨC ĐÁNH GIÁ

- Sẽ có thông báo cụ thể cho từng khóa học. Tuy nhiên, thường là như được cho bên dưới.
- Thi trắc nghiệm
 - Thời gian: **120** phút
 - Số lượng: **50** câu
 - Được phép xem tài liệu trong 4 tờ giấy A4
- Làm bài tập lớn cộng điểm (không bắt buộc)
 - Nộp bài tập lớn và báo cáo vào cuối học kỳ
 - Cộng tối đa 2 điểm



CÁC MÔN LIÊN QUAN

- Ngôn ngữ lập trình
- Trình biên dịch (*)
- Toán tin học



Chương 1 Giới thiệu về lý thuyết tính toán

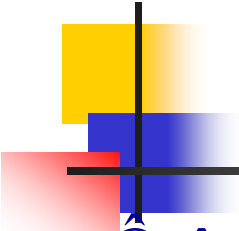
1.1 Giới thiệu

1.2 Yêu cầu về kiến thức nền

1.3 Ba khái niệm cơ bản

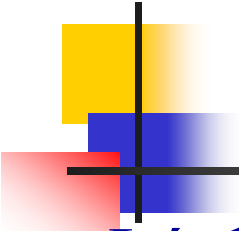
- **Ngôn ngữ (languages)**
- **Văn phạm (grammar)**
- **Ôtômát (máy tự động)**

1.4 Một vài ứng dụng



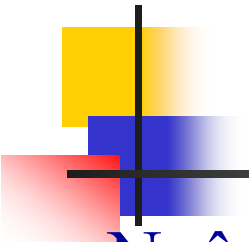
Giới thiệu

- Ôtômát
 - Các mô hình tính toán tự động
- Ngôn ngữ hình thức (formal languages):
 - Định nghĩa
 - Phân loại ngôn ngữ
 - Quan hệ với ôtômát
 - Ứng dụng vào việc xây dựng các ngôn ngữ lập trình
 - ...



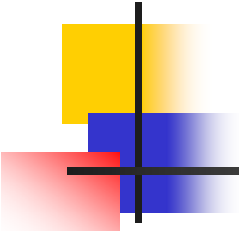
Yêu cầu về kiến thức nền

- Lý thuyết
 - Tập hợp
 - Đồ thị
- Kỹ thuật chứng minh
 - Qui nạp
 - Phản chứng
- Kỹ thuật mô phỏng



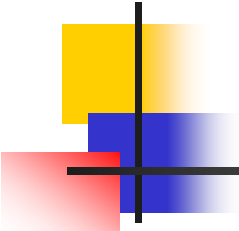
Ba khái niệm cơ bản

- Ngôn ngữ (languages)
- Văn phạm (grammar)
- Ôtômát (automata)



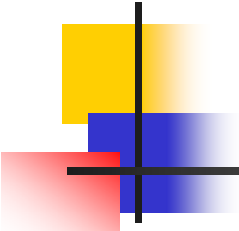
Ngôn ngữ

- Ngôn ngữ là gì?
 - *Các từ điển định nghĩa ngôn ngữ một cách không chính xác là một hệ thống thích hợp cho việc biểu thị các ý nghĩ, các sự kiện, hay các khái niệm, bao gồm một tập các kí hiệu và các qui tắc để vận dụng chúng.*
- Định nghĩa trên chưa đủ chính xác để nghiên cứu về NNHT
- Chúng ta cần xây dựng một định nghĩa toán học cho khái niệm ngôn ngữ



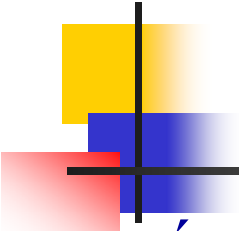
Các khái niệm

- Bảng chữ cái (alphabet), Σ
 - Là tập hợp Σ hữu hạn không trống các kí hiệu (symbol).
- Ví dụ
 - $\{A, B, C, \dots, Z\}$: Bảng chữ cái La tinh.
 - $\{\alpha, \beta, \gamma, \dots, \varphi\}$: Bảng chữ cái Hi Lạp.
 - $\{0, 1, 2, \dots, 9\}$: Bảng chữ số thập phân.
 - $\{I, V, X, L, C, D, M\}$: Bảng chữ số La Mã.



Các khái niệm (tt)

- Chuỗi (string), w
 - Là một dãy hữu hạn các kí hiệu từ bảng chữ cái.
- Ví dụ
 - Với $\Sigma = \{a, b\}$, thì ***abab*** và ***aaabbba*** là các chuỗi trên Σ .
- Qui ước
 - Với một vài ngoại lệ, chúng ta sẽ sử dụng các chữ cái thường $a, b, c, \dots$ cho các phần tử của Σ còn các chữ cái $u, v, w, \dots$ cho các tên chuỗi.



Các phép toán trên chuỗi

- Kết nối (concatenation), wv

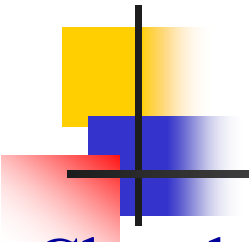
- $w = a_1a_2 \dots a_n$ và $v = b_1b_2 \dots b_m$ là chuỗi:

$$wv = a_1a_2 \dots a_nb_1b_2 \dots b_m$$

- Đảo (reverse), w^R

- Đảo của chuỗi $w = a_1a_2 \dots a_n$ là chuỗi:

$$w^R = a_n \dots a_2a_1$$



Các khái niệm (tt)

Cho chuỗi $w = uv$

- Tiếp đầu ngữ (prefix)
 - u được gọi là tiếp đầu ngữ của w
- Tiếp vĩ ngữ (suffix)
 - v được gọi là tiếp vĩ ngữ của w
- Chiều dài của chuỗi w
 - Là số kí hiệu trong chuỗi, và được kí hiệu là $|w|$
- Chuỗi trống (empty string)
 - Là chuỗi không có kí hiệu nào, thường được kí hiệu là λ



Các khái niệm (tt)

■ Nhận xét

1 . Các quan hệ sau đây đúng với mọi w :

$$|\lambda| = 0; \lambda w = w\lambda = w$$

2 . Nếu u, v là các chuỗi thì :

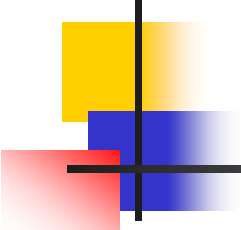
$$|uv| = |u| + |v|$$

■ Lũy thừa (power), w^n

- w là một chuỗi thì w^n là một chuỗi nhận được bằng cách kết nối chuỗi w với chính nó n lần.

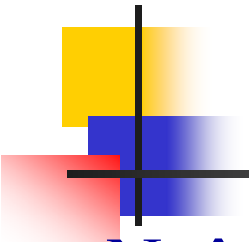
- $w^0 = \lambda$

$$w^n = \underbrace{w \cdots w}_{n \text{ lần}}$$



Các khái niệm (tt)

- Σ^*, Σ^+ (bao đóng sao và bao đóng dương)
 - Σ^* là tập tất cả các chuỗi trên Σ kể cả chuỗi trống.
 - Σ^+ là tập tất cả các chuỗi trên Σ ngoại trừ chuỗi trống.
 - $\Sigma^* = \Sigma^+ \cup \{\lambda\}$; $\Sigma^+ = \Sigma^* - \{\lambda\}$
- Σ thì hữu hạn còn Σ^+ và Σ^* là vô hạn đếm được



Định nghĩa ngôn ngữ

■ Ngôn ngữ

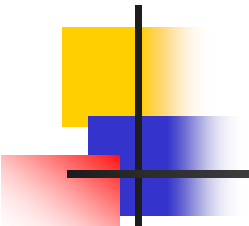
- Là một tập con của Σ^* , hay nói cách khác là một tập bất kỳ các câu trên bộ chữ cái.

■ Ví dụ

- Cho $\Sigma = \{a, b\}$

$$\Sigma^* = \{\lambda, a, b, aa, ab, ba, bb, aaa, aab, \dots\}$$

- Tập $\{a, aa, aab\}$ là một ngôn ngữ trên Σ . Nó là một ngôn ngữ hữu hạn.
- Tập $L = \{a^n b^n : n \geq 0\}$ cũng là một ngôn ngữ trên Σ . Nó là một ngôn ngữ vô hạn.



Các phép toán trên ngôn ngữ

- Bù (complement), $\overline{L}$

- Bù của ngôn ngữ L trên bảng chữ cái Σ , được kí hiệu là:

$$\overline{L} = \Sigma^* - L$$

- Kết nối, $L_1 L_2$

- Cho 2 ngôn ngữ L_1, L_2 . Kết nối của 2 ngôn ngữ L_1, L_2 là:

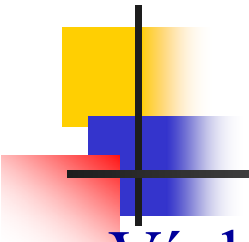
$$L_1 L_2 = \{ xy : x \in L_1, y \in L_2 \}$$

- Lũy thừa, L^n

- Lũy thừa bậc n của L , kí hiệu là L^n , là việc kết nối L với chính nó n lần

- $L^0 = \{\lambda\}$

$$L^n = \underbrace{L \cdots L}_{n \text{ lần}}$$



Các phép toán trên ngôn ngữ (tt)

- Ví dụ

- Cho $L = \{a^n b^n : n \geq 0\}$, thì

$$L^2 = \{a^n b^n a^m b^m : n \geq 0, m \geq 0\}$$

- Bao đóng-sao (star-closure) của L

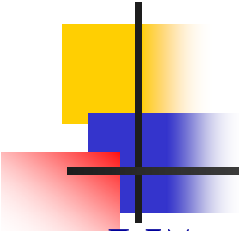
- Kí hiệu là L^* và được định nghĩa là

$$L^* = L^0 \cup L^1 \cup L^2 \cup \dots$$

- Bao đóng dương (positive closure) của L

- Kí hiệu là L^+

$$L^+ = L^1 \cup L^2 \cup L^3 \cup \dots$$



Văn phạm

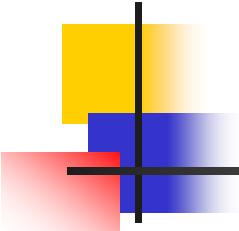
- Văn phạm là gì?

- *Các từ điển định nghĩa văn phạm một cách không chính xác là một tập các qui tắc về cấu tạo từ và các qui tắc về cách liên kết các từ lại thành câu.*

- Ví dụ

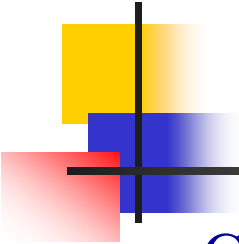
- Cho đoạn văn phạm tiếng Anh sau

<sentence> → <noun phrase><predicate>,
<noun phrase> → <article><noun>,
<predicate> → <verb>,
<article> → a | the,
<noun> → boy | dog,
<verb> → runs | walks,



Định nghĩa văn phạm

- Các câu “*a boy runs*” và “*the dog walks*” là có "dạng đúng“, tức là được sinh ra từ các luật của văn phạm.
- Định nghĩa 1.1
 - Văn phạm G được định nghĩa như là một bộ bốn
$$G = (V, T, S, P)$$
 - V : tập các ***kí hiệu không kết thúc*** (nonterminal symbol), còn được gọi là các ***biến*** (variable),
 - T : tập các ***kí hiệu kết thúc*** (terminal symbol),
 - $S \in V$: được gọi là ***biến khởi đầu*** (start variable), đôi khi còn được gọi là ***kí hiệu mục tiêu***,
 - P : tập hữu hạn các ***luật sinh*** (production),



Định nghĩa văn phạm (tt)

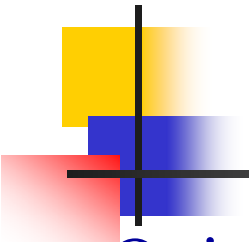
- Các luật sinh có dạng $x \rightarrow y$ trong đó $x \in (V \cup T)^+$ và có chứa ít nhất một biến, $y \in (V \cup T)^*$.
- Các **luật sinh** (production) đôi khi còn được gọi là các **qui tắc** (rule) hay **luật viết lại** (written rule).
- Ví dụ
 - Cho văn phạm sau:

$G = (\{S, A, B\}, \{a, b\}, S, P)$, với P :

$$S \rightarrow aAS \mid bBS \mid \lambda,$$

$$A \rightarrow aaA \mid b,$$

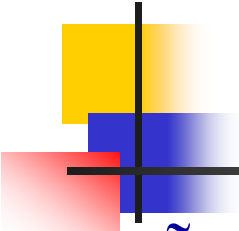
$$B \rightarrow bbB \mid a,$$



Văn phạm (tt)

■ Qui ước:

- Các kí tự chữ hoa A, B, C, D, E và S biểu thị các biến; S là kí hiệu khởi đầu trừ phi được phát biểu khác đi.
- Các kí tự chữ thường a, b, c, d, e , các kí số, các chuỗi in đậm biểu thị các kí hiệu kết thúc (terminal).
- Các kí tự chữ hoa X, Y, Z biểu thị các kí hiệu có thể là terminal hoặc biến.
- Các kí tự chữ thường u, v, w, x, y, z biểu thị chuỗi các terminal.
- Các kí tự chữ thường Hi Lạp α, β, γ biểu thị chuỗi các biến và các terminal.



Các khái niệm

- Dẫn xuất trực tiếp (directly derive), $\Rightarrow$
 - Cho luật sinh $x \rightarrow y$ và chuỗi $w = uxv$.
 - Luật sinh trên có thể áp dụng tới chuỗi w . Khi áp dụng ta sẽ nhận được chuỗi mới

$$z = uyv$$

- w **dẫn xuất ra** z hay ngược lại z **được dẫn xuất ra từ** w và kí hiệu là:

$$uxv \Rightarrow uyv$$

Ngôn ngữ được sinh ra bởi văn phạm

■ Dẫn xuất gián tiếp $\Rightarrow^*$, $\Rightarrow^+$

- Nếu $w_1 \Rightarrow w_2 \Rightarrow \dots \Rightarrow w_n$ thì ta nói w_1 dẫn xuất ra w_n và viết

$$w_1 \Rightarrow^* w_n$$

- Nếu có ít nhất một luật sinh phải được áp dụng chúng ta viết:

$$w_1 \Rightarrow^+ w_n$$

■ Định nghĩa 1.2

- Cho $G = (V, T, S, P)$ là một văn phạm, thì tập:

$$L(G) = \{w \in T^* : S \Rightarrow^* w\}$$

được gọi là ngôn ngữ được sinh ra bởi G .



Các khái niệm (tt)

- Sự dẫn xuất câu (derivation)

- Nếu $w \in L(G)$ thì phải tồn tại dãy dẫn xuất:

$$S \Rightarrow w_1 \Rightarrow w_2 \Rightarrow \dots \Rightarrow w_n \Rightarrow w$$

Dãy này được gọi là một sự dẫn xuất câu của w .

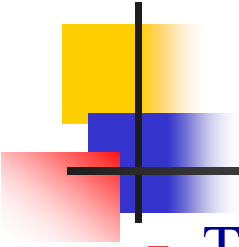
- Dạng câu (sentential forms)

- Dãy $S, w_1, w_2, \dots, w_n$ được gọi là **các dạng câu** của sự dẫn xuất. Câu w cũng được xem là một dạng câu đặc biệt.

- Ví dụ

- Cho văn phạm

$$G = (\{S\}, \{a, b\}, S, P), \text{ với } P$$
$$S \rightarrow aSb \mid \lambda.$$



Các khái niệm (tt)

- Thì

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$$

là một dãy dẫn xuất. Vì vậy có thể viết

$$S \xRightarrow{*} aabb$$

- Chuỗi $aabb$ là một câu của ngôn ngữ được sinh ra bởi G , còn $aaSbb$ là một dạng câu.
- Ngôn ngữ tương ứng với văn phạm này là:

$$L(G) = \{a^n b^n : n \geq 0\} .$$



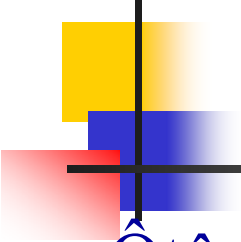
Bài tập văn phạm

- Mô tả toán học cho ngôn ngữ
 - Ngôn ngữ L_1 bao gồm các chuỗi từ khóa ***begin***, ***end*** của ngôn ngữ Pascal. Các chuỗi biểu diễn cấu trúc lồng nhau của các cặp từ khóa này trong các chương trình trên ngôn ngữ Pascal.
 - Ngôn ngữ L_2 bao gồm tập các danh hiệu của Pascal.
- Xác định ngôn ngữ của văn phạm
 - G_1 $S \rightarrow aSbS \mid bSaS \mid \lambda$
 - G_2 $E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid a \mid b$



Bài tập văn phạm (tt)

- Xây dựng văn phạm cho ngôn ngữ
 - Ngôn ngữ L_1 và L_2 ở trang trên
 - $L_3 = \{ww^R : w \in \{a, b\}^*\}$
 - $L_4 = \{a^n b^m c^{n+m} : n, m \geq 0\}$
 - $L_5 = \{a^n b^{n+m} c^m : n, m \geq 0\}$



Ôtômát

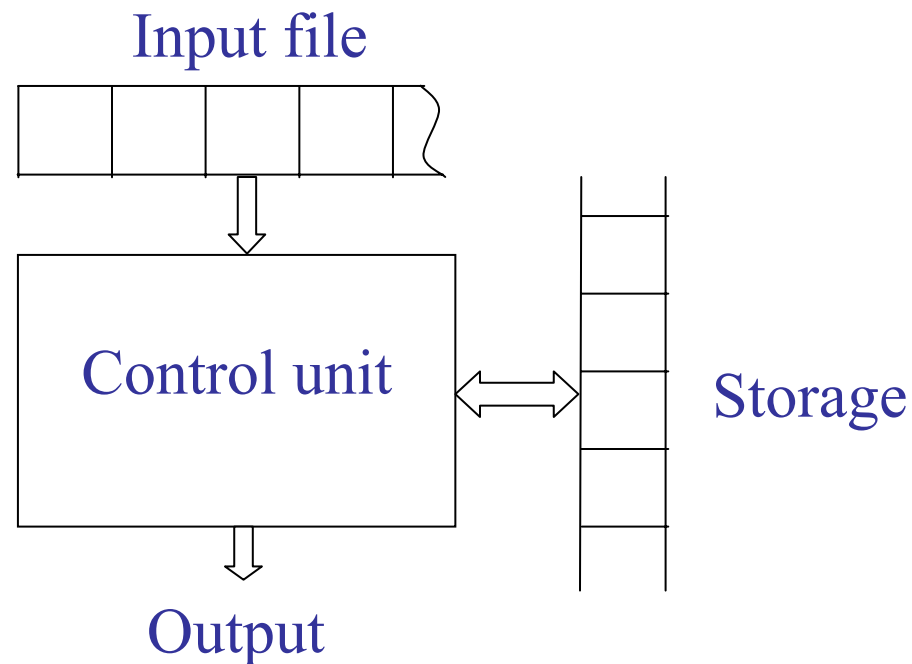
■ Ôtômát là gì?

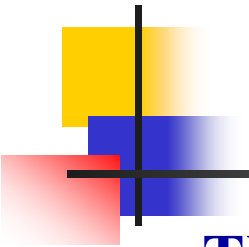
- Ôtômát, dịch nghĩa là máy tự động, là thiết bị có thể tự thực hiện công việc mà *không cần sự can thiệp của con người*.
- Nó hoạt động dựa trên một số quy tắc và dựa vào các quy tắc này con người lập trình cho nó hoạt động theo ý muốn của mình.
- Máy tính số ngày nay chính là một máy tự động điển hình và mạnh nhất hiện nay.

Định nghĩa ô tô máy

■ Ô tô máy

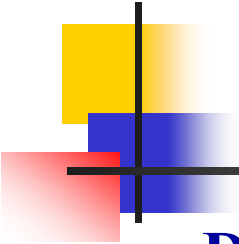
- Là một mô hình trừu tượng của máy tính số bao gồm các thành phần chủ yếu sau





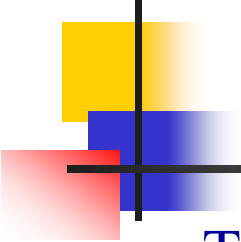
Định nghĩa ô tô m át (tt)

- **Thiết bị đầu vào** (input file): là nơi mà các chuỗi nhập (input string) được ghi lên, và được ô tô m át đọc nhưng không thay đổi được nội dung của nó. Nó được chia thành các ô (cells, squares), mỗi ô giữ được một kí hiệu.
- **Cơ cấu nhập** (input mechanism): là bộ phận có thể đọc input file từ trái sang phải, một kí tự tại một thời điểm. Nó cũng có thể dò tìm được điểm kết thúc của chuỗi nhập (eof, #).
- **Bộ nhớ tạm** (temporary storage): là thiết bị bao gồm một số không giới hạn các ô nhớ (cell), mỗi ô có thể giữ một kí hiệu từ một bảng chữ cái (không nhất thiết giống với bảng chữ cái ng ữ nhập). Ô tô m át có thể đọc và thay đổi được nội dung của các ô nhớ lưu trữ (storage cell).



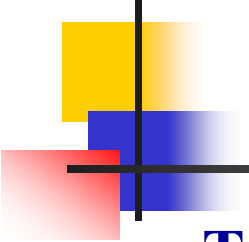
Hoạt động của ô tô máy

- **Đơn vị điều khiển** (control unit): mỗi ô tô máy có một đơn vị điều khiển, cái mà có thể ở trong một trạng thái bất kỳ trong một số hữu hạn các **trạng thái nội**, và có thể chuyển đổi trạng thái trong một kiểu được định nghĩa sẵn nào đó.
- Hoạt động của ô tô máy
 - Một ô tô máy được giả thiết là hoạt động trong một khung **thời gian rời rạc** (discrete time frame).
 - Tại một thời điểm bất kỳ đã cho, đơn vị điều khiển đang ở trong một **trạng thái nội** (internal state) nào đó, và cơ cấu nhập là đang quét (scanning) một kí hiệu cụ thể nào đó trên input file.



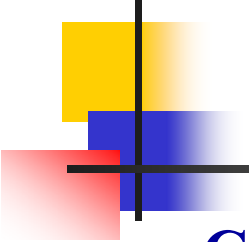
Hoạt động của ô tô mát (tt)

- Trạng thái nội của đơn vị điều khiển tại thời điểm kế tiếp được xác định bởi **trạng thái kế** (next state) hay bởi **hàm chuyển trạng thái** (transition function).
- Trong suốt quá trình chuyển trạng thái từ khoảng thời gian này đến khoảng thời gian kế, **kết quả** (output) có thể được sinh ra và thông tin trong bộ nhớ lưu trữ có thể được thay đổi.



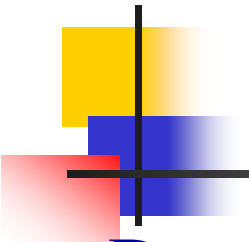
Các khái niệm

- **Trạng thái nội (internal state):** là một trạng thái của đơn vị điều khiển mà nó có thể ở vào.
- **Trạng thái kế (next state):** là một trạng thái nội của đơn vị điều khiển mà nó sẽ ở vào tại thời điểm kế tiếp.
- **Hàm chuyển trạng thái (transition function):** là hàm gởi ra trạng thái kế của ô tômát dựa trên trạng thái hiện hành, kí hiệu nhập hiện hành được quét, và thông tin hiện hành trong bộ nhớ tạm.



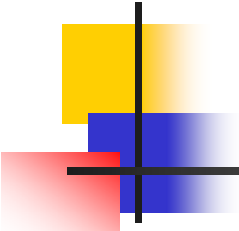
Các khái niệm (tt)

- **Cấu hình (configuration):** được sử dụng để tham khảo đến bộ ba thông tin: trạng thái cụ thể mà đơn vị điều khiển đang ở vào, vị trí của cơ cấu nhập trên thiết bị nhập (hay nói cách khác ô tô máy đang đọc đến kí hiệu nào của thiết bị nhập), và nội dung hiện hành của bộ nhớ tạm.
- **Di chuyển (move):** là sự chuyển trạng thái của ô tô máy từ một cấu hình này sang cấu hình kế tiếp.



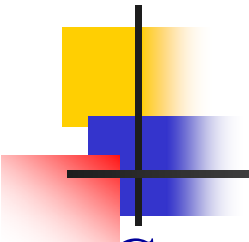
Phân loại ô tô máy

- Dựa vào hoạt động của ô tô máy, có đơn định hay không: có hai loại ô tô máy.
 - **Ô tô máy đơn định (deterministic automata):** là ô tô máy trong đó mỗi di chuyển (move) được xác định duy nhất bởi cấu hình hiện tại. Sự duy nhất này thể hiện tính đơn định.
 - **Ô tô máy không đơn định (non-deterministic automata):** là ô tô máy mà tại mỗi thời điểm nó có một vài khả năng lựa chọn để di chuyển. Việc có một vài khả năng lựa chọn thể hiện tính không đơn định.



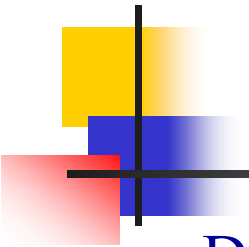
Phân loại ô tô m át (tt)

- Dựa vào kết quả xuất ra của ô tô m át: có hai loại ô tô m át.
 - **Acceptor:** là ô tô m át mà đáp ứng ở ngõ ra của nó được giới hạn trong hai trạng thái đơn giản “**yes**” hay “**no**”. "Yes" tương ứng với việc chấp nhận chuỗi nhập, "no" tương ứng với việc từ chối, không chấp nhận, chuỗi nhập.
 - **Transducer:** là ô tô m át tổng quát hơn, có khả năng sinh ra các chuỗi kí tự ở ngõ xuất. Máy tính số là một transducer điển hình.



Một vài ứng dụng

- Cung cấp kiến thức nền tảng cho việc xây dựng các ngôn ngữ lập trình (NNLT), các trình dịch.
 - Dùng văn phạm để định nghĩa các NNLT.
 - Dùng accepter để định nghĩa một vài thành phần của NNLT.
 - Xây dựng các bộ phân tích từ vựng, phân tích cú pháp cho các NNLT.



Ví dụ

- Dùng văn phạm mô tả danh hiệu của Pascal.

$\langle id \rangle \rightarrow \langle letter \rangle \langle rest \rangle,$

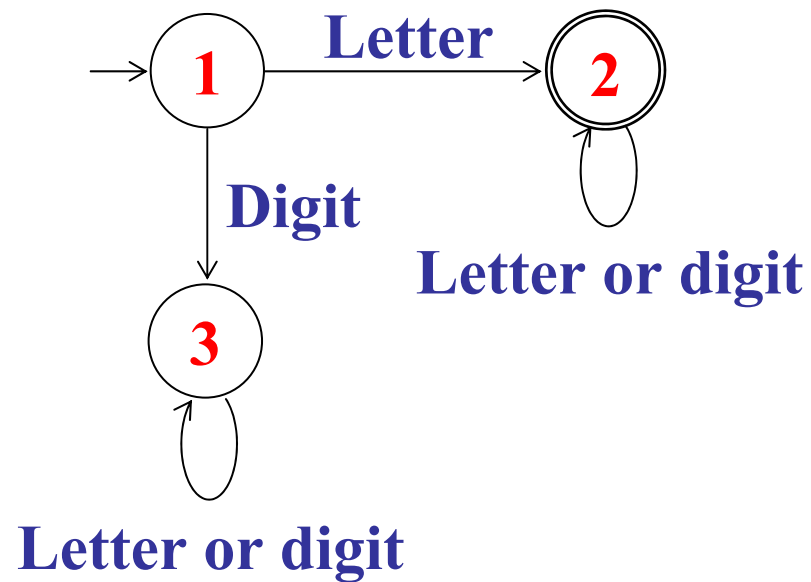
$\langle rest \rangle \rightarrow \langle letter \rangle \langle rest \rangle \mid \langle digit \rangle \langle rest \rangle \mid \lambda,$

$\langle letter \rangle \rightarrow a \dots z \mid A \dots Z$

$\langle digit \rangle \rightarrow 0 \dots 9$

Ví dụ (tt)

- Dùng acceptor mô tả danh hiệu của Pascal.





Ví dụ - Văn phạm Pascal đơn giản

- Một văn phạm đơn giản của ngôn ngữ Pascal

[prog] ::= [prog header] [var part] [stat part]

[prog header] ::= **program** [id] (**input** , **output**) ;

[var part] ::= **var** [var dec list]

[stat part] ::= **begin** [stat list] **end** .

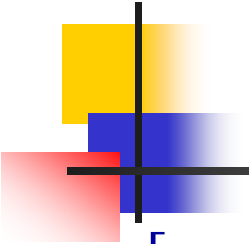
[var dec list] ::= [var dec] | [var dec list] [var dec]

[var dec] ::= [id list] : [type] ;

[stat list] ::= [stat] | [stat list] ; [stat]

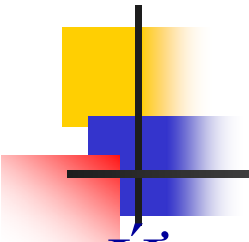
[stat] ::= [assign stat]

[assign stat] ::= [id] := [expr]



Văn phạm Pascal đơn giản (tt)

[assign stat]	::=	[id] := [expr]
[expr]	::=	[operand] [expr] [operator] [operand]
[type]	::=	integer
[id list]	::=	[id] [id list] , [id]
[operand]	::=	[id] [number]
[id]	::=	[letter] [id] [letter] [id] [digit]
[number]	::=	[digit]
[operator]	::=	+ *
[digit]	::=	0 1 2 3 4 5 6 7 8 9
[letter]	::=	a .. z A .. Z

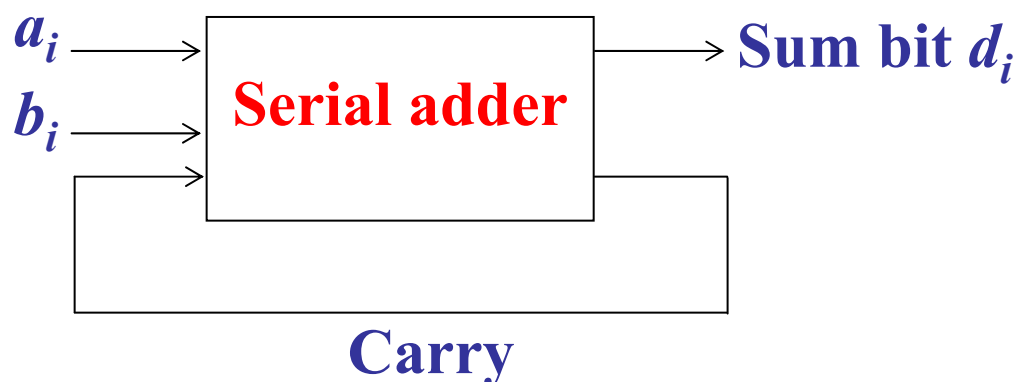


Một vài ứng dụng (tt)

- Ứng dụng vào các lĩnh vực xử lý chuỗi.
 - Các chức năng tìm kiếm, thay thế trong các trình soạn thảo văn bản hoặc xử lý chuỗi.
 - Xử lý ngôn ngữ tự nhiên: chú thích loại từ cho các từ, sửa lỗi chính tả, ...
- Ứng dụng vào lĩnh vực thiết kế số.
- ...

Ví dụ - Mạch cộng

- Xét một bộ cộng nhị phân tuần tự hai số nguyên dương



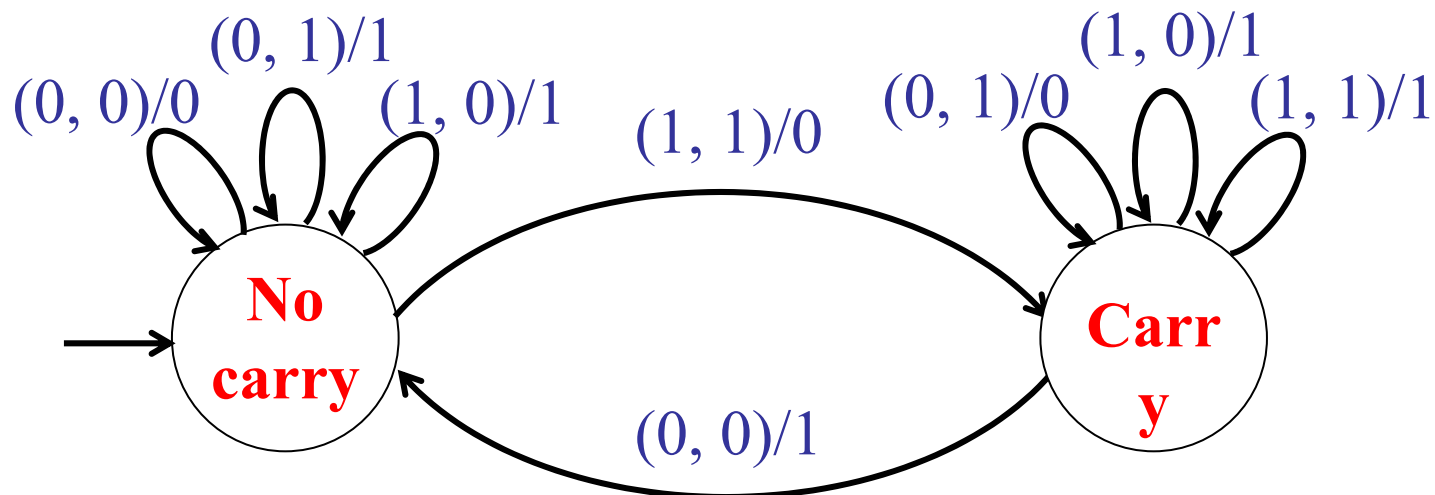
- Trong đó hai chuỗi cộng $x = a_0a_1 \dots a_n$
 $y = b_0b_1 \dots b_m$

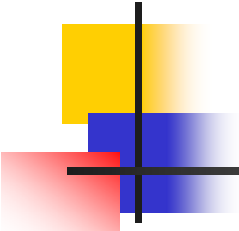
biểu diễn cho hai số nguyên

$$v(x) = \sum_{i=0}^n a_i 2^i \qquad v(y) = \sum_{i=0}^m b_i 2^i$$

Mạch cộng (tt)

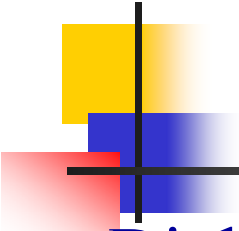
- Sơ đồ khối trên chỉ mô tả những gì mà một bộ cộng phải làm chứ không giải thích chút gì về hoạt động bên trong.
- Sau đây là một ô tômát (cụ thể là một transducer) mô tả hoạt động bên trong của bộ cộng nói trên.





Chương 2 Ôtômát hữu hạn

- 2.1 Acceptor hữu hạn đơn định
- 2.2 Acceptor hữu hạn không đơn định
- 2.3 Sự tương đương giữa acceptor hữu hạn đơn định và acceptor hữu hạn không đơn định
- 2.4 Rút gọn số trạng thái của một ôtômát hữu hạn



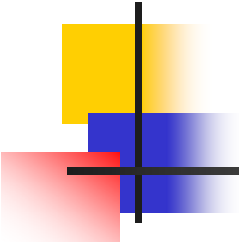
Accepter hữu hạn đơn định

■ Định nghĩa 2.1

Một **accepter hữu hạn đơn định** (deterministic finite state acceptor) hay **dfa** được định nghĩa bởi bộ năm

$$M = (Q, \Sigma, \delta, q_0, F),$$

- Q là một tập hữu hạn các **trạng thái nội** (internal states),
- Σ là một tập hữu hạn các ký hiệu được gọi là **bảng chữ cái ngõ nhập** (input alphabet),
- $\delta: Q \times \Sigma \rightarrow Q$ là **hàm chuyển trạng thái** (transition function).
Để chuyển trạng thái ô tô mát dựa vào trạng thái hiện hành $q \in Q$ nó đang ở vào và kí hiệu nhập $a \in \Sigma$ nó đang đọc được, nó sẽ chuyển sang trạng thái kế được định nghĩa sẵn trong δ .

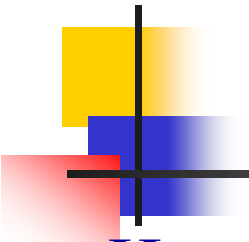


Accepter hữu hạn đơn định (tt)

- $q_0 \in Q$ là **trạng thái khởi đầu** (initial state),
- $F \subseteq Q$ là một tập các **trạng thái kết thúc** (final states) (hay còn được gọi là trạng thái **chấp nhận**).

■ Chú ý

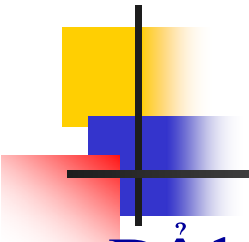
- Ôtômát hữu hạn không có bộ nhớ so với mô hình tổng quát.



Hoạt động của một dfa

■ Hoạt động của một dfa

- Tại thời điểm khởi đầu, nó được giả thiết ở trong trạng thái khởi đầu q_0 , với cơ cấu nhập (đầu đọc) của nó đang ở trên kí hiệu đầu tiên bên trái của chuỗi nhập.
- Trong suốt mỗi lần di chuyển, cơ cấu nhập tiến về phía phải một kí hiệu, như vậy mỗi lần di chuyển sẽ lấy một kí hiệu ngõ nhập.
- Khi gặp kí hiệu kết thúc chuỗi, chuỗi là **được chấp nhận** (accept) nếu ô tô-mát đang ở vào một trong các **trạng thái kết thúc** của nó. Ngược lại thì có nghĩa là chuỗi bị từ chối.



Đồ thị chuyển trạng thái

- Để biểu diễn một cách trực quan cho dfa người ta sử dụng đồ thị chuyển trạng thái. Cách biểu diễn như sau.
 - Các **đỉnh** biểu diễn các **trạng thái**.
 - Các **cạnh** biểu diễn các **chuyển trạng thái**.
 - Các **nhãn** trên các **đỉnh** là **tên các trạng thái**.
 - Các **nhãn** trên các **cạnh** là giá trị hiện tại của **kí hiệu nhập**.
 - **Trạng thái khởi đầu** sẽ được nhận biết bằng một **mũi tên đi vào** không mang nhãn mà không xuất phát từ bất kỳ đỉnh nào
 - Các **trạng thái kết thúc** được vẽ bằng một **vòng tròn đôi**.

Ví dụ

- Cho DFA sau

$$M = (Q, \Sigma, \delta, q_0, F)$$

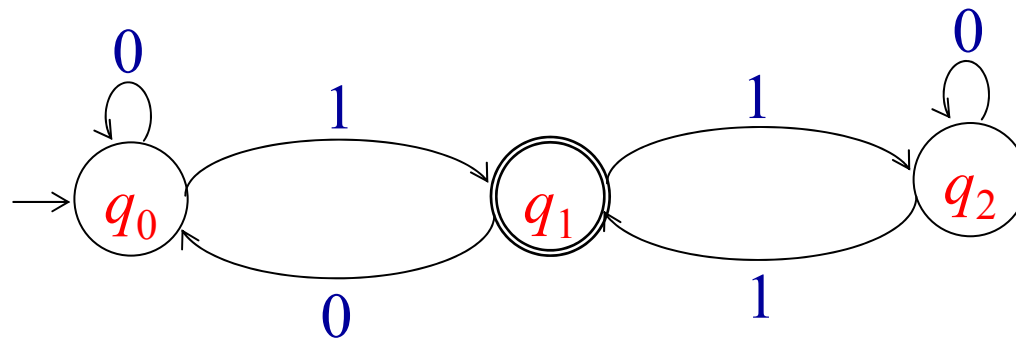
$Q = \{q_0, q_1, q_2\}$, $\Sigma = \{0, 1\}$, $F = \{q_1\}$, còn δ được cho bởi

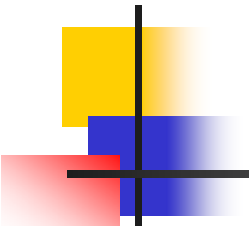
$$\delta(q_0, 0) = q_0, \quad \delta(q_0, 1) = q_1,$$

$$\delta(q_1, 0) = q_0, \quad \delta(q_1, 1) = q_2,$$

$$\delta(q_2, 0) = q_2, \quad \delta(q_2, 1) = q_1,$$

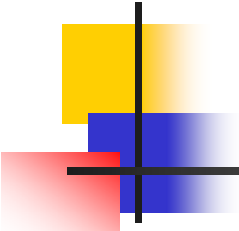
- Đồ thị chuyển trạng thái tương ứng là





Hàm chuyển trạng thái mở rộng

- Hàm chuyển trạng thái mở rộng δ^* được định nghĩa một cách đệ qui như sau
 - $\delta^*(q, \lambda) = q$,
 - $\delta^*(q, wa) = \delta(\delta^*(q, w), a), \forall q \in Q, w \in \Sigma^*, a \in \Sigma$.
- Ví dụ
 - Nếu $\delta(q_0, a) = q_1$, và $\delta(q_1, b) = q_2$,
 - Thì $\delta^*(q_0, ab) = q_2$
- Chú ý
 - δ không có định nghĩa cho chuyển trạng thái rỗng, tức là không định nghĩa cho $\delta(q, \lambda)$.



Ngôn ngữ và dfa

■ Định nghĩa 2.2

- Ngôn ngữ được chấp nhận bởi dfa $M = (Q, \Sigma, \delta, q_0, F)$ là tập tất cả các chuỗi trên Σ được chấp nhận bởi M .
- $L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \in F\}$.

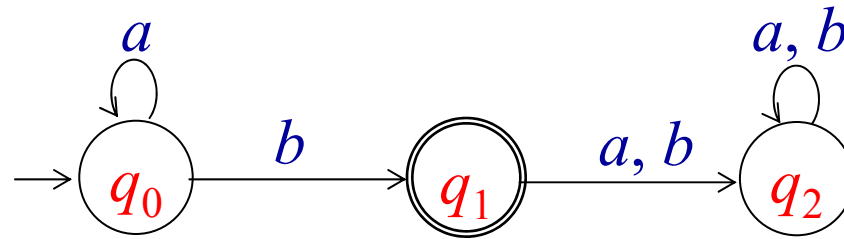
■ Nhận xét:

- $\overline{L(M)} = \{w \in \Sigma^* : \delta^*(q_0, w) \notin F\}$.

Ví dụ

■ Ví dụ

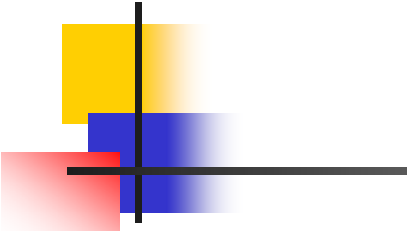
- Xét dfa M sau



- Dfa trên chấp nhận ngôn ngữ sau

$$L(M) = \{a^n b : n \geq 0\}$$

- **Trạng thái bẫy (trap state):** là trạng thái mà sau khi ô tô măt đi vào sẽ không bao giờ thoát ra được.
- Trạng thái bẫy có thể là trạng thái kết thúc hoặc không.
- Định nghĩa trên cũng có thể mở rộng ra cho nhóm các trạng thái bẫy kết thúc hay không kết thúc.



Định lý, bảng truyền

■ Định lý 2.1

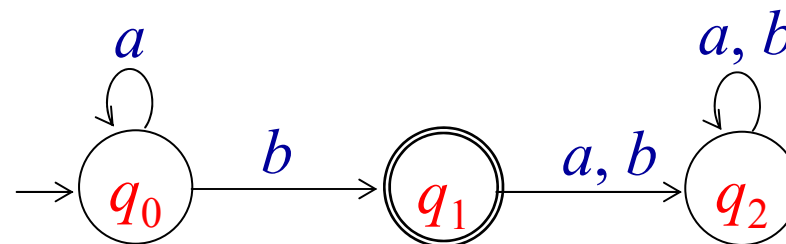
- Cho $M = (Q, \Sigma, \delta, q_0, F)$ là một accepter hữu hạn đơn định, và G_M là đồ thị chuyển trạng thái tương ứng của nó. Thì $\forall q_i, q_j \in Q$, và $w \in \Sigma^+$, $\delta^*(q_i, w) = q_j$ nếu và chỉ nếu có trong G_M một **con đường mang nhãn** là w đi từ q_i đến q_j .

■ Bảng truyền - (transition table)

- Là bảng trong đó các nhãn của hàng (ô tô đậm trên hàng trong hình bên) biểu diễn cho trạng thái hiện tại, còn nhãn của cột (ô tô đậm trên cột trong hình bên) biểu diễn cho ký hiệu nhập hiện tại. Các điểm nhập (entry) trong bảng định nghĩa cho trạng thái kế tiếp.

Bảng truyền (tt)

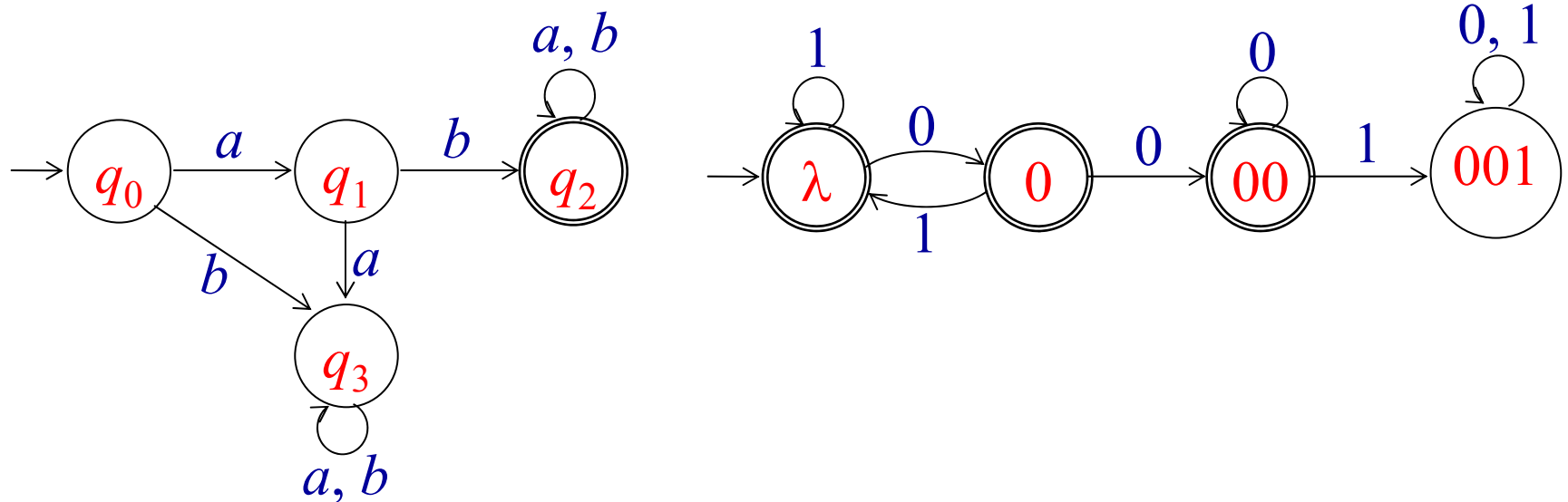
	a	b
q_0	q_0	q_1
q_1	q_2	q_2
q_2	q_2	q_2

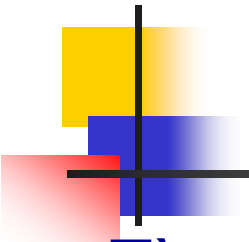


- Bảng truyền gợi ý cho chúng ta một cấu trúc dữ liệu để mô tả cho ô tô máy hữu hạn.
- Đồng thời cũng gợi ý cho chúng ta rằng một DFA có thể dễ dàng được hiện thực thành một chương trình máy tính; chẳng hạn bằng một dãy các phát biểu “if”.

Ví dụ

- Tìm dfa chấp nhận ngôn ngữ
 - Tìm dfa M_1 chấp nhận tập tất cả các chuỗi trên $\Sigma = \{a, b\}$ được bắt đầu bằng chuỗi ab .
 - Tìm dfa M_2 chấp nhận tập tất cả các chuỗi trên $\Sigma = \{0, 1\}$, ngoại trừ những chuỗi chứa chuỗi con 001.





Bài tập dfa

- Tìm dfa chấp nhận ngôn ngữ
 - $L_1 = \{vwv^R \in \{a, b\}^*: |v| = 2\}$
 - $L_2 = \{abab^n: n \geq 0\} \cup \{aba^n: n \geq 0\}$
 - $L_3 = \{a^n b^m : (n+m) \bmod 2 = 0\}$
 - $L_4 = \{w \in \{a, b\}^*: n_a(w) \text{ chẵn}, n_b(w) \text{ lẻ}\}$
 - $L_5 = \{w \in \{0, 1\}^*: \text{giá trị thập phân của } w \text{ chia hết cho } 5\}$
 - $L_6 = \{w \in \{a, b\}^*: \text{số kí tự } a \text{ trong chuỗi là một số lẻ}\}$

Ngôn ngữ chính qui

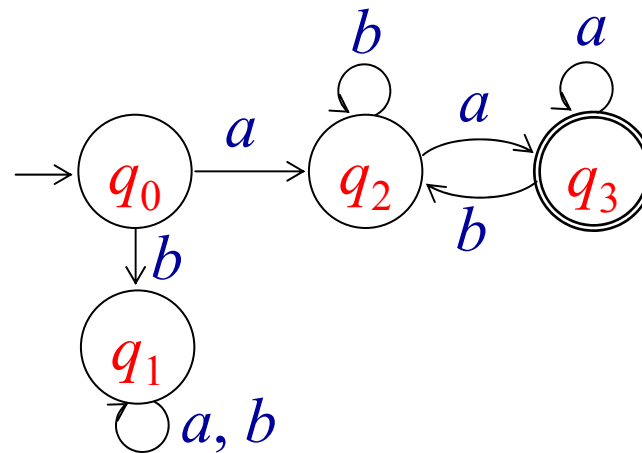
■ Định nghĩa 2.3

- Một ngôn ngữ L được gọi là chính qui nếu và chỉ nếu tồn tại một accepter hữu hạn đơn định M nào đó sao cho

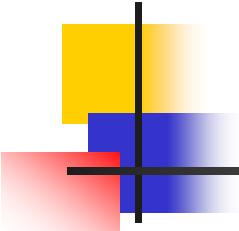
$$L = L(M)$$

■ Ví dụ

- Chứng minh rằng ngôn ngữ $L = \{awa : w \in \{a,b\}^*\}$ là chính qui.



Trang 60



Accepter hữu hạn không đơn định

■ Định nghĩa 2.4

- Một **accepter hữu hạn không đơn định** (nondeterministic finite state acceptor) hay **nfa** được định nghĩa bằng bộ năm:

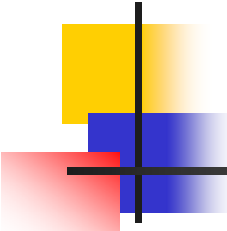
$$M = (Q, \Sigma, \delta, q_0, F)$$

trong đó Q, Σ, q_0, F được định nghĩa như đối với accepter hữu hạn đơn định còn δ được định nghĩa là:

$$\delta : Q \times (\Sigma \cup \{ \lambda \}) \rightarrow 2^Q$$

■ Nhận xét

- Có hai khác biệt chính giữa định nghĩa này và định nghĩa của một dfa.

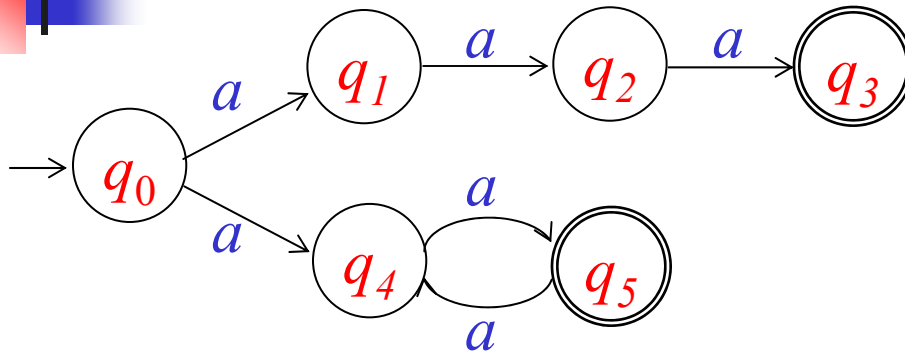


Acceptor hữu hạn không đơn định (tt)

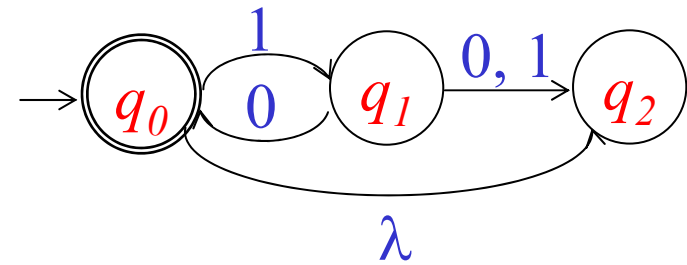
■ Nhận xét (tt)

- Đối với nfa miền trị của δ là tập 2^Q , vì vậy giá trị của nó không còn là một phần tử đơn của Q , mà là một tập con của nó và đặc biệt có thể là $\emptyset$, tức là có thể không có định nghĩa cho một $\delta(q, a)$ nào đó. Người ta gọi trường hợp này là một **cấu hình chết (dead configuration)**, và có thể hình dung trong trường hợp này ô tô máy dừng lại, không hoạt động nữa.
- Thứ hai định nghĩa này cho phép λ như là một đối số thứ hai của δ . Điều này có nghĩa là nfa có thể thực hiện một sự chuyển trạng thái mà không cần phải lấy vào một kí hiệu nhập nào.
- Tương tự như dfa, một nfa cũng có thể được biểu diễn bằng một ĐTCTT.

Ví dụ



(a)



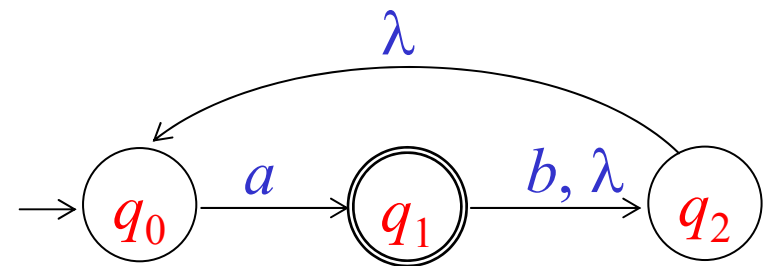
(b)

- Hàm chuyển trạng thái mở rộng
- Định nghĩa 2.5
 - Cho một nfa, hàm chuyển trạng thái mở rộng được định nghĩa sao cho $\delta^*(q_i, w)$ chứa q_j nếu và chỉ nếu có một con đường trong ĐTCTT đi từ q_i đến q_j mang nhãn w . Điều này đúng với mọi $q_i, q_j \in Q$ và $w \in \Sigma^*$.

Hàm chuyển trạng thái mở rộng

■ Ví dụ

- $\delta^*(q_1, \lambda) = \{q_1, q_2, q_0\}$
- $\delta^*(q_2, \lambda) = \{q_2, q_0\}$
- $\delta^*(q_0, a) = \{q_1, q_2, q_0\}$
- $\delta^*(q_1, a) = \{q_1, q_2, q_0\}$
- $\delta^*(q_1, b) = \{q_2, q_0\}$



Ngôn ngữ của nfa

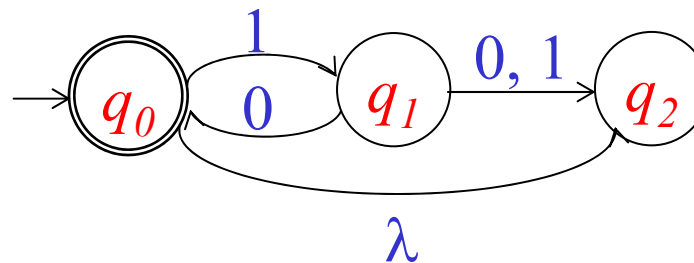
■ Định nghĩa 2.6

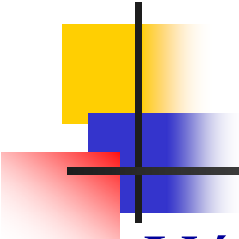
- Ngôn ngữ được chấp nhận bởi nfa $M = (Q, \Sigma, \delta, q_0, F)$, được định nghĩa như là một tập tất cả các chuỗi được chấp nhận bởi nfa trên. Một cách hình thức,

$$L(M) = \{w \in \Sigma^*: \delta^*(q_0, w) \cap F \neq \emptyset\}.$$

■ Ví dụ

- Ngôn ngữ được chấp nhận bởi ôôtômát bên dưới là $L = \{(10)^n: n \geq 0\}$





Cách tính δ^*

- Với T là một tập con của Q , ta định nghĩa

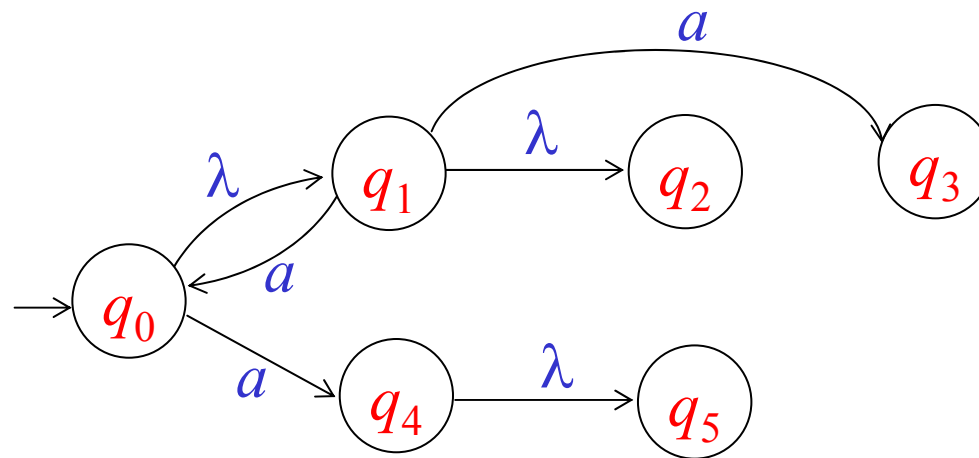
$$\delta(T, a) = \bigcup_{q \in T} \delta(q, a)$$

$$\delta^*(T, \lambda) = \bigcup_{q \in T} \delta^*(q, \lambda)$$

$$\delta^*(T, a) = \bigcup_{q \in T} \delta^*(q, a)$$

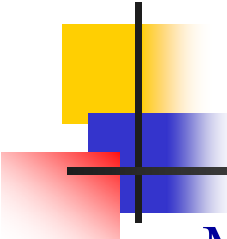
- Người ta thường hiện thực cách tính các hàm này $\delta(q, a)$, $\delta(T, a)$, $\delta^*(q, \lambda)$, $\delta^*(T, \lambda)$ lần lượt bằng các hàm **move**(q, a), **move**(T, a), **λ -closure**(q), **λ -closure**(T) (**λ -closure** đọc là bao đóng- λ)
- $\delta^*(q, a) = \lambda\text{-closure}(\text{move}(\lambda\text{-closure}(q), a))$
- $\delta^*(T, a) = \lambda\text{-closure}(\text{move}(\lambda\text{-closure}(T))$

Ví dụ



	a	λ
q_0	q_4	q_1
q_1	q_0, q_3	q_2
q_2		
q_3		
q_4		q_5
q_5		

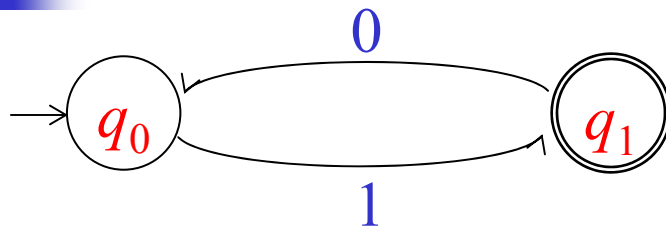
- Hãy tính $\delta^*(q_0, a)$.
- $\delta^*(q_0, a) = \lambda\text{-closure}(\text{move}(\lambda\text{-closure}(q_0), a))$
- $\lambda\text{-closure}(q_0) = \{q_0, q_1, q_2\}$
- $\text{move}(\{q_0, q_1, q_2\}, a) = \{q_4, q_0, q_3\}$
- $\lambda\text{-closure}(\{q_4, q_0, q_3\}) = \{q_4, q_0, q_3, q_5, q_1, q_2\}$
- Vậy $\delta^*(q_0, a) = \{q_0, q_1, q_2, q_3, q_4, q_5\}$



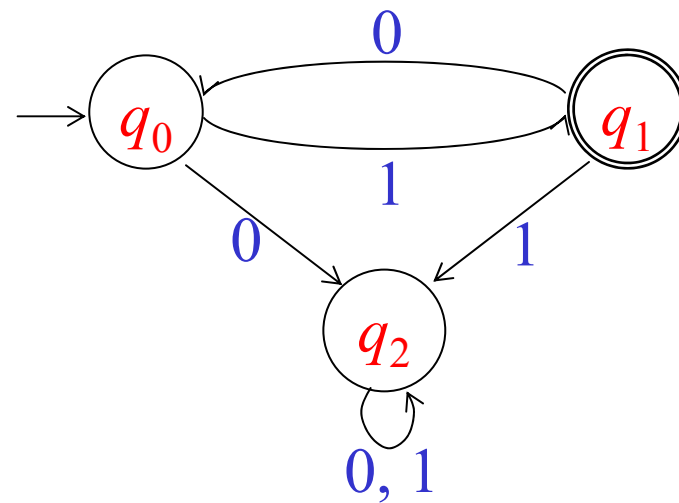
Một định nghĩa khác về dfa - dfa mở rộng

- Một dfa là một trường hợp đặc biệt của một nfa trong đó
 - Không có chuyển trạng thái-rỗng,
 - Đối với mỗi trạng thái q và một kí hiệu nhập a , có tối đa một cạnh chuyển trạng thái đi ra khỏi q và có nhãn là a .
- Về bản chất định nghĩa này và định nghĩa trước đây là tương đương nhau (cùng định nghĩa tính đơn định của dfa). Nó chỉ khác định nghĩa thứ nhất ở chỗ cho phép khả năng không có một sự chuyển trạng thái đối với một cặp trạng thái và kí hiệu nhập. Trong trường hợp này thì ta xem như nó rơi vào một trạng thái bẫy không kết thúc mà trạng thái này không được vẽ ra.

Ví dụ

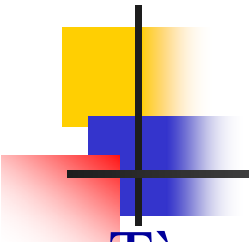


(a)



(b)

- Dfa trong hình (a) đơn giản hơn dfa trong hình (b) mặc dù chúng cùng chấp nhận một ngôn ngữ như nhau.
- Vậy dfa mở rộng và dfa đầy đủ theo định nghĩa ban đầu thật sự là tương đương nhau và chúng chỉ **khác nhau ở một trạng thái bất kỳ không kết thúc**.

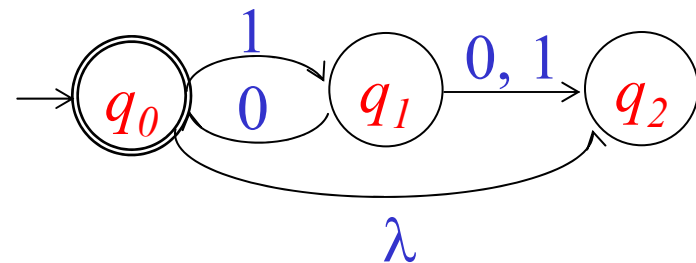
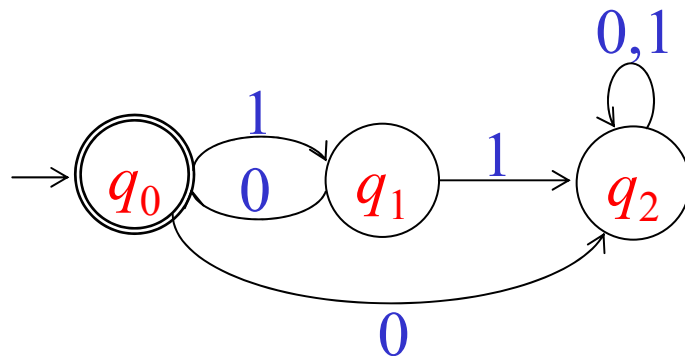


Bài tập nfa

- Tìm nfa chấp nhận ngôn ngữ
 - $L_1 = \{\text{tập tất cả các số thực của Pascal}\}$
 - Một “run” trong một chuỗi là một chuỗi con có chiều dài tối thiểu 2 kí tự, dài nhất có thể và bao gồm toàn các kí tự giống nhau. Chẳng hạn, chuỗi *abbbaabba* chứa một “run” của *b* có chiều dài 3, một “run” của *a* có chiều dài 2 và một “run” của *b* có chiều dài 2. Tìm các nfa và dfa cho mỗi ngôn ngữ sau trên $\{a, b\}$.
 - $L_2 = \{w: w \text{ không chứa “run” nào có chiều dài nhỏ hơn 3}\}$
 - $L_3 = \{w: \text{mỗi “run” của } a \text{ có chiều dài hoặc 2 hoặc 3}\}$
 - $L_4 = \{w \in \{0, 1\}^*: \text{mỗi chuỗi con bốn kí hiệu có tối đa hai kí hiệu 0}\}$.

Sự tương đương giữa nfa và dfa

- Sự tương đương giữa hai ô tô măt
 - Hai accepter được gọi là tương đương nhau nếu chúng cùng chấp nhận một ngôn ngữ như nhau.
- Ví dụ
 - Dfa và nfa sau là tương đương nhau vì cùng chấp nhận ngôn ngữ $\{(10)^n: n \geq 0\}$



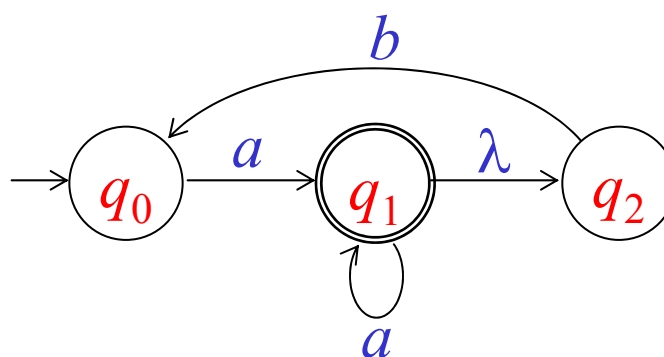
Sự tương đương giữa nfa và dfa (tt)

■ Nhận xét

- Dfa về bản chất là một loại giới hạn của nfa, nên lớp các dfa là một lớp con của lớp nfa. Nhưng nó có phải là một lớp con thực sự hay không? Rất hay là người ta đã chứng minh được rằng hai lớp này là tương đương nhau, tức là với một nfa thì sẽ có một dfa tương đương với nó.

■ Ví dụ

- Hãy xây dựng dfa tương đương với nfa bên.



	a	b	λ
q_0	q_1		
q_1	q_1		q_2
q_2		q_0	

Ví dụ

	a	b	λ
q_0	q_1		
q_1	q_1		q_2
q_2		q_0	

- Xây dựng dfa bằng cách **mô phỏng** lại quá trình chấp nhận một chuỗi bất kỳ của nfa

- $\delta^*(q_0, \lambda) = \{q_0\}$

- $\delta^*(\{q_0\}, a) = \{q_1, q_2\}$

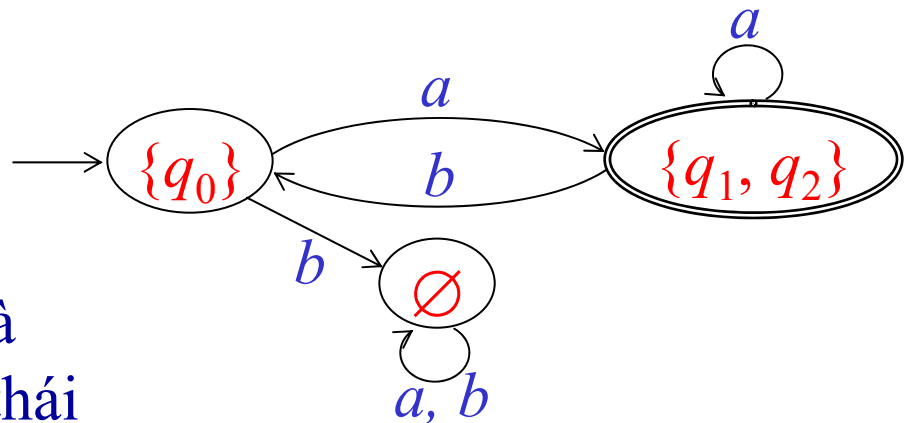
$$\delta^*(\{q_0\}, b) = \emptyset$$

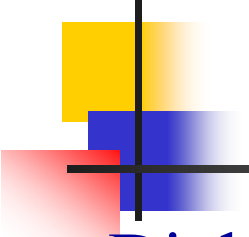
- $\delta^*(\{q_1, q_2\}, a) = \{q_1, q_2\}$

$$\delta^*(\{q_1, q_2\}, b) = \{q_0\}$$

■ Chú ý

- Một trạng thái của nfa là một tập trạng thái của dfa
- Trạng thái kết thúc của nfa là trạng thái mà có chứa trạng thái kết thúc của dfa.





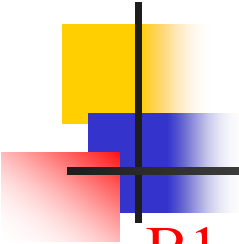
Định lý về sự tương đương

■ Định lý 2.2

- Cho L là ngôn ngữ được chấp nhận bởi một accepter hữu hạn không đơn định $M_N = (Q_N, \Sigma, \delta_N, q_0, F_N)$, thì tồn tại một accepter hữu hạn đơn định $M_D = (Q_D, \Sigma, \delta_D, \{q_0\}, F_D)$ sao cho
$$L = L(M_D).$$

■ Thủ tục: nfa_to_dfa

- Input: nfa $M_N = (Q_N, \Sigma, \delta_N, q_0, F_N)$
- Output: ĐTCTT G_D của dfa M_D

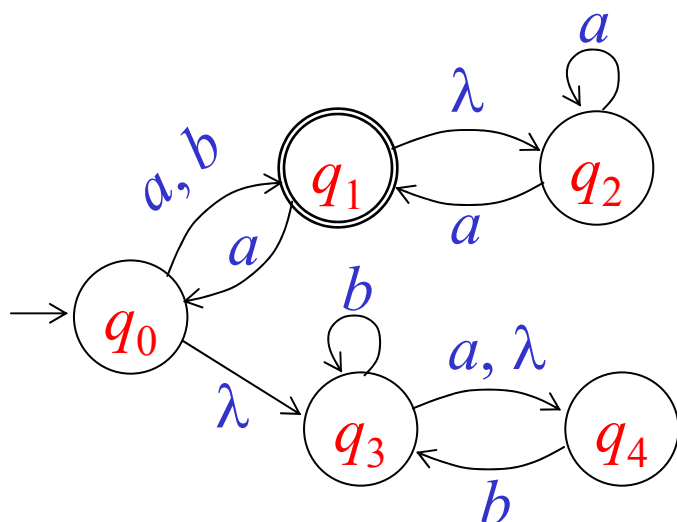


Thủ tục: **nfa_to_dfa**

- B1.** Tạo một đồ thị G_D với đỉnh khởi đầu là tập $\delta_N^*(q_0, \lambda)$.
- B2.** Lặp lại các bước B3 đến B6 cho đến khi không còn cạnh nào thiếu.
- B3.** Lấy một đỉnh bất kỳ $\{q_i, q_j, \dots, q_k\}$ của G_D mà có một cạnh còn chưa được định nghĩa đối với một a nào đó $\in \Sigma$.
- B4.** Tính $\delta_N^*(\{q_i, q_j, \dots, q_k\}, a) = \{q_l, q_m, \dots, q_n\}$.
- B5.** Tạo một đỉnh cho G_D có nhãn $\{q_l, q_m, \dots, q_n\}$ nếu nó chưa tồn tại.
- B6.** Thêm vào G_D một cạnh từ $\{q_i, q_j, \dots, q_k\}$ đến $\{q_l, q_m, \dots, q_n\}$ và gán nhãn cho nó bằng a .
- B7.** Mỗi trạng thái của G_D mà nhãn của nó chứa một q_f bất kỳ $\in F_N$ thì được coi là một đỉnh kết thúc.

Ví dụ

- Hãy biến đổi nfa dưới (có bảng truyền tương ứng bên cạnh) thành dfa tương đương.



	a	b	λ
q_0	q_1	q_1	q_3
q_1	q_0		q_2
q_2	q_1, q_2		
q_3	q_4	q_3	q_4
q_4		q_3	

Ví dụ (tt)

- $\delta^*(q_0, \lambda) = \{q_0, q_3, q_4\}$
- $\delta^*(\{q_0, q_3, q_4\}, a) = \{q_1, q_2, q_4\}$
- $\delta^*(\{q_0, q_3, q_4\}, b) = \{q_1, q_2, q_3, q_4\}$
- $\delta^*(\{q_1, q_2, q_4\}, a) = \{q_0, q_1, q_2, q_3, q_4\}$
- $\delta^*(\{q_1, q_2, q_4\}, b) = \{q_3, q_4\}$
- $\delta^*(\{q_1, q_2, q_3, q_4\}, a) = \{q_0, q_1, q_2, q_3, q_4\}$
- $\delta^*(\{q_1, q_2, q_3, q_4\}, b) = \{q_3, q_4\}$
- $\delta^*(\{q_0, q_1, q_2, q_3, q_4\}, a) = \{q_0, q_1, q_2, q_3, q_4\}$
- $\delta^*(\{q_0, q_1, q_2, q_3, q_4\}, b) = \{q_1, q_2, q_3, q_4\}$
- $\delta^*(\{q_3, q_4\}, a) = \{q_4\}$
- $\delta^*(\{q_4\}, a) = \emptyset$

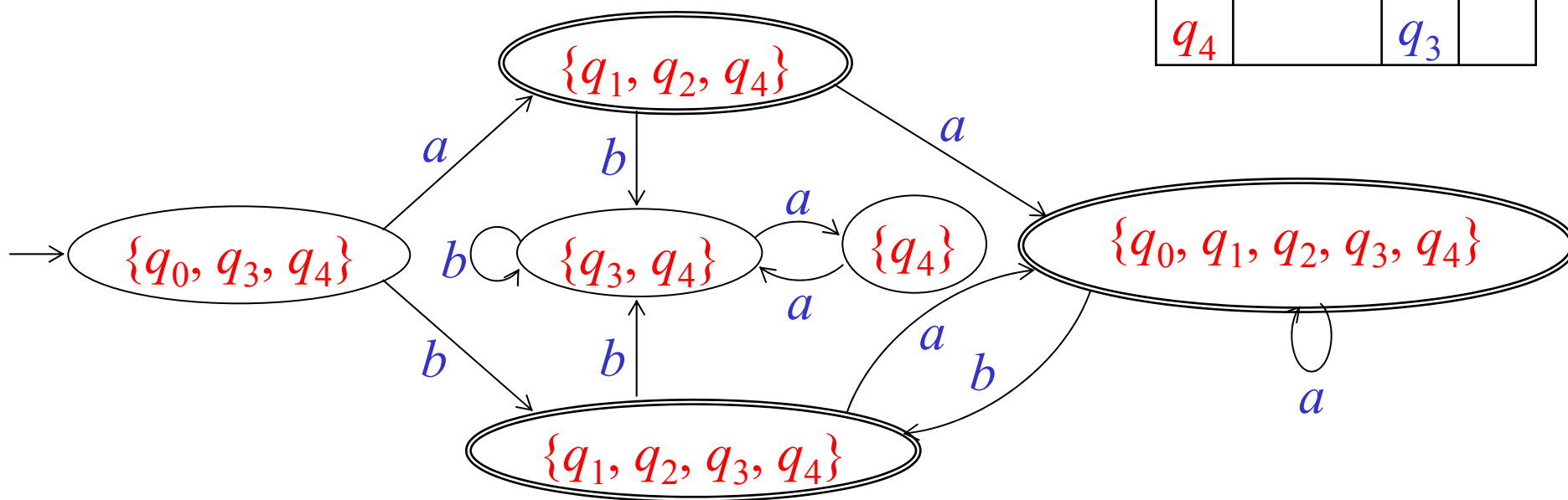
	a	b	λ
q_0	q_1	q_1	q_3
q_1	q_0		q_2
q_2	q_1, q_2		
q_3	q_4	q_3	q_4
q_4		q_3	

$$\delta^*(\{q_3, q_4\}, b) = \{q_3, q_4\}$$

$$\delta^*(\{q_4\}, b) = \{q_3, q_4\}$$

Ví dụ (tt)

	a	b	λ
q_0	q_1	q_1	q_3
q_1	q_0		q_2
q_2	q_1, q_2		
q_3	q_4	q_3	q_4
q_4		q_3	



Bài tập biến đổi nfa thành dfa

- Biến đổi những nfa sau thành dfa tương đương

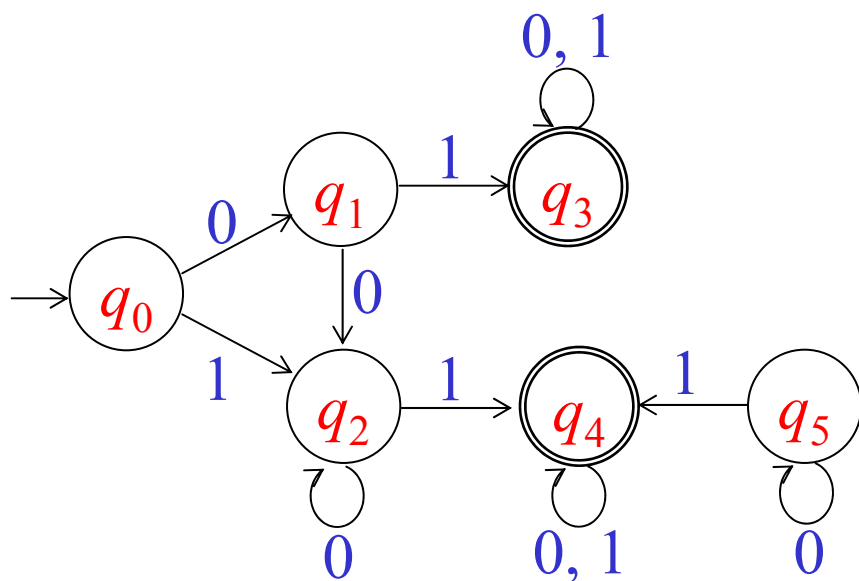
Nfa M_1			
	a	b	λ
q_0	q_1	q_3	q_1
q_1	q_2	q_2, q_0	
q_2		q_1	
q_3	q_0, q_4	q_3	q_4
q_4	q_3, q_4	q_4	
$F = \{q_2\}$			

Nfa M_2			
	a	b	λ
q_0	q_1, q_3	q_3	q_3
q_1	q_2	q_2	q_0
q_2	q_1		
q_3	q_4		q_4
q_4	q_4	q_3	
$F = \{q_4\}$			

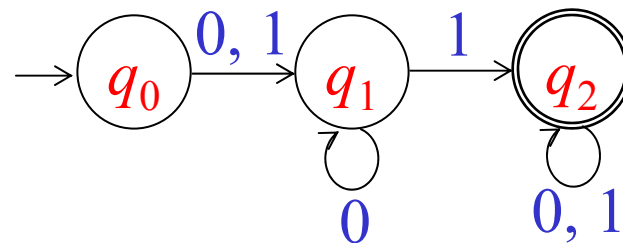
Nfa M_3			
	a	b	λ
q_0	q_1	q_2	q_1
q_1	q_1, q_2	q_3	q_3
q_2		q_0, q_2	
q_3	q_2, q_3		
$F = \{q_0\}$			

Rút gọn số trạng thái của một dfa

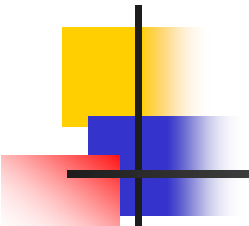
- Hai dfa được vẽ trong (a) và (b) là tương đương nhau



(a)



(b)



Rút gọn số trạng thái của một dfa (tt)

■ Nhận xét

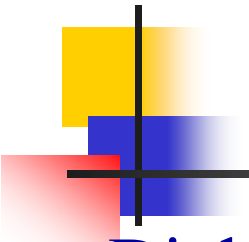
- Trong hình (a) có một trạng thái đặc biệt, trạng thái q_5 , nó là trạng thái không đạt tới được từ trạng thái khởi đầu, người ta gọi nó là trạng thái không đạt tới được.
- Trạng thái không đạt tới được (inaccessible state)
 - Là trạng thái mà không tồn tại con đường đi từ trạng thái khởi đầu đến nó.
 - Những trạng thái không đạt tới được (TTKĐTĐ) có thể bỏ đi (kèm với các cạnh chuyển trạng thái liên quan tới nó) mà không làm ảnh hưởng tới ngôn ngữ được chấp nhận bởi ô tô máy.



Rút gọn số trạng thái của một dfa (tt)

■ Nhận xét (tt)

- Các chuyển trạng thái từ sau đỉnh q_1 và q_2 "có vẻ giống nhau", đối xứng nhau và ô tô măt thứ hai "có vẻ như" kết hợp hai phần này.
- Từ đây dẫn tới định nghĩa hai trạng thái giống nhau hay không phân biệt được.
- Khái niệm *giống nhau* được định nghĩa tổng quát dựa trên việc: *với mọi chuỗi nếu xuất phát từ hai trạng thái này thì kết quả về mặt chấp nhận chuỗi là giống nhau tức là hoặc cùng rơi vào trạng thái kết thúc, hoặc không cùng rơi vào trạng thái kết thúc.*
- Như vậy hai trạng thái này có thể gom chung lại với nhau mà kết quả chấp nhận chuỗi không thay đổi.



Định nghĩa hai trạng thái giống nhau

■ Định nghĩa 2.7

- Hai trạng thái p và q của một dfa được gọi là **không phân biệt được** (indistinguishable) hay **giống nhau** nếu với mọi $w \in \Sigma^*$

$\delta^*(q, w) \in F$ suy ra $\delta^*(p, w) \in F$, và

$\delta^*(q, w) \notin F$ suy ra $\delta^*(p, w) \notin F$,

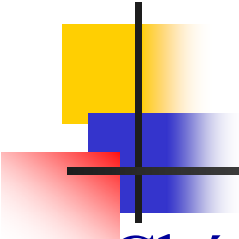
Còn nếu tồn tại một chuỗi w nào đó $\in \Sigma^*$ sao cho

$\delta^*(q, w) \in F$ còn $\delta^*(p, w) \notin F$,

hay ngược lại thì p và q được gọi là **phân biệt được** (distinguishable) hay **khác nhau** bởi chuỗi w .

■ Nhận xét

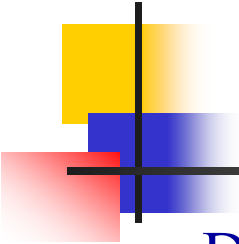
- Trạng thái kết thúc và không kết thúc không thể giống nhau.



Nhận xét (tt)

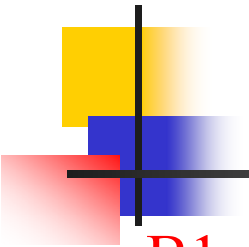
■ Chú ý

- Quan hệ giống nhau là một *quan hệ tương đương*.
- Vì vậy quan hệ này sẽ phân hoạch tập trạng thái Q thành các tập con rời nhau, mỗi tập con bao gồm các trạng thái giống nhau.



Thủ tục đánh dấu - mark

- Để xác định các cặp trạng thái không phân biệt được, người ta thực hiện công việc ngược lại là xác định các cặp trạng thái không giống nhau
- Để làm điều này người ta sử dụng thủ tục **mark** (đánh dấu) bên dưới.
- Thủ tục: **mark**
 - **Input**: Các cặp trạng thái, gồm $(|Q| \times (|Q| - 1)/2)$ cặp, của DFA đầy đủ.
 - **Output**: Các cặp trạng thái được đánh dấu phân biệt được.



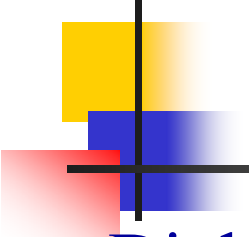
Thủ tục đánh dấu - mark

B1. Loại bỏ tất cả các TTKĐTĐ.

B2. Xét tất cả các cặp trạng thái (p, q) . Nếu $p \in F$ và $q \notin F$ hay ngược lại, đánh dấu cặp (p, q) là phân biệt được. Các cặp trạng thái được đánh dấu ở bước này sẽ được ghi là đánh dấu ở bước số **0** (gọi là bước cơ bản).

Lặp lại bước B3 cho đến khi không còn cặp nào không được đánh dấu trước đó được đánh dấu ở bước này.

B3. Đối với mọi cặp (p, q) chưa được đánh dấu và mọi $a \in \Sigma$, tính $\delta(p, a) = p_a$ và $\delta(q, a) = q_a$. Nếu cặp (p_a, q_a) đã được đánh dấu là phân biệt được ở lần lặp trước đó, thì đánh dấu (p, q) là phân biệt được. Các cặp được đánh dấu ở bước này sẽ được ghi là được đánh dấu ở bước thứ ***i*** nếu đây là lần thứ ***i*** băng qua vòng lặp.



Thủ tục đánh dấu – mark (tt)

■ Định lý 2.3

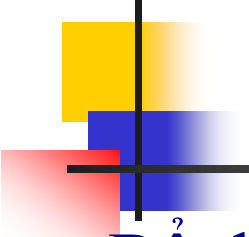
- Thủ tục **mark**, áp dụng cho một DFA đầy đủ bất kỳ $M = (Q, \Sigma, \delta, q_0, F)$, kết thúc và xác định tất cả các trạng thái phân biệt được.

■ Bổ đề 1

- Cặp trạng thái q_i và q_j là phân biệt được bằng chuỗi có độ dài n , nếu và chỉ nếu có các chuyển trạng thái

$$\delta(q_i, a) = q_k \text{ và } \delta(q_j, a) = q_l$$

với một a nào đó $\in \Sigma$, và q_k và q_l là cặp trạng thái phân biệt được bằng chuỗi có độ dài $n-1$.



Thủ tục đánh dấu – mark (tt)

■ Bổ đề 2

- Khi băng qua vòng lặp trong bước n , thủ tục sẽ đánh dấu được thêm **tất cả** các cặp trạng thái phân biệt được bằng chuỗi có độ dài n mà chưa được đánh dấu.

■ Bổ đề 3

- Nếu thủ tục dừng lại sau n lần băng qua vòng lặp trong bước 3, thì không có cặp trạng thái nào của dfa mà phân biệt được bằng chuỗi có chiều dài lớn hơn n .



Thủ tục rút gọn - reduce

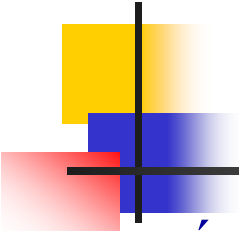
- Thủ tục: **reduce**

- Input: dfa $M = (Q, \Sigma, \delta, q_0, F)$

- Output: dfa tối giản $\hat{M} = \left(\hat{Q}, \hat{\Sigma}, \hat{\delta}, \hat{q}_0, \hat{F} \right)$

B1. Sử dụng thủ tục *mark* để tìm tất cả các cặp trạng thái phân biệt được. Từ đây tìm ra các tập của tất cả các trạng thái không phân biệt được, gọi là $\{q_i, q_j, \dots, q_k\}, \{q_l, q_m, \dots, q_n\}, \dots$

B2. Đối với mỗi tập $\{q_i, q_j, \dots, q_k\}$ các trạng thái không phân biệt được, tạo ra một trạng thái có nhãn $ij \dots k$ cho $\hat{M}$.

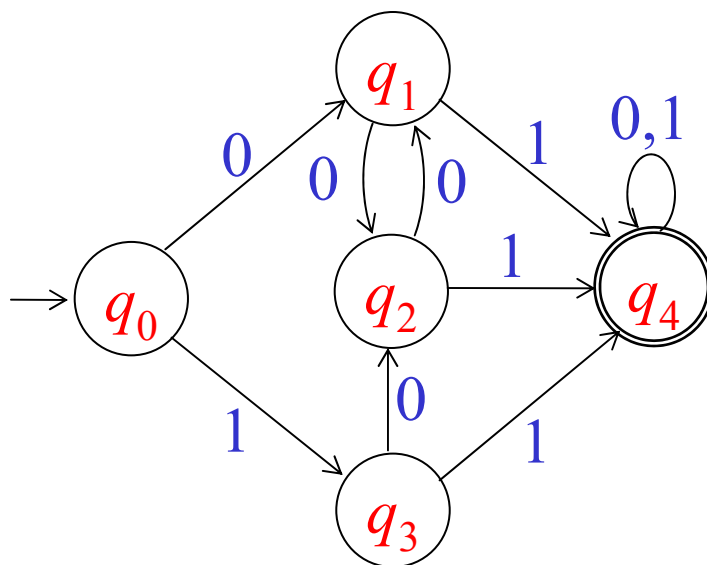


Thủ tục rút gọn - reduce

- B3.** Đối với mỗi quy tắc chuyển trạng thái của M có dạng $\delta(q_r, a) = q_p$, tìm các tập mà q_r và q_p thuộc về. Nếu $q_r \in \{q_i, q_j, \dots, q_k\}$ và $q_p \in \{q_l, q_m, \dots, q_n\}$, thì thêm vào $\hat{\delta}$ quy tắc $\hat{\delta}(ij \dots k, a) = lm \dots n$.
- B4.** Trạng thái khởi đầu q_0 là trạng thái của mà nhãn của nó có chứa 0.
- B5.** $\hat{F}$ là tập tất cả các trạng thái mà nhãn của nó chứa i sao cho $q_i \in F$.

Ví dụ

- Hãy rút gọn trạng thái của dfa sau (cho kèm bảng truyền tương ứng bên cạnh).

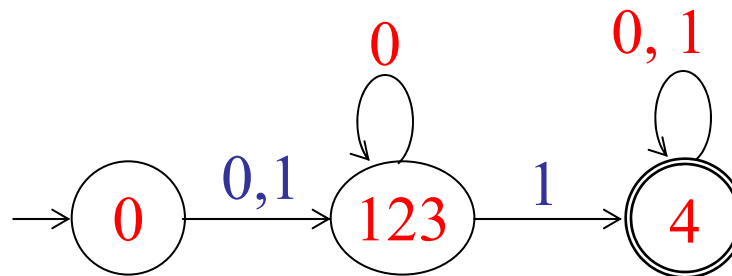
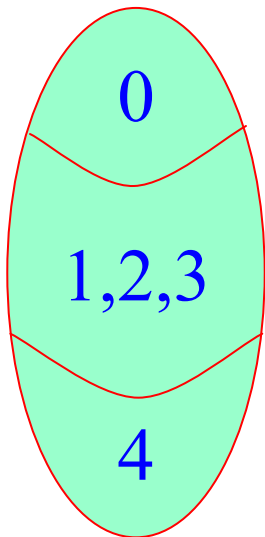


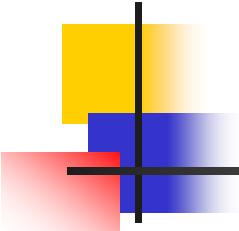
	0	1
q_0	q_1	q_3
q_1	q_2	q_4
q_2	q_1	q_4
q_3	q_2	q_4
q_4	q_4	q_4

Ví dụ (tt)

$(q_0, q_1) \checkmark 1$	$(q_0, q_3) \checkmark 1$	(q_1, q_2)	$(q_1, q_4) \checkmark 0$	$(q_2, q_4) \checkmark 0$
$(q_0, q_2) \checkmark 1$	$(q_0, q_4) \checkmark 0$	(q_1, q_3)	(q_2, q_3)	$(q_3, q_4) \checkmark 0$

	0	1
q_0	q_1	q_3
q_1	q_2	q_4
q_2	q_1	q_4
q_3	q_2	q_4
q_4	q_4	q_4





Định lý

■ Định lý 2.4

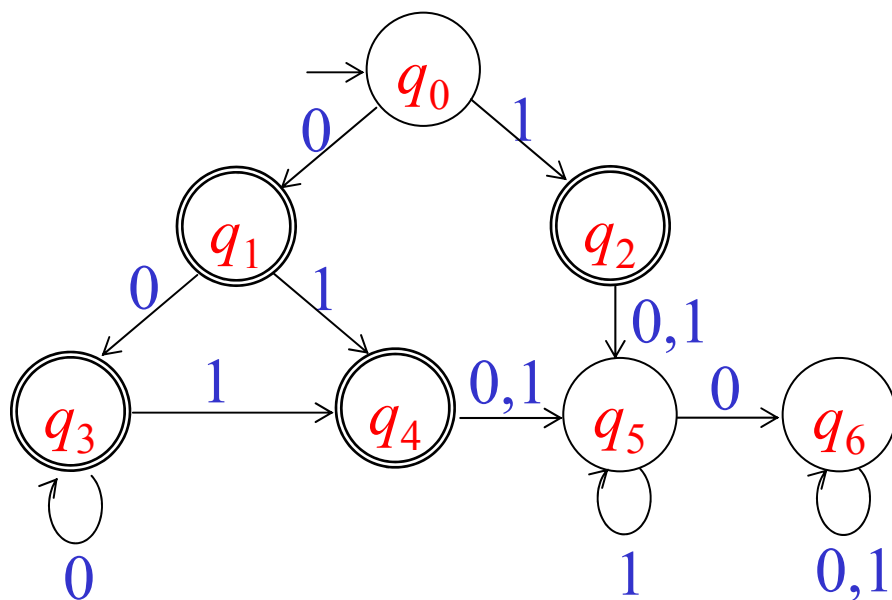
- Cho một dfa M bất kỳ, áp dụng thủ tục **reduce** tạo ra một dfa khác $\hat{M}$ sao cho

$$L(M) = L(\hat{M})$$

Hơn nữa là tối giản theo nghĩa không có một dfa nào khác có số trạng thái nhỏ hơn mà cũng chấp nhận $L(M)$.

Ví dụ

- Hãy rút gọn trạng thái của dfa sau (cho kèm bảng truyền tương ứng bên cạnh).

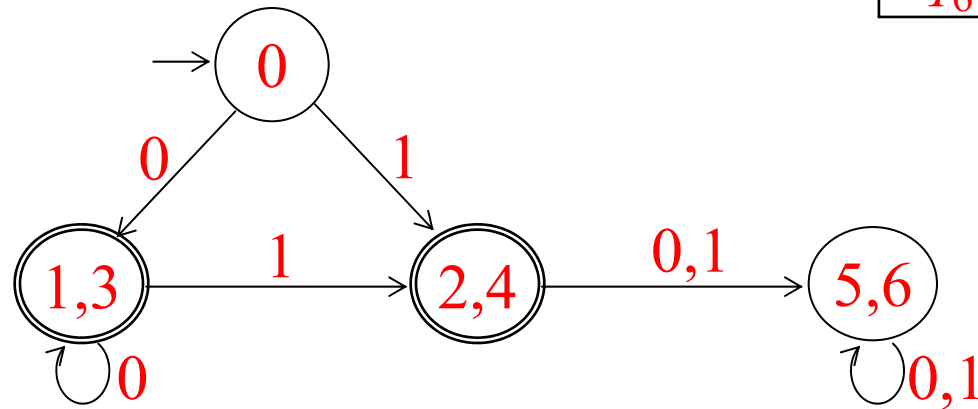
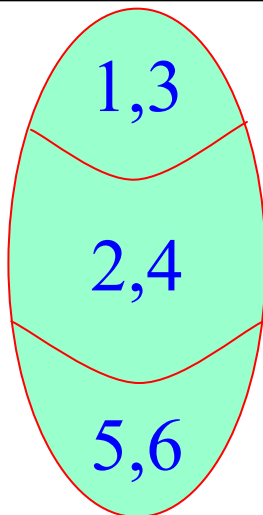


	0	1
q_0	q_1	q_2
q_1	q_3	q_4
q_2	q_5	q_5
q_3	q_3	q_4
q_4	q_5	q_5
q_5	q_6	q_5
q_6	q_6	q_6

Ví dụ (tt)

$(q_0, q_1) \checkmark 0$	$(q_1, q_2) \checkmark 0$	(q_2, q_4)	$(q_4, q_5) \checkmark 0$
$(q_0, q_2) \checkmark 0$	(q_1, q_3)	$(q_2, q_5) \checkmark 0$	$(q_4, q_6) \checkmark 0$
$(q_0, q_3) \checkmark 0$	$(q_1, q_4) \checkmark 1$	$(q_2, q_6) \checkmark 0$	(q_5, q_6)
$(q_0, q_4) \checkmark 0$	$(q_1, q_5) \checkmark 0$	$(q_3, q_4) \checkmark 1$	
$(q_0, q_5) \checkmark 1$	$(q_1, q_6) \checkmark 0$	$(q_3, q_5) \checkmark 0$	
$(q_0, q_6) \checkmark 1$	$(q_2, q_3) \checkmark 1$	$(q_3, q_6) \checkmark 0$	

	0	1
q_0	q_1	q_2
q_1	q_3	q_4
q_2	q_5	q_5
q_3	q_3	q_4
q_4	q_5	q_5
q_5	q_6	q_5
q_6	q_6	q_6



Bài tập rút gọn dfa

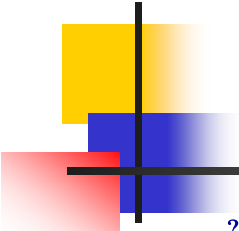
Rút gọn những dfa sau thành dfa tối giản

Dfa M_1		
	a	b
q_0	q_1	q_4
q_1	q_4	q_2
q_2	q_4	q_3
q_3	q_3	
q_4	q_4	q_5
q_5	q_4	q_6
q_6	q_7	q_6
q_7		q_7

Dfa M_2		
	a	b
q_0	q_1	q_2
q_1	q_2	q_3
q_2	q_2	q_3
q_3	q_5	q_4
q_4	q_5	q_3
q_5	q_5	q_5
q_6	q_1	q_7
q_7	q_6	q_4

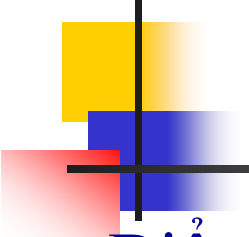
Dfa M_3		
	a	b
q_0	q_1	q_2
q_1	q_2	q_3
q_2	q_1	q_4
q_3	q_4	q_0
q_4	q_3	q_0

Dfa M_4		
	a	b
q_0	q_1	q_3
q_1	q_2	q_4
q_2	q_0	q_3
q_3	q_1	q_4
q_4	q_2	q_3



Chương 3 Ngôn ngữ chính qui và văn phạm chính qui

- 3.1 Biểu thức chính qui (Regular Expression)
- 3.2 Mối quan hệ giữa BTCQ và ngôn ngữ chính qui
- 3.3 Văn phạm chính qui (Regular Grammar)



Biểu thức chính qui

- Biểu thức chính qui (BTCQ) là gì?
 - Là một sự kết hợp các chuỗi kí hiệu của một bảng chữ cái Σ nào đó, các dấu ngoặc, và các phép toán $+$, $.$, và $*$. trong đó phép $+$ biểu thị cho phép hội, phép $.$ biểu thị cho phép kết nối, phép $*$ biểu thị cho phép bao đóng sao.
- Ví dụ
 - Ngôn ngữ $\{a\}$ được biểu thị bởi BTCQ a .
 - Ngôn ngữ $\{a, b, c\}$ được biểu thị bởi BTCQ $a + b + c$.
 - Ngược lại BTCQ $(a + b.c)^*$ biểu thị cho ngôn ngữ $\{\lambda, a, bc, aa, abc, bca, bcbc, aaa, aabc, \dots\}$.



Định nghĩa hình thức BTCQ

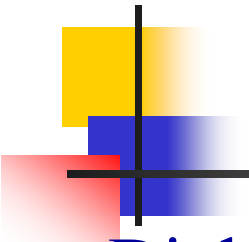
■ Định nghĩa 3.1

■ Cho Σ là một bảng chữ cái, thì

1. $\emptyset$, λ , và $a \in \Sigma$ tất cả đều là những BTCQ hơn nữa chúng được gọi là những **BTCQ nguyên thủy**.
2. Nếu r_1 và r_2 là những BTCQ, thì $r_1 + r_2$, $r_1 \cdot r_2$, r_1^* , và (r_1) cũng vậy.
3. Một chuỗi là một BTCQ nếu và chỉ nếu nó có thể được dẫn xuất từ các BTCQ nguyên thủy bằng một số lần hữu hạn áp dụng các quy tắc trong (2).

■ Ví dụ

■ Cho $\Sigma = \{a, b, c\}$, thì chuỗi $(a + b.c)^*.(c + \emptyset)$ là BTCQ, vì nó được xây dựng bằng cách áp dụng các qui tắc ở trên. Còn $(a + b +)$ không phải là BTCQ.



Ngôn ngữ tương ứng với BTCQ

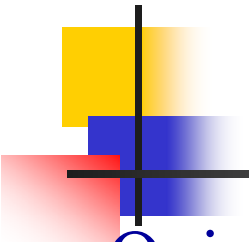
■ Định nghĩa 3.2

- Ngôn ngữ $L(r)$ được biểu thị bởi BTCQ bất kỳ là được định nghĩa bởi các qui tắc sau.

1. $\emptyset$ là BTCQ biểu thị tập trống,
2. λ là BTCQ biểu thị $\{\lambda\}$,
3. Đối với mọi $a \in \Sigma$, a là BTCQ biểu thị $\{a\}$,

Nếu r_1 và r_2 là những BTCQ, thì

4. $L(r_1 + r_2) = L(r_1) \cup L(r_2)$,
5. $L(r_1.r_2) = L(r_1).L(r_2)$,
6. $L((r_1)) = L(r_1)$,
7. $L(r_1^*) = (L(r_1))^*$.



Ngôn ngữ tương ứng với BTCQ (tt)

- Quy định về độ ưu tiên

- Độ ưu tiên của các phép toán theo thứ tự từ cao đến thấp là

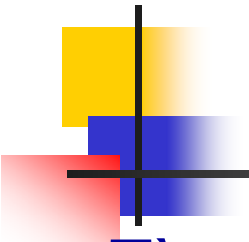
1. bao đóng – sao,

2. kết nối,

3. hội.

- Ví dụ

- $$\begin{aligned} L(a^* \cdot (a + b)) &= L(a^*) L(a + b) \\ &= (L(a))^* (L(a) \cup L(b)) \\ &= \{\lambda, a, aa, aaa, \dots\} \{a, b\} \\ &= \{a, aa, aaa, \dots, b, ab, aab, \dots\} \end{aligned}$$



Xác định ngôn ngữ cho BTCQ

- Tìm ngôn ngữ của các BTCQ sau

- $r_1 = (aa)^*(bb)^*b$

- $r_2 = (ab^*a + b)^*$

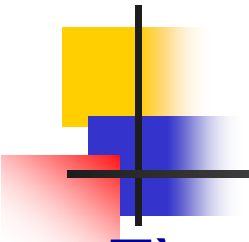
- $r_3 = a(a + b)^*$

- Kết quả

- $L(r_1) = \{a^{2n}b^{2m+1} : n \geq 0, m \geq 0\}$

- $L(r_2) = \{w \in \{a, b\}^* : n_a(w) \text{ chẵn}\}$

- $L(r_3) = \{w \in \{a, b\}^* : w \text{ được bắt đầu bằng } a\}$



Tìm BTCQ cho ngôn ngữ

■ Tìm BTCQ cho các ngôn ngữ sau

- $L_1 = \{\text{tập tất cả các số thực của Pascal}\}$
- $L_2 = \{w \in \{0, 1\}^*: w \text{ không có một cặp số } 0 \text{ liên tiếp nào}\}$
- $L_3 = \{w \in \{0, 1\}^*: n_0(w) = n_1(w)\}$

■ Kết quả

- $r_1 = ('+' + '-' + \lambda)(0 + 1 + \dots + 9)^+ ('.' (0 + 1 + \dots + 9)^+ + \lambda)$
 $(\text{'E'} ('+' + '-' + \lambda)(0 + 1 + \dots + 9)^+ + \lambda)$
- $r_2 = [(1^* 011^*)^* + 1^*] (0 + \lambda) \text{ hoặc } (1 + 01)^* (0 + \lambda)$
- Không tồn tại BTCQ biểu diễn cho L_3



Một số phép toán mở rộng

- Phép chọn lựa $r?$ hoặc $[r]$

$$r ? = [r] = (r + \lambda)$$

- Phép bao đóng dương $^+$

$$r^+ = r.r^*$$

■ Chú ý

- $(r^*)^* = r^*$
- $(r_1^* + r_2)^* = (r_1 + r_2)^*$
- $(r_1 r_2^* + r_2)^* = (r_1 + r_2)^*$
- Trong một số tài liệu phép cộng (+) được kí hiệu bằng dấu | thay cho dấu +. Chẳng hạn $(a + b).c$ thì được viết là $(a | b).c$

BTCQ biểu thị NNCQ

■ Định lý 3.1

- Cho r là một BTCQ, thì tồn tại một nfa mà chấp nhận $L(r)$. Vì vậy, $L(r)$ là NNCQ.

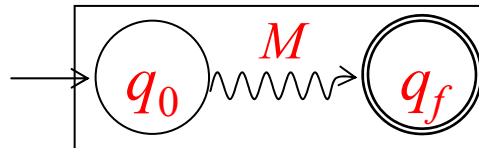
■ Bổ đề

- Với mọi nfa có nhiều hơn một trạng thái kết thúc luôn luôn có một nfa tương đương với chỉ một trạng thái kết thúc.



Thủ tục: re-to-nfa

- Từ bỏ đề trên mọi nfa có thể được biểu diễn bằng sơ đồ như sau

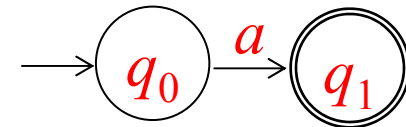
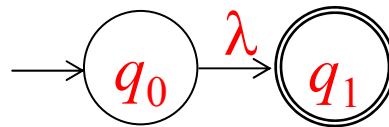


- Chứng minh

- Thủ tục: re-to-nfa

- Input: Biểu thức chính qui r .
- Output: nfa $M = (Q, \Sigma, \delta, q_0, F)$.

B1. Xây dựng các nfa cho các BTCQ nguyên thủy

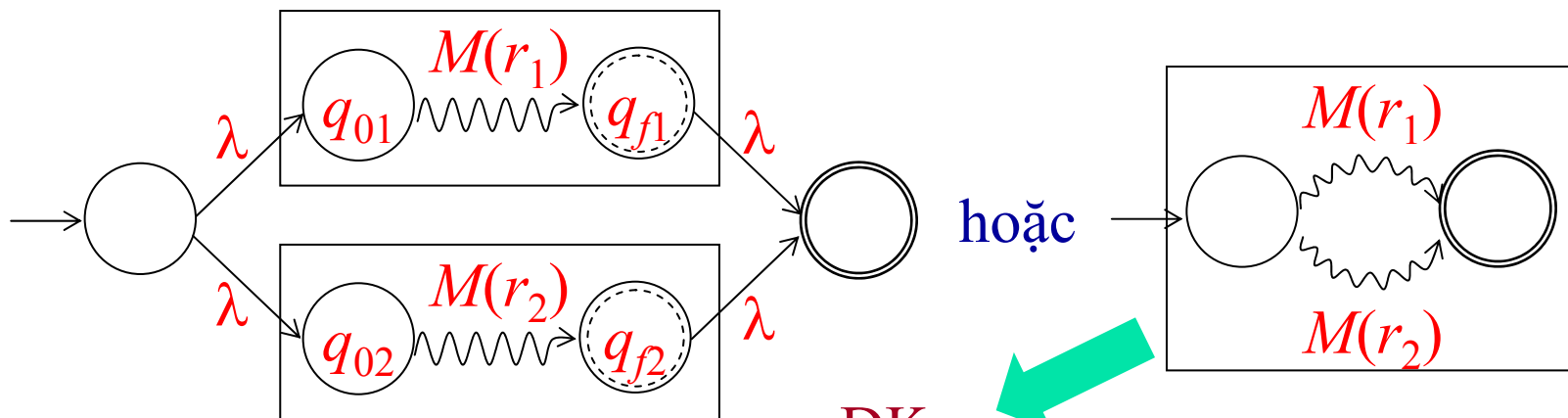


(a) nfa chấp nhận $\emptyset$ (b) nfa chấp nhận $\{\lambda\}$ (c) nfa chấp nhận $\{a\}$

Thủ tục: re-to-nfa (tt)

B2. Xây dựng các nfa cho các BTCQ phức tạp

■ nfa cho BTCQ $r_1 + r_2$

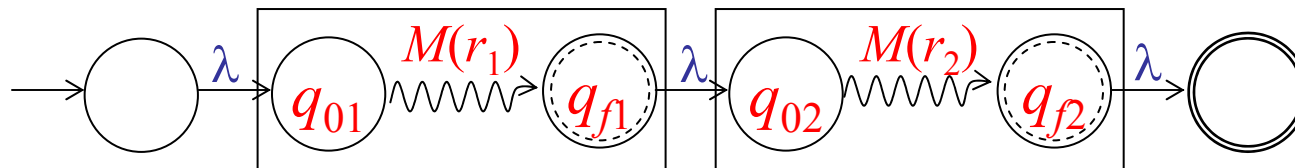


ĐK:

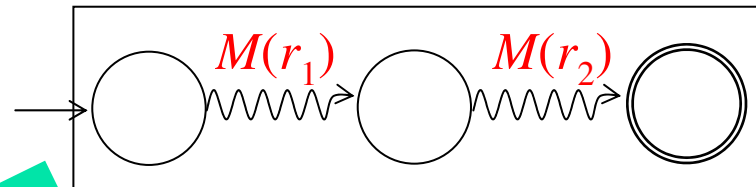
1. Không có cạnh đi vào q_{01} và q_{02}
2. Không có cạnh đi ra q_{f1} và q_{f2}

Thủ tục: re-to-nfa (tt)

- nfa cho BTCQ $r_1 r_2$



hoặc

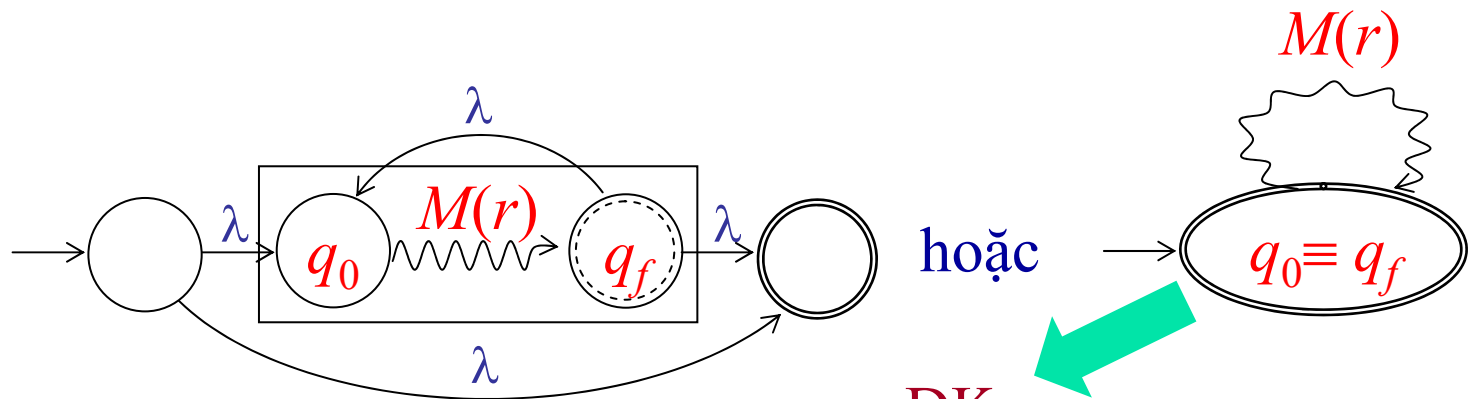


ĐK:

1. Không có cạnh đi ra q_{f1} hoặc
2. Không có cạnh đi vào q_{02}

Thủ tục: re-to-nfa (tt)

- nfa cho BTCQ r^*



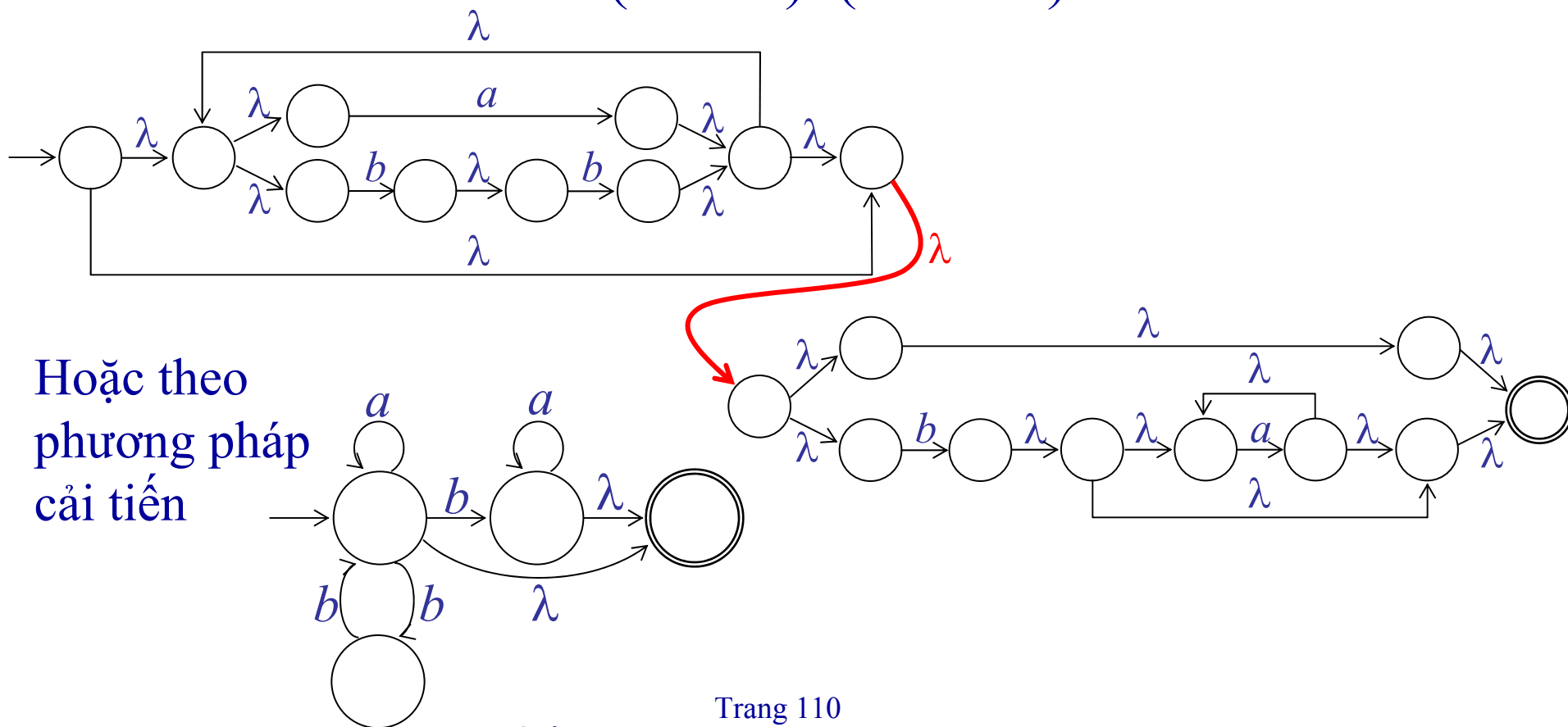
ĐK:

1. Không có cạnh đi vào q_0
2. Không có cạnh đi ra q_f

Ví dụ

- Xây dựng nfa cho BTCQ sau

$$r = (a + bb)^*(ba^* + \lambda)$$

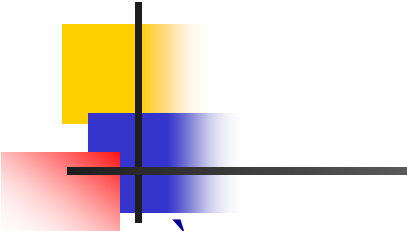




Bài tập BTCQ

■ Xây dựng nfa cho các BTCQ sau

- $r_1 = aa^* + aba^*b^*$
- $r_2 = ab(a + ab)^* (b + aa)$
- $r_3 = ab^*aa + bba^*ab$
- $r_4 = a^*b(ab + b)^*a^*$
- $r_5 = (ab^* + a^*b)(a + b^*a)^* b$
- $r_6 = (b + a^*)(ba^* + ab)^*(b^*a + ab)$

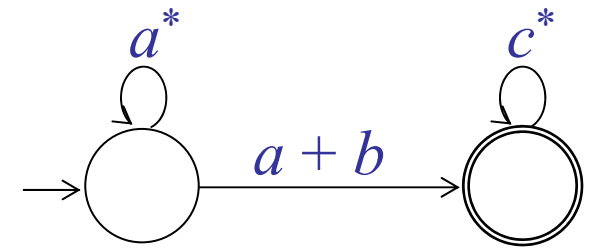


BTCQ cho NNCQ

- Đồ thị chuyển trạng thái tổng quát (generalized transition graphs):
 - Là một ĐTCTT ngoại trừ các cạnh của nó được gán nhãn bằng các BTCQ.
 - Ngôn ngữ được chấp nhận bởi nó là tập tất cả các chuỗi được sinh ra bởi các BTCQ mà là nhãn của một con đường nào đó đi từ trạng thái khởi đầu đến một trạng thái kết thúc nào đó của ĐTCTT tổng quát (ĐTCTTTQ).

Đồ thị chuyển trạng thái tổng quát

- Hình bên biểu diễn một ĐTCTTTQ.



- NN được chấp nhận bởi nó là $L(a^*(a + b)c^*)$
- Nhận xét
 - ĐTCTT của một nfa bất kỳ có thể được xem là ĐTTCTTTQ nếu các nhãn cạnh được diễn dịch như sau.
 - Một cạnh được gán nhãn là một kí hiệu đơn a được diễn dịch thành cạnh được gán nhãn là biểu thức a .
 - Một cạnh được gán nhãn với nhiều kí hiệu $a, b, \dots$ thì được diễn dịch thành cạnh được gán nhãn là biểu thức $a + b + \dots$.
 - Mọi NNCQ đều $\exists$ một ĐTCTTTQ chấp nhận nó. Ngược lại, mỗi NN mà được chấp nhận bởi một ĐTCTTTQ là chính qui.

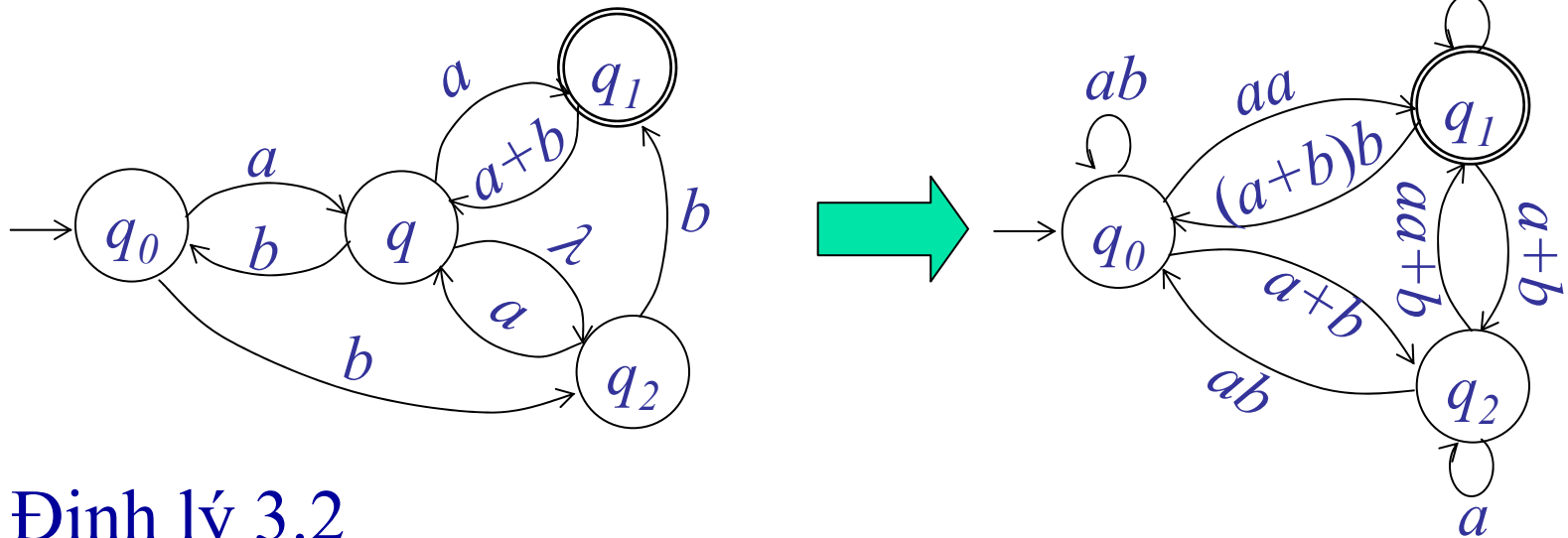
Rút gọn trạng thái của ĐTCTTTQ

- Để tìm BTCQ cho một ĐTCTTTQ ta sẽ thực hiện quá trình rút gọn các trạng thái trung gian của nó thành ĐTCTTTQ tương đương đơn giản nhất có thể được.
- Trạng thái trung gian
 - Là trạng thái mà không phải là trạng thái khởi đầu, cũng không phải là trạng thái kết thúc.



Định lý

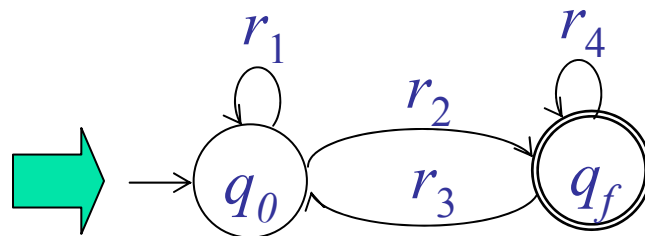
- Rút gọn trạng thái q của ĐTCTT sau



Định lý 3.2

- Cho L là một NNCQ, thì tồn tại một BTCQ r sao cho $L = L(r)$.

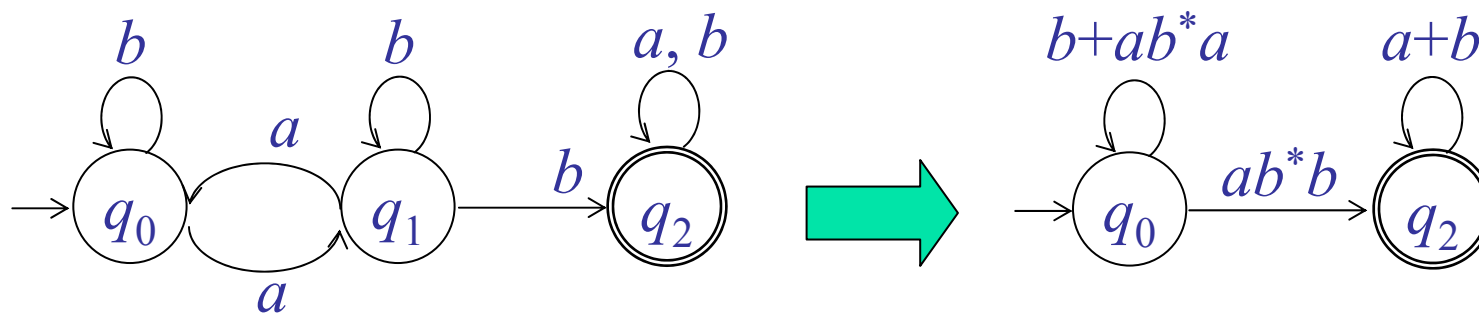
Đồ thị chuyển trạng thái



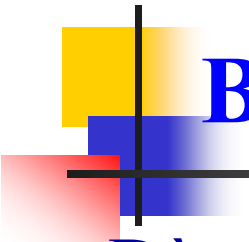
$$r = r_1 * r_2 (r_4 + r_3 r_1 * r_2)^*$$

Ví dụ

- Xác định BTCQ cho nfa sau

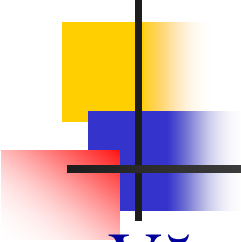


$$r = (b + ab^*a)^* ab^*b(a + b)^*$$



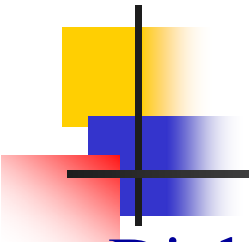
BTCQ dùng để mô tả các mẫu đơn giản

- Dùng trong các ngôn ngữ lập trình
 - BTCQ được dùng để mô tả các token chẳng hạn như
 - Danh hiệu
 - Số nguyên thực
 - ...
- Dùng trong các trình soạn thảo văn bản, các ứng dụng xử lý chuỗi
 - BTCQ được dùng để mô tả các mẫu tìm kiếm, thay thế ...
 - `del tmp*.*??`



Văn phạm chính qui

- Văn phạm tuyến tính - phải và – trái.
- Văn phạm tuyến tính - phải sinh ra NNCQ.
- Văn phạm tuyến tính - phải cho NNCQ.
- Sự tương đương giữa VPCQ và NNCQ.



Văn phạm tuyến tính - phải và - trái

■ Định nghĩa 3.3

- Một văn phạm $G = (V, T, S, P)$ được gọi là tuyến tính - phải (TT-P) (right-linear) nếu tất cả luật sinh là có dạng

$$A \rightarrow xB$$

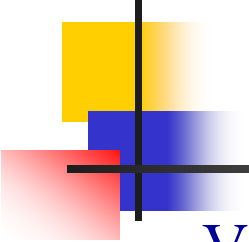
$$A \rightarrow x$$

trong đó $A, B \in V, x \in T^*$. Một văn phạm được gọi là tuyến tính - trái (TT-T) (left-linear) nếu tất cả các luật sinh là có dạng

$$A \rightarrow Bx$$

$$A \rightarrow x$$

- Một văn phạm chính qui (VPCQ) là hoặc tuyến tính-phải hoặc tuyến tính-trái.



Ví dụ

- VP $G_1 = (\{S\}, \{a, b\}, S, P_1)$, với P_1 được cho như sau là TT- P
$$S \rightarrow abS \mid a$$

- VP $G_2 = (\{S, S_1, S_2\}, \{a, b\}, S, P_2)$, với P_2 như sau là TT- T
$$S \rightarrow S_1ab,$$

$$S_1 \rightarrow S_1ab \mid S_2,$$

$$S_2 \rightarrow a,$$

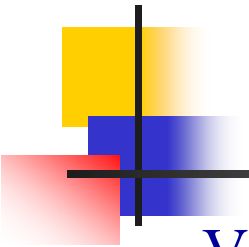
- Dãy

$$S \Rightarrow abS \Rightarrow ababS \Rightarrow ababa$$

là một dẫn xuất trong G_1 .

$$L(G_1) = L((ab)^*a)$$

$$L(G_2) = L(a(ab)^*ab)$$



Văn phạm tuyến tính

- VP $G = (\{S, A, B\}, \{a, b\}, S, P)$, với các luật sinh

$$S \rightarrow A,$$

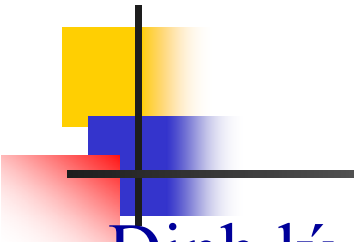
$$A \rightarrow aB \mid \lambda,$$

$$B \rightarrow Ab,$$

không phải là một VPCQ. Đây là một ví dụ của văn phạm tuyến tính (VPTT).

- Văn phạm tuyến tính (Linear Grammar)

- Một văn phạm được gọi là tuyến tính nếu mọi luật sinh của nó có dạng có tối đa một biến xuất hiện ở vế phải của luật sinh và không có sự giới hạn nào trên vị trí xuất hiện của biến này.



Văn phạm TT-P sinh ra NNCQ

■ Định lý 3.3

- Cho $G = (V, T, S, P)$ là một VPTT-P. Thì $L(G)$ là NNCQ.

■ Chứng minh

■ Thủ tục: G_P to nfa

- Input: Văn phạm tuyến tính-phải $G_P = (V, T, S, P)$
- Output: nfa $M = (Q, \Sigma, \delta, q_0, F)$

B1. Ứng với mỗi biến V_i của văn phạm ta xây dựng một trạng thái mang nhãn V_i cho nfa tức là: $Q \supset V$.

B2. Ứng với biến khởi đầu V_0 , trạng thái V_0 của nfa sẽ trở thành trạng thái khởi đầu, tức là: $S = V_0$

B3. Nếu trong văn phạm có một luật sinh nào đó dạng $V_i \rightarrow a_1 a_2 \dots a_m$ thì thêm vào nfa một và chỉ một trạng thái kết thúc V_f .

Văn phạm TT-P sinh ra NNCQ (tt)

B4. Ứng với mỗi luật sinh của văn phạm có dạng

$$V_i \rightarrow a_1 a_2 \dots a_m V_j$$

thêm vào nfa các chuyển trạng thái

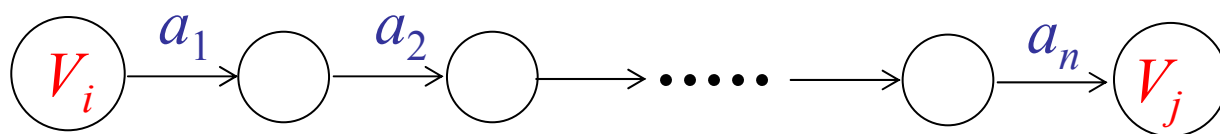
$$\delta^*(V_i, a_1 a_2 \dots a_m) = V_j$$

B5. Ứng với mỗi luật sinh dạng

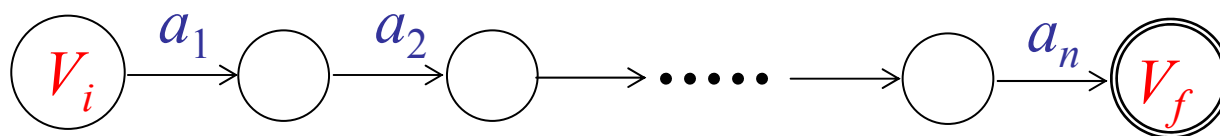
$$V_i \rightarrow a_1 a_2 \dots a_m$$

thêm vào nfa các chuyển trạng thái

$$\delta^*(V_i, a_1 a_2 \dots a_m) = V_f$$



Biểu diễn
 $V_i \rightarrow a_1 a_2 \dots a_m V_j$



Biểu diễn
 $V_i \rightarrow a_1 a_2 \dots a_m$

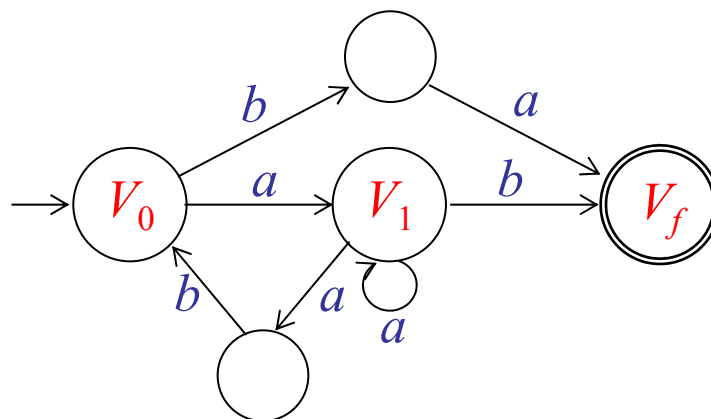
Ví dụ

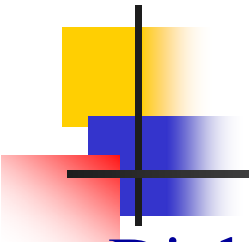
- Xây dựng một nfa chấp nhận ngôn ngữ của văn phạm sau:

$$V_0 \rightarrow aV_1 \mid ba$$

$$V_1 \rightarrow aV_1 \mid abV_0 \mid b$$

- Nfa kết quả





Văn phạm TT-P cho NNCQ

■ Định lý 3.4

- Nếu L là một NNCQ trên bảng chữ cái Σ , thì $\exists$ một VPTT-P $G = (V, \Sigma, S, P)$ sao cho $L = L(G)$.

■ Chứng minh

■ Thủ tục: nfa to G_P

- **Input**: nfa $M = (Q, \Sigma, \delta, q_0, F)$
- **Output**: Văn phạm tuyến tính-phải $G_P = (V, \Sigma, S, P)$
- Giả thiết $Q = \{q_0, q_1, \dots, q_n\}$ và $\Sigma = \{a_1, a_2, \dots, a_m\}$.

B1. $V = Q$, $S = q_0$ (tức là mỗi trạng thái trong nfa trở thành một biến trong văn phạm)

Thủ tục: nfa to G_P

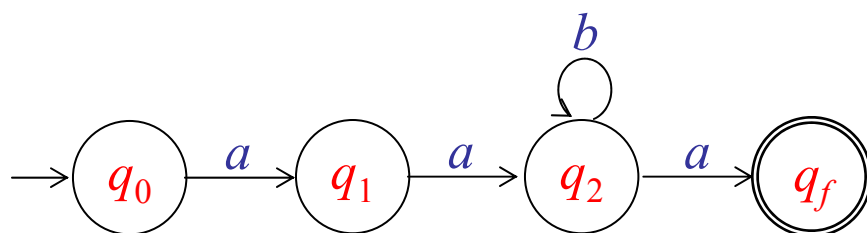
B2. Với mỗi chuyển trạng thái $\delta(q_i, a_j) = q_k$ của M ta xây dựng luật sinh TT-P tương ứng

$$q_i \rightarrow a_j q_k.$$

B3. Đối với mỗi trạng thái $q_f \in F$ chúng ta xây dựng luật sinh $q_f \rightarrow \lambda$.

■ Ví dụ

- Xây dựng VPTT- P cho ngôn ngữ $L(aab^*a)$.

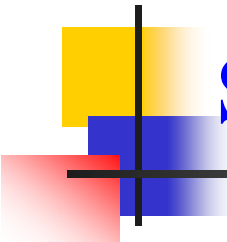


$$\begin{aligned} G_P: \quad & q_0 \rightarrow aq_1 \\ & q_1 \rightarrow aq_2 \\ & q_2 \rightarrow aq_f \mid bq_2 \\ & q_f \rightarrow \lambda \end{aligned}$$



Sự tương đương giữa VPCQ và NNCQ

- Nhận xét
 - Lớp VPTT-P tương đương với lớp NNCQ
- Định lý 3.5
 - Ngôn ngữ L là chính qui nếu và chỉ nếu tồn tại một VPTT-T G sao cho $L = L(G)$.
 - Ta chứng minh mối quan hệ tương đương thông qua VPTT-P.
- Bổ đề 1
 - Từ VPTT-T G_T đã cho ta xây dựng VPTT-P G_P tương ứng như sau



Sự tương đương giữa VPCQ và NNCQ

1. Ứng với luật sinh TT- $TA \rightarrow Bv$ ta xây dựng luật sinh TT- $PA \rightarrow v^RB$.
2. Ứng với luật sinh TT- $TA \rightarrow v$ ta xây dựng luật sinh TT- $PA \rightarrow v^R$.

- G_P được xây dựng theo cách trên có quan hệ với G_T như sau

$$L(G_T) = L(G_P)^R$$

- Bổ đề 2

- Nếu L là chính qui thì L^R cũng chính qui.

- Nhận xét

- Lớp VPTT-T tương đương với lớp NNCQ

- Định lý 3.6

- Một ngôn ngữ L là chính qui nếu và chỉ nếu tồn tại một VPCQ G sao cho $L = L(G)$.

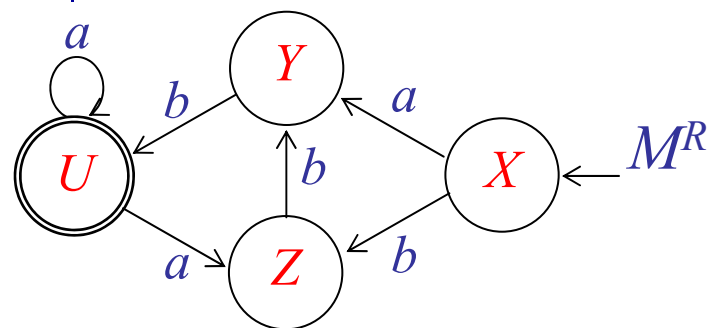
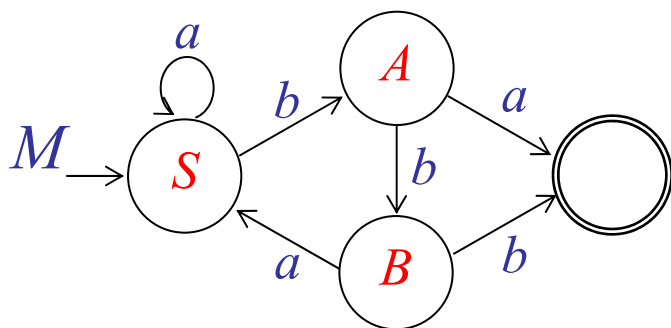
Ví dụ

- Xây dựng nfa M , VPTT-T G_T tương đương với VPTT-P G_P sau

$$S \rightarrow aS \mid bA$$

$$A \rightarrow bB \mid a$$

$$B \rightarrow aS \mid b$$



$$\begin{aligned} G_P^R \quad & X \rightarrow aY \mid bZ \\ & Y \rightarrow bU \\ & Z \rightarrow bY \\ & U \rightarrow aU \mid aZ \mid \lambda \end{aligned}$$

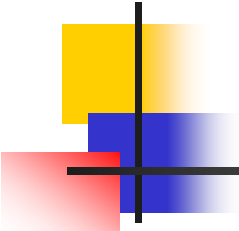


$$\begin{aligned} G_T \quad & X \rightarrow Ya \mid Zb \\ & Y \rightarrow Ub \\ & Z \rightarrow Yb \\ & U \rightarrow Ua \mid Za \mid \lambda \end{aligned}$$



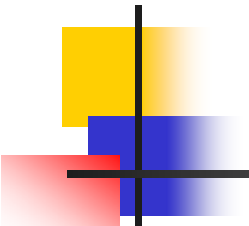
Chương 4 Các tính chất của ngôn ngữ chính qui

- NNCQ tổng quát là như thế nào? Có phải chẳng mọi ngôn ngữ hình thức đều là chính qui?
- Khi chúng ta thực hiện các phép toán trên NNCQ thì kết quả sẽ như thế nào, có còn là một NNCQ không?
- Một ngôn ngữ nào đó có hữu hạn không? Có rỗng không?
- Làm thế nào để biết một ngôn ngữ đã cho có là chính qui không?



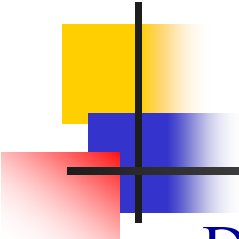
Chương 4 Các tính chất của ngôn ngữ chính qui

- 4.1 Tính đóng của ngôn ngữ chính qui.
- 4.2 Các câu hỏi cơ bản về ngôn ngữ chính qui..
- 4.3 Nhận biết các ngôn ngữ không chính qui



Tính đóng của NNCQ

- Đóng dưới các phép toán tập hợp đơn giản.
- Định lý 4.1
 - Nếu L_1 và L_2 là các NNCQ, thì $L_1 \cup L_2$, $L_1 \cap L_2$, $L_1 L_2$, $\bar{L}$ và L_1^* cũng vậy. Chúng ta nói rằng họ NNCQ là đóng dưới các phép hội, giao, kết nối, bù và bao đóng-sao.
- Chứng minh
 - Nếu L_1, L_2 là chính qui thì $\exists$ các BTCQ r_1, r_2 sao cho $L_1 = L(r_1)$, $L_2 = L(r_2)$. Theo định nghĩa $r_1 + r_2$, $r_1 r_2$ và r_1^* là các BTCQ định nghĩa các ngôn ngữ $L_1 \cup L_2$, $L_1 L_2$, và L_1^* . Vì vậy họ NNCQ là đóng đối với các phép toán này.
 - Để CM tính đóng đối với phép bù, cho $M = (Q, \Sigma, \delta, q_0, F)$ là dfa chấp nhận L_1 , thì dfa $\hat{M} = (Q, \Sigma, \delta, q_0, Q - F)$ chấp nhận $\bar{L}$.



Đóng dưới các phép toán tập hợp đơn giản

- Để CM tính đóng đối với phép giao ta có hai cách như sau.

Cách thứ nhất

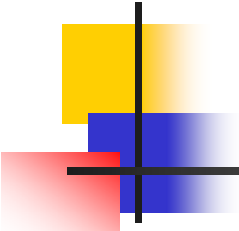
- Dựa vào qui tắc De Morgan ta có

$$L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}} = \overline{\overline{L_1}} \cap \overline{\overline{L_2}} = L_1 \cap L_2$$

Dựa vào tính đóng của phép bù và phép hội vừa được chứng minh ở trên ta suy ra tính đóng đối với phép giao.

Cách thứ hai

- Ta sẽ xây dựng một dfa cho $L_1 \cap L_2$.
- Cho $M_1 = (Q, \Sigma, \delta_1, q_0, F_1)$ và $M_2 = (P, \Sigma, \delta_2, p_0, F_2)$ là các dfa lần lượt chấp nhận L_1, L_2 .
- Ta xây dựng dfa $\hat{M} = (\hat{Q}, \Sigma, \hat{\delta}, (q_0, p_0), \hat{F})$ chấp nhận $L_1 \cap L_2$ bằng thủ tục *intersection* sau.



Thủ tục: intersection

■ Thủ tục: intersection

- **Input:** dfa $M_1 = (Q, \Sigma, \delta_1, q_0, F_1)$ và dfa $M_2 = (P, \Sigma, \delta_2, p_0, F_2)$
- **Output:** dfa $\hat{M} = (\hat{Q}, \Sigma, \hat{\delta}, (q_0, p_0), \hat{F})$
- $\hat{Q} = Q \times P$, tức là $\hat{Q} = \{(q_i, p_j): \text{trong đó } q_i \in Q \text{ còn } p_j \in P\}$.

Các chuyển trạng thái được xây dựng như sau

$$\hat{\delta}((q_i, p_j), a) = (q_k, p_l)$$

nếu và chỉ nếu

$$\delta_1(q_i, a) = q_k \text{ và } \delta_2(p_j, a) = p_l$$

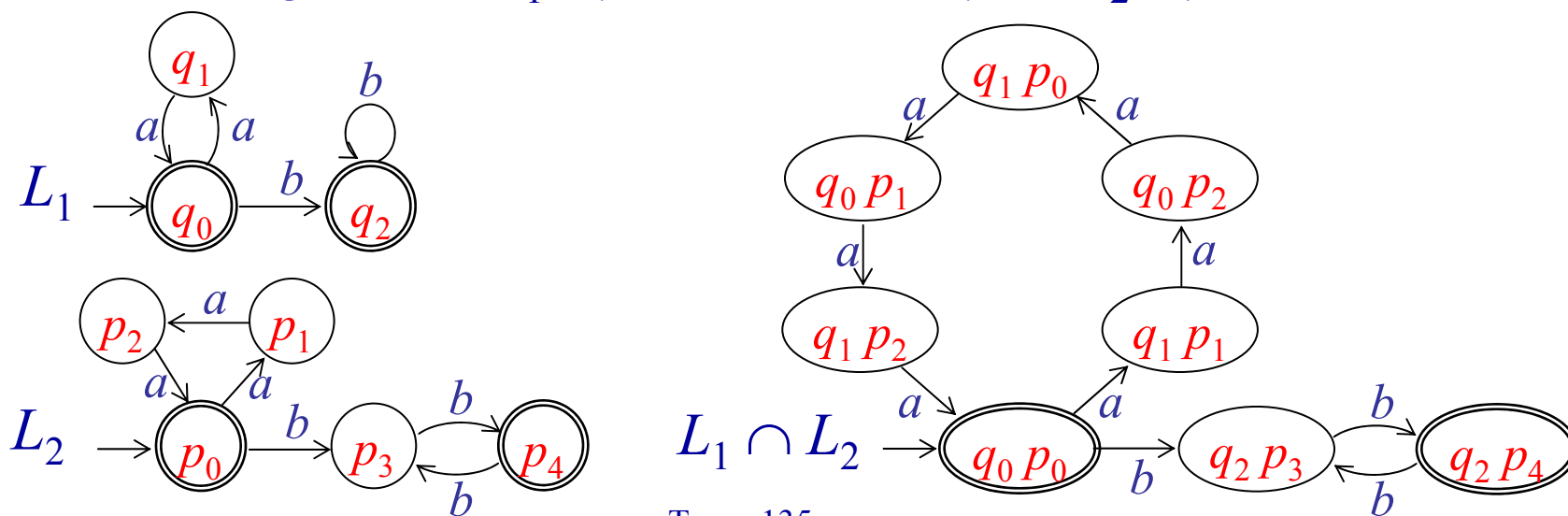
$$\hat{F} = \{(q_i, p_j): \text{trong đó } q_i \in F_1 \text{ còn } p_j \in F_2\}$$

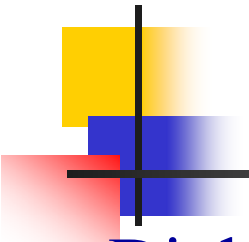
Thủ tục: intersection (tt)

- Cách xây dựng trên mô phỏng lại quá trình xử lý của đồng thời hai dfa M_1 và M_2 . Ngoài ra dựa vào định nghĩa của $\hat{\delta}$ ta thấy $\hat{M}$ chỉ chấp nhận những chuỗi mà được đồng thời cả hai dfa M_1 và M_2 chấp nhận. Vì vậy $\hat{M}$ chấp nhận $L_1 \cap L_2$.

■ Ví dụ

- Tìm dfa giao của $L_1 = \{a^{2n}b^m : n, m \geq 0\}$ và $L_2 = \{a^{3n}b^{2m} : n, m \geq 0\}$





Đóng dưới các phép toán tập hợp đơn giản (tt)

■ Định lý 4.2

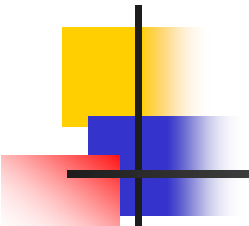
- Họ NNCQ là đóng dưới phép hiệu và nghịch đảo.

■ Chứng minh

- Để chứng minh tính đóng đối với phép hiệu dựa vào các qui tắc tập hợp ta có:

$$L_1 - L_2 = L_1 \cap \overline{L_2}$$

- Dựa vào tính đóng của phép bù và phép giao đã được chứng minh, suy ra tính đóng cho phép hiệu.
- Tính đóng của phép nghịch đảo đã được chứng minh ở Chương 3, slide 128.



Đóng dưới các phép toán khác

- Phép đồng hình (homomorphism)
- Định nghĩa 4.1

- Giả sử Σ và Γ là các bảng chữ cái, thì một hàm

$$h: \Sigma \rightarrow \Gamma^*$$

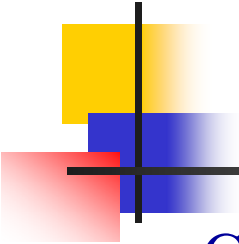
được gọi là một phép **đồng hình**. Bằng lời, một phép đồng hình là một sự thay thế trong đó mỗi kí hiệu đơn được thay thế bằng một chuỗi.

- Mở rộng nếu $w = a_1 a_2 \dots a_n$, thì

$$h(w) = h(a_1)h(a_2) \dots h(a_n)$$

- Nếu L là ngôn ngữ trên Σ , thì **ảnh đồng hình (homomorphic image)** của nó được định nghĩa là

$$h(L) = \{h(w) : w \in L\}.$$



Ví dụ

- Cho $\Sigma = \{a, b\}$, $\Gamma = \{a, b, c\}$ và h được định nghĩa như sau

$$h(a) = ab,$$

$$h(b) = bbc.$$

Thì $h(aba) = abbbcab$. Ảnh đồng hình của $L = \{aa, aba\}$ là ngôn ngữ $h(L) = \{abab, abbbcab\}$.

- Cho $\Sigma = \{a, b\}$, $\Gamma = \{b, c, d\}$ và h được định nghĩa như sau

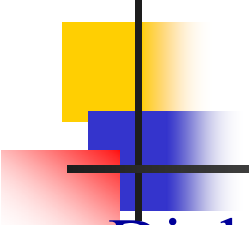
$$h(a) = dbcc, h(b) = bdc.$$

Nếu L là ngôn ngữ được biểu thị bởi BTCQ

$$r = (a + b^*)(aa)^*, \text{ thì}$$

$$r_1 = (dbcc + (bdc)^*)(dbccdbcc)^*,$$

là BTCQ biểu thị cho $h(L)$. Từ đó dẫn ta tới định lý sau



Định lý

■ Định lý 4.3

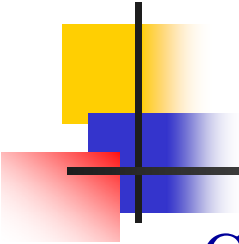
- Cho h là một đồng hình. Nếu L là một NNCQ, thì ảnh đồng hình của nó $h(L)$ cũng là NNCQ. Họ các NNCQ vì vậy là đóng dưới phép đồng hình bất kỳ.

■ Phép thương đúng

■ Định nghĩa 4.2

- Cho $w, v \in \Sigma^*$ thì **thương đúng (right quotient)** của w cho v được kí hiệu và định nghĩa là $w/v = u$ nếu $w = uv$, nghĩa là nếu v là **tiếp vĩ ngữ** của w thì w/v là **tiếp đầu ngữ** tương ứng của w .
- Cho L_1 và L_2 là các ngôn ngữ trên bảng chữ cái giống nhau, thì thương đúng của L_1 với L_2 được định nghĩa là

$$\begin{aligned} L_1/L_2 &= \{w/v: w \in L_1, v \in L_2\} \\ &= \{x : xy \in L_1 \text{ với một } y \text{ nào đó } \in L_2\} \end{aligned}$$



Ví dụ

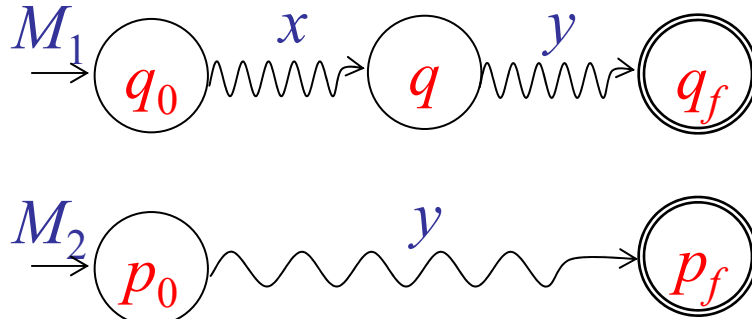
- Cho $L_1 = \{a^n b^m: n \geq 1, m \geq 0\} \cup \{ba\}$ và $L_2 = \{b^m: m \geq 1\}$, thì
$$L_1 / L_2 = \{a^n b^m: n \geq 1, m \geq 0\}.$$

- Vì L_1 , L_2 , và L_1 / L_2 là các NNCQ, điều này gợi ý cho chúng ta rằng thương đúng của hai NNCQ cũng là NNCQ.

■ Bổ đề

- Cho $M_1 = (Q_1, \Sigma, \delta_1, q_0, F_1)$ là một dfa cho L_1 . Nếu một trạng thái q nào đó $\in Q_1$ có tính chất tồn tại một chuỗi y nào đó $\in L_2$ sao cho $\delta_1^*(q, y) \in F_1$ thì $\forall x$ mà $\delta_1^*(q_0, x) = q$, x sẽ $\in L_1 / L_2$. Và vì vậy nếu thay những trạng thái kết thúc của M bằng những trạng thái q có tính chất này thì ta sẽ có một dfa mà chấp nhận L_1 / L_2 .

Định lý



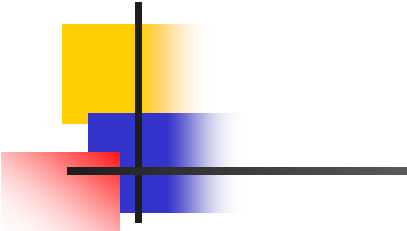
$\forall x$ mà $\delta_1^*(q_0, x) = q$
thì $x \in L_1/L_2$

■ Định lý 4.4

- Nếu L_1 và L_2 là các NNCQ, thì L_1/L_2 cũng chính qui. Chúng ta nói rằng họ NNCQ là đóng dưới phép thương đúng.

■ Chứng minh

- Cho $L_1 = L(M)$ trong đó $M = (Q, \Sigma, \delta, q_0, F)$ là một dfa. Ta xây dựng một dfa khác $\hat{M} = (Q, \Sigma, \delta, q_0, \hat{F})$, chấp nhận L_1/L_2 , bằng cách chỉ thay đổi tập F thông qua thủ tục sau.



Thủ tục: right quotient

■ Thủ tục: right quotient

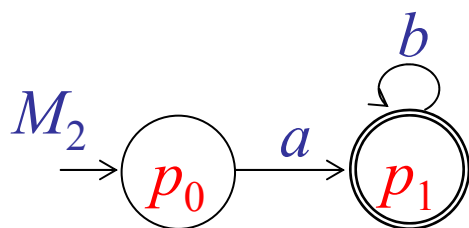
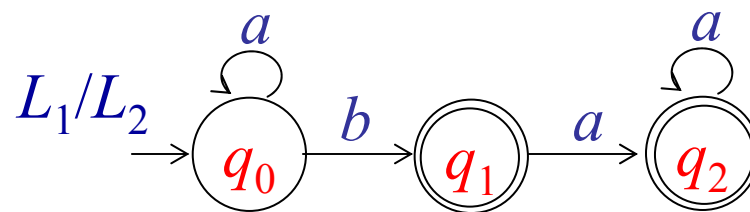
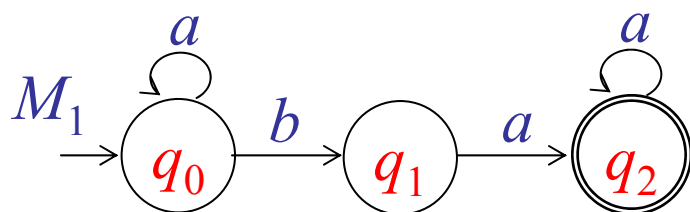
- **Input:** dfa $M_1 = (Q, \Sigma, \delta_1, q_0, F_1)$ và dfa $M_2 = (P, \Sigma, \delta_2, p_0, F_2)$
- **Output:** dfa $\hat{M} = (Q, \Sigma, \delta_1, q_0, \hat{F})$
- Ta xác định $\hat{F}$ bằng cách xác định với mỗi $q_i \in Q$, có tồn tại hay không chuỗi $y \in L_2$ sao cho $\delta^*(q_i, y) \in F$. Nếu đúng thì đưa q_i vào $\hat{F}$.
- Điều này có thể thực hiện được bằng cách xét các dfa $M_i = (Q, \Sigma, \delta_1, q_i, F)$. chính là M nhưng trạng thái khởi đầu q_0 được thay bằng q_i . Rồi xét xem $L_2 \cap L(M_i) \neq \emptyset$ không. Nếu khác thì q_i có tính chất đã nói ở trên và thêm q_i vào $\hat{F}$. Thực hiện điều này $\forall q_i \in Q$, ta sẽ xác định được $\hat{F}$ và vì vậy xây dựng được $\hat{M}$.

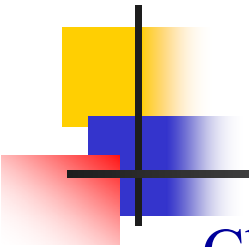
Ví dụ

- Tìm L_1/L_2 cho

$$L_1 = L(a^*baa^*),$$

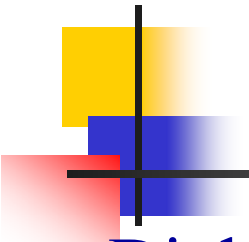
$$L_2 = L(ab^*).$$





Các câu hỏi cơ bản về NNCQ

- Cho một ngôn ngữ L và một chuỗi w , chúng ta có thể xác định được w có phải là một phần tử của L hay không?
 - Đây là một câu hỏi thành viên (membership) và phương pháp để trả lời nó được gọi là giải thuật thành viên.
- Một ngôn ngữ đã cho là hữu hạn hay vô hạn?
- Hai ngôn ngữ nào đó có giống nhau không?
- Có hay không một ngôn ngữ là tập con của một ngôn ngữ khác?
- Biểu diễn chuẩn (Standard representation)
 - Chúng ta nói rằng một NNCQ là được cho trong một dạng biểu diễn chuẩn nếu và chỉ nếu nó được mô tả bởi một trong ba dạng sau đây: một ôôtômát hữu hạn, một BTCQ hoặc một VPCQ.
 - Chú ý từ một dạng biểu diễn chuẩn này luôn có thể xác định được các dạng biểu diễn chuẩn khác nhờ vào các định lý đã được CM trước đây.



Các định lý

■ Định lý 4.5

- Cho một biểu diễn chuẩn của một NNCQ L bất kỳ trên Σ và một chuỗi w bất kỳ $\in \Sigma^*$, thì tồn tại giải thuật để xác định w có $\in L$ hay không.

■ Chứng minh

- Chúng ta biểu diễn ngôn ngữ bằng một dfa rồi kiểm tra xem w có được chấp nhận bởi dfa này không.

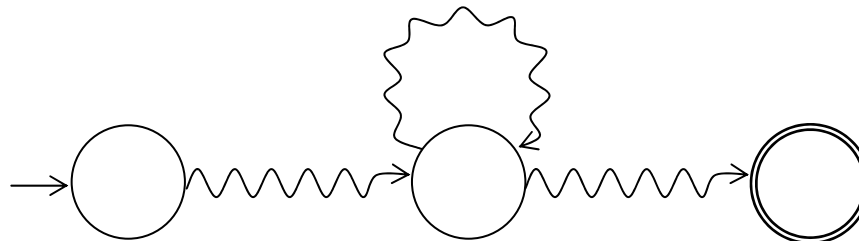
■ Định lý 4.6

- Tồn tại giải thuật để xác định một NNCQ đã cho trong một dạng biểu diễn chuẩn có trống, hữu hạn, vô hạn hay không.

Các định lý (tt)

■ Chứng minh

- Chúng ta biểu diễn ngôn ngữ bằng một dfa. Nếu tồn tại một con đường đi từ trạng thái khởi đầu đến một trạng thái kết thúc nào đó thì ngôn ngữ là khác trống.
- Để xác định ngôn ngữ có vô hạn không, ta tìm tất cả các đỉnh mà có chu trình đi qua nó. Nếu có một đỉnh trong số này thuộc một con đường nào đó đi từ trạng thái khởi đầu đến một trạng thái kết thúc thì ngôn ngữ là vô hạn, ngược lại thì là hữu hạn.





Các định lý (tt)

■ Định lý 4.7

- Cho các biểu diễn chuẩn của hai NNCQ L_1 và L_2 , tồn tại giải thuật để xác định có hay không $L_1 = L_2$.

■ Chứng minh

- Sử dụng L_1 và L_2 chúng ta xây dựng ngôn ngữ:

$$L_3 = (L_1 \cap \overline{L_2}) \cup (\overline{L_1} \cap L_2)$$

- Theo lý thuyết tập hợp ta có $L_1 = L_2$ khi và chỉ khi $L_3 = \emptyset$. Vậy thay vì kiểm tra L_1 có bằng L_2 không ta chuyển về kiểm tra L_3 có bằng $\emptyset$ không. Bằng tính đóng L_3 là chính qui, và chúng ta có thể tìm thấy DFA M mà chấp nhận L_3 . Thêm vào đó chúng ta đã có giải thuật trong Định lý 4.6 để xác định xem L_3 có bằng trống không. Nếu $L_3 = \emptyset$ thì $L_1 = L_2$, ngược lại thì không.



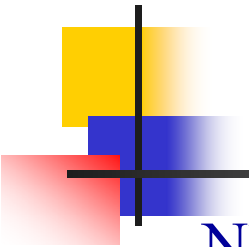
Nhận biết các NN không CQ

- Sử dụng nguyên lý chuồng chim bồ câu
 - Nếu chúng ta đặt n vật thể vào trong m hộp, và nếu $n > m$, thì ít nhất có một hộp chứa nhiều hơn một vật thể.
- Ngôn ngữ $L = \{a^n b^n : n \geq 0\}$ có chính qui không?
 - Câu trả lời là không, như chúng ta sẽ chứng tỏ bằng cách sử dụng phương pháp phản chứng sau.

Giả sử L là chính qui thì $\exists$ dfa $M = (Q, \{a, b\}, \delta, q_0, F)$ nào đó cho L .

Xét $\delta^*(q_0, a^i)$ với $i = 0, 1, 2, 3, \dots$. Vì có một số không giới hạn các i , nhưng chỉ có một số hữu hạn các trạng thái trong M , theo nguyên lý chuồng chim bồ câu thì phải có một trạng thái nào đó, chẳng hạn q , sao cho

$$\delta^*(q_0, a^n) = q \text{ và } \delta^*(q_0, a^m) = q, \text{ với } n \neq m.$$



Nhận biết các NN không CQ

Nhưng vì M chấp nhận $a^n b^n$ nên ta có

$$\delta^*(q, b^n) = q_f \in F.$$

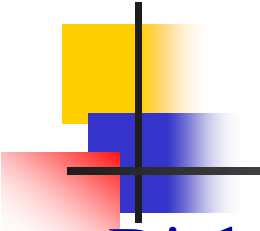
Kết hợp với ở trên ta suy ra

$$\delta^*(q_0, a^m b^n) = \delta^*(q, b^n) = q_f.$$

Vì vậy M chấp nhận cả chuỗi $a^m b^n$ với $n \neq m$. Điều này mâu thuẫn với định nghĩa của L , suy ra L không chính qui.

■ Nhận xét

- Trong lý luận này, nguyên lý chuồng chim bồ câu đơn giản phát biểu rằng một ô tô máy hữu hạn có một bộ nhớ hữu hạn. Để chấp nhận tất cả các chuỗi $a^n b^n$, một ô tô máy phải phân biệt giữa mọi tiếp đầu ngữ a^n và a^m . Nhưng vì chỉ có một số hữu hạn các trạng thái nội để thực hiện điều này, nên phải có một n và một m nào đó mà đối với chúng ô tô máy không thể phân biệt được.



Bổ đề bơm

■ Định lý 4.8

- Cho L là một NNCQ vô hạn, thì tồn tại một số nguyên dương m nào đó sao cho $\forall w \in L$ và $|w| \geq m$ đều tồn tại một cách phân tích w thành bộ ba

$$w = xyz,$$

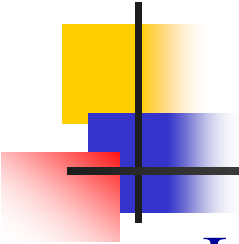
với $|xy| \leq m$, và $|y| \geq 1$, sao cho

$$w_i = xy^i z \in L$$

$$\forall i = 0, 1, 2, \dots$$

■ Chứng minh

- Nếu L là chính qui, thì $\exists$ một dfa chấp nhận nó. Lấy một dfa như thế có tập trạng thái $Q = \{q_0, q_1, q_2, \dots, q_n\}$. Chọn $m = |Q| = n + 1$.



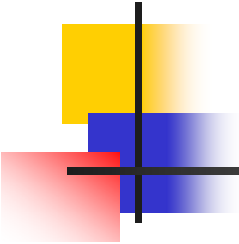
Chứng minh bổ đề bơm (tt)

Lấy một chuỗi w bất kỳ $\in L$ và $|w| = k \geq m$. Xét một dãy các trạng thái mà ô tô-mát đi qua khi xử lý chuỗi w , giả sử là

$$q_0, q_i, q_j, \dots, q_f$$

Vì $|w| = k$ suy ra dãy này có $k + 1$ phần tử. Vì $k + 1 > n + 1$ nên có ít nhất một trạng thái phải được lặp lại, và sự lặp lại này nằm trong $n + 2$ phần tử đầu tiên của dãy. Vì vậy dãy trên phải có dạng

$$q_0, q_i, q_j, \dots, \mathbf{q_r}, \dots, \mathbf{q_r}, \dots, q_f$$



Chứng minh bổ đề bơm (tt)

suy ra phải có các chuỗi con x, y, z của w sao cho

$$\delta^*(q_0, x) = q_r,$$

$$\delta^*(q_r, y) = q_r,$$

$$\delta^*(q_r, z) = q_f,$$

với $|xy| \leq n + 1 = m$, vì sự lặp lại trạng thái xảy ra trong $n + 2$ phần tử đầu tiên, và $|y| \geq 1$. Từ điều này suy ra

$$\delta^*(q_r, xz) = q_f,$$

cũng như

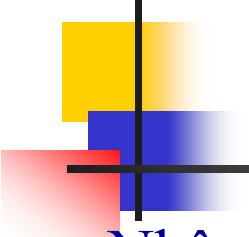
$$\delta^*(q_r, xy^iz) = q_f,$$

$\forall i = 0, 1, 2, \dots$ Đến đây định lý được chứng minh.



Vận dụng bổ đề bơm

- Sử dụng bổ đề bơm để chứng minh $L = \{a^n b^n: n \geq 0\}$ là không chính qui.
 - Giả sử L là chính qui, dễ thấy L vô hạn. Theo bổ đề bơm tồn tại số nguyên dương m .
 - Chọn $w = a^m b^m \in L$, $|w|=2m \geq m$. Theo bổ đề bơm $\exists$ một cách phân tích w thành bộ ba
$$w = xyz, \text{ trong đó } |xy| \leq m \text{ (1), } |y| = k \geq 1 \text{ (2).}$$
 - Từ cách chọn w có m kí hiệu a đi đầu, kết hợp với (1) suy ra xy chỉ chứa a , từ đây suy ra y cũng chỉ chứa a . Vậy $y = a^k$.
 - Xét $w_i = xy^i z$ với $i = 0$, ta có $w_0 = a^{n-k} b^n \in L$ theo bổ đề bơm, nhưng điều này mâu thuẫn với định nghĩa của L . Vậy L là không chính qui.



Vận dụng bổ đề bơm (tt)

■ Nhận xét

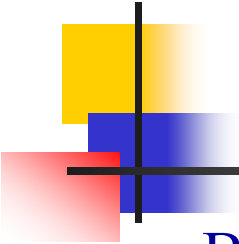
- Lý luận này có thể được trực quan hóa như một trò chơi chúng ta đấu với một đối thủ. Mục đích của chúng ta là thắng ván chơi bằng cách tạo ra một sự mâu thuẫn của bổ đề bơm, trong khi đối thủ thử chặn đứng chúng ta. Có bốn bước đi trong trò chơi này như sau.

(1) Đối thủ lấy m .

(2) Với m đã cho chúng ta lấy một chuỗi $w \in L$ thỏa $|w| \geq m$.

(3) Đối thủ chọn phân hoạch xyz , thỏa $|xy| \leq m$, $|y| \geq 1$. Chúng ta phải giả thiết rằng đối thủ chọn lựa làm sao cho chúng ta khó thắng ván chơi nhất.

(4) Chúng ta chọn i sao cho chuỗi được bơm lên $\notin L$.



Vận dụng bổ đề bơm (tt)

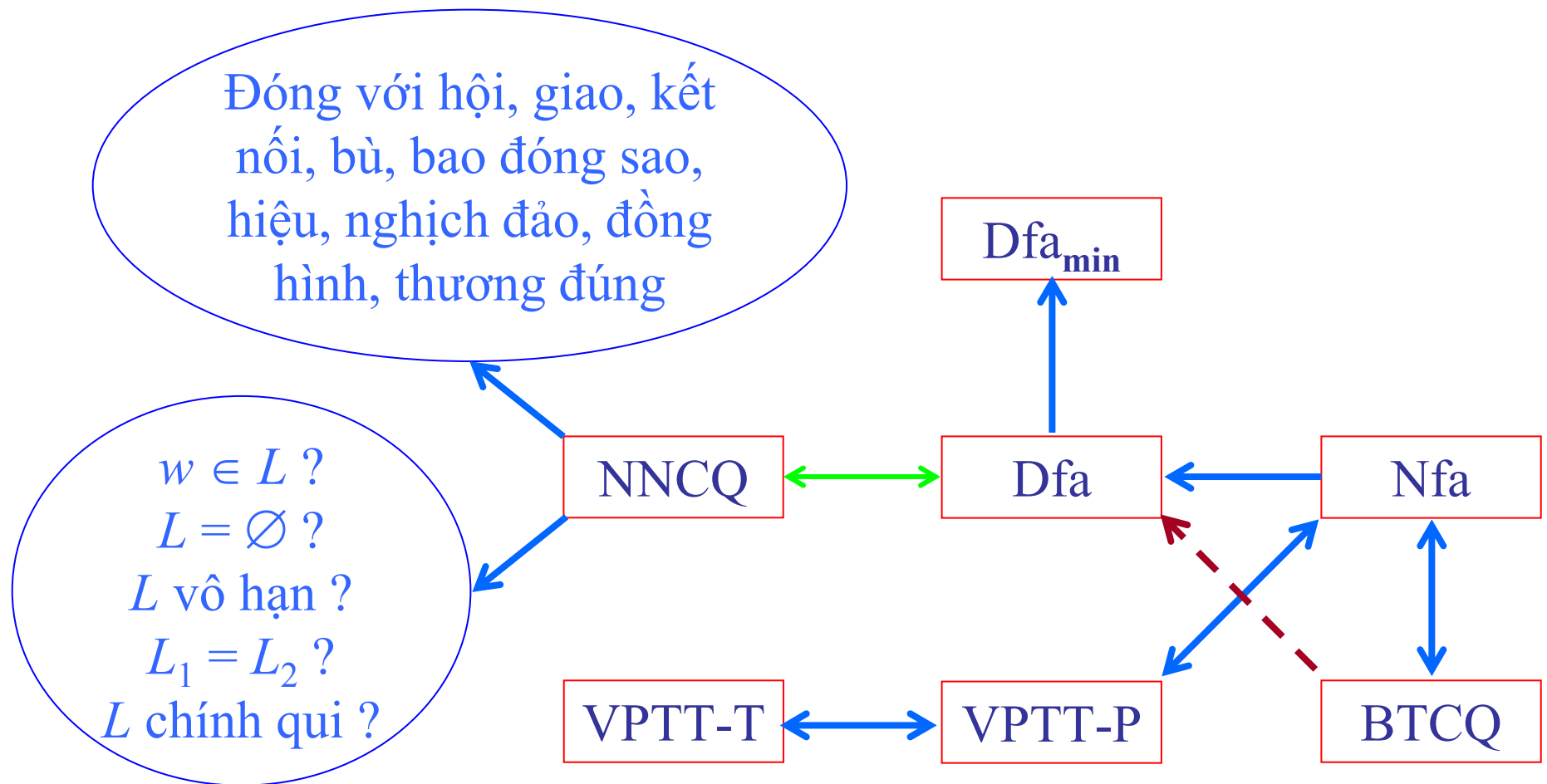
- Bước quyết định ở đây là bước (2). Trong khi chúng ta không thể ép buộc đối thủ lấy một phân hoạch cụ thể của chuỗi w , chúng ta có thể chọn chuỗi w sao cho đối thủ bị hạn chế nghiêm ngặt trong bước (3), ép buộc một sự chọn lựa của x, y, z sao cho cho phép chúng ta tạo ra một mâu thuẫn với bổ đề bơm trên bước kế tiếp của chúng ta.

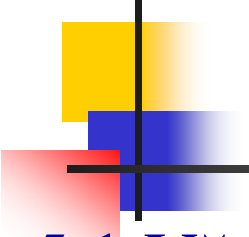
■ Ví dụ

Chứng minh các ngôn ngữ sau là không chính qui.

- $L_1 = \{ww^R: w \in \{a, b\}^*\}$
- $L_2 = \{a^n b^l: n \neq l\}$

Tóm tắt họ NNCQ



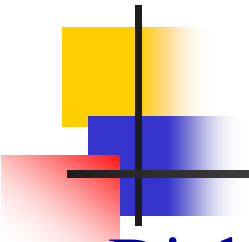


Chương 5 Ngôn ngữ phi ngữ cảnh

5.1 Văn phạm phi ngữ cảnh

5.2 Phân tích cú pháp và tính nhập nhằng

5.3 Văn phạm phi ngữ cảnh và ngôn ngữ lập trình



Văn phạm phi ngữ cảnh

■ Định nghĩa 5.1

- Một văn phạm $G = (V, T, S, P)$ được gọi là phi ngữ cảnh (context free) nếu mọi luật sinh trong P có dạng

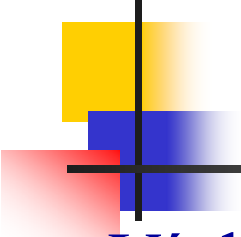
$$A \rightarrow x,$$

trong đó $A \in V$ còn $x \in (V \cup T)^*$.

- Một ngôn ngữ được gọi là phi ngữ cảnh nếu và chỉ nếu có một VPPNC G sao cho $L = L(G)$.

■ Nhận xét

- Mọi NNCQ đều là PNC, nhưng điều ngược lại thì không. Như chúng ta sẽ thấy sau này họ NNCQ là một tập con thực sự của họ NNPNC.



Các ví dụ về NNPNC

■ Ví dụ 1

- Văn phạm $G = (\{S\}, \{a, b\}, S, P)$, có các luật sinh

$$S \rightarrow aSa \mid bSb \mid \lambda,$$

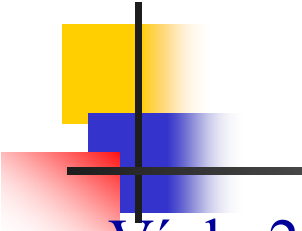
là PNC. Một dẫn xuất điển hình trong văn phạm này là

$$S \Rightarrow aSa \Rightarrow aaSaa \Rightarrow aabSbaa \Rightarrow aabbaa$$

Dễ thấy

$$L(G) = \{ww^R: w \in \{a, b\}^*\}$$

- Văn phạm trong ví dụ trên không những là PNC mà còn là tuyến tính. Các VPCQ và tuyến tính rõ ràng là PNC, nhưng một VPPNC không nhất thiết là tuyến tính.



Các ví dụ về NNPNC (tt)

■ Ví dụ 2

- Ngôn ngữ sau là PNC.

$$L = \{a^n b^n : n \geq 0\}$$

VPPNC cho ngôn ngữ này là:

$$S \rightarrow aSb \mid \lambda$$

■ Ví dụ 3

- Ngôn ngữ sau là PNC.

$$L = \{a^n b^m : n \neq m\}$$

Trường hợp $n > m$

$$S \rightarrow AS_1$$

$$S_1 \rightarrow aS_1b \mid \lambda$$

$$A \rightarrow aA \mid a$$

Trường hợp $m > n$

$$S \rightarrow S_1B$$

$$S_1 \rightarrow aS_1b \mid \lambda$$

$$B \rightarrow bB \mid b$$

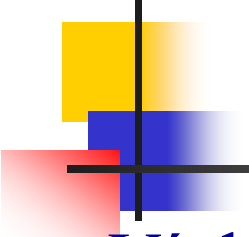
VP kết quả

$$S \rightarrow AS_1 \mid S_1B$$

$$S_1 \rightarrow aS_1b \mid \lambda$$

$$A \rightarrow aA \mid a$$

$$B \rightarrow bB \mid b$$



Các ví dụ về NNPNC (tt)

■ Ví dụ 4

- Xét văn phạm sau

$$S \rightarrow aSb \mid SS \mid \lambda.$$

Văn phạm này sinh ra ngôn ngữ

$$L = \{w \in \{a, b\}^*: n_a(w) = n_b(w) \text{ và } n_a(v) \geq n_b(v), \text{ với } v \text{ là một tiếp đầu ngữ bất kỳ của } w\}$$

■ Nhận xét

- Nếu trong ngôn ngữ trên thay a bằng dấu mở ngoặc $($, b bằng dấu đóng ngoặc $)$, thì ngôn ngữ sẽ tương ứng với cấu trúc ngoặc lồng nhau, chẳng hạn $(())$ hay $(() ())$, phổ biến trong các ngôn ngữ lập trình.

Dẫn xuất trái nhất và phải nhất

- Trong VPPNC mà không tuyến tính, một dẫn xuất có thể bao gồm nhiều dạng câu với nhiều hơn một biến. Như vậy, chúng ta có một sự lựa chọn thứ tự biến để thay thế.
- Xét văn phạm $G = (\{A, B, S\}, \{a, b\}, S, P)$ với các luật sinh
 1. $S \rightarrow AB$,
 2. $A \rightarrow aaA$,
 3. $A \rightarrow \lambda$,
 4. $B \rightarrow Bb$,
 5. $B \rightarrow \lambda$.

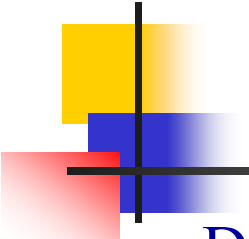
Dễ dàng thấy rằng văn phạm này sinh ra ngôn ngữ

$$L(G) = \{a^{2n}b^m : n \geq 0, m \geq 0\}.$$

Bây giờ xét hai dẫn xuất của chuỗi aab

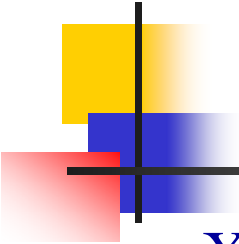
$$S \xRightarrow{1} AB \xRightarrow{2} aaAB \xRightarrow{3} aaB \xRightarrow{4} aaBb \xRightarrow{5} aab$$

$$S \xRightarrow{1} AB \xRightarrow{4} ABb \xRightarrow{2} aaABb \xRightarrow{5} aaAb \xRightarrow{3} aab.$$



Dẫn xuất trái nhất và phải nhất (tt)

- Để trình bày luật sinh nào được sử dụng, chúng ta đã đánh số các luật sinh và ghi số thích hợp trên kí hiệu dẫn xuất $\Rightarrow$.
 - Từ đây chúng ta thấy rằng hai dẫn xuất không chỉ tạo ra cùng một câu mà còn sử dụng chính xác các luật sinh giống nhau chỉ khác biệt về thứ tự các luật sinh được áp dụng.
 - Để loại bỏ các yếu tố không quan trọng như thế, chúng ta thường yêu cầu rằng các biến được thay thế trong một thứ tự chỉ định. Từ đây chúng ta đưa ra định nghĩa sau.
- Định nghĩa 5.2
- Một dẫn xuất được gọi là **trái nhất** (DXTN - leftmost derivation) nếu trong mỗi bước biến trái nhất trong dạng câu được thay thế. Nếu biến phải nhất được thay thế, chúng ta gọi là dẫn xuất **phải nhất** (DXPN - rightmost derivation).



Ví dụ

- Xét văn phạm với các luật sinh (được đánh chỉ số bên tay phải)

$$S \rightarrow aAB, \quad 1$$

$$A \rightarrow bBb, \quad 2$$

$$B \rightarrow A \mid \lambda, \quad 3, 4$$

$$S \xRightarrow{1} aAB \xRightarrow{2} abBbB \xRightarrow{3} abAbB \xRightarrow{2} abbBbbB \xRightarrow{4} abbbbB \xRightarrow{4} abbbb$$

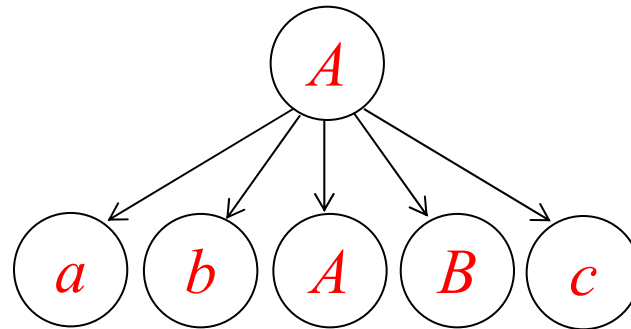
là một **DXTN** của chuỗi *abbbb*. Một **DXPN** của chuỗi này là

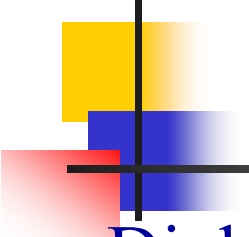
$$S \xRightarrow{1} aAB \xRightarrow{4} aA \xRightarrow{2} abBb \xRightarrow{3} abAb \xRightarrow{2} abbBbb \xRightarrow{4} abbbb$$

- DXTN và DXPN có lợi điểm là ta chỉ cần trình bày dãy số hiệu luật sinh được dùng để sinh ra câu đó mà không sợ bị nhầm lẫn.
- DXTN của *abbbb* là: 123244.
- DXPN của *abbbb* là: 142324.

Cây dẫn xuất

- Một cách thứ hai để trình bày các dẫn xuất, độc lập với thứ tự các luật sinh được áp dụng, là bằng *cây dẫn xuất* (CDX).
- Một CDX là một cây có thứ tự trong đó các nốt được gán nhãn với vế trái của luật sinh còn các con của các nốt biểu diễn vế phải tương ứng của nó. Chẳng hạn, bên dưới trình bày một phần của CDX biểu diễn luật sinh $A \rightarrow abABc$.





Cây dẫn xuất (tt)

■ Định nghĩa 5.3

- Cho $G = (V, T, S, P)$ là một VPPNC. Một cây có thứ tự là một cây dẫn xuất cho G nếu và chỉ nếu có các tính chất sau.

1. Gốc được gán nhãn là S .
2. Mỗi lá có một nhãn lấy từ tập $T \cup \{\lambda\}$.
3. Mỗi nốt bên trong (không phải là lá) có một nhãn lấy từ V .
4. Nếu mỗi nốt có nhãn $A \in V$, và các con của nó được gán nhãn (từ trái sang phải) $a_1, a_2, \dots, a_n$, thì P phải chứa một luật sinh có dạng

$$A \rightarrow a_1 a_2 \dots a_n$$

5. Một lá được gán nhãn λ thì không có anh chị em, tức là, một nốt với một con được gán nhãn λ không thể có con nào khác.



Cây dẫn xuất (tt)

- Một cây mà có các tính chất 3, 4 và 5, còn tính chất (1) không nhất thiết được giữ và tính chất 2 được thay thế bằng 2'. Mỗi lá có một nhãn lấy từ tập $V \cup T \cup \{\lambda\}$ thì được gọi là một **cây dẫn xuất riêng phần** (CDXRP).
- Chuỗi kí hiệu nhận được bằng cách đọc các nốt lá của cây từ trái sang phải, bỏ qua bất kỳ λ nào được bắt gặp, được gọi là **kết quả** (yield) của cây.

Ví dụ

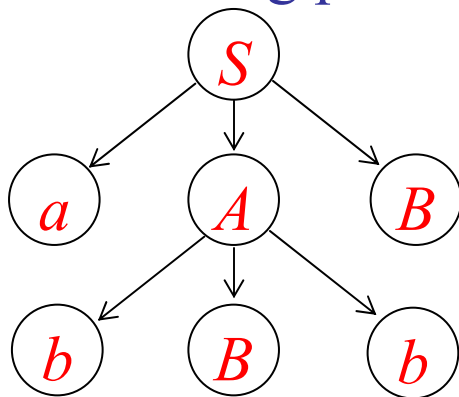
- Xét văn phạm G với các luật sinh sau

$$S \rightarrow aAB,$$

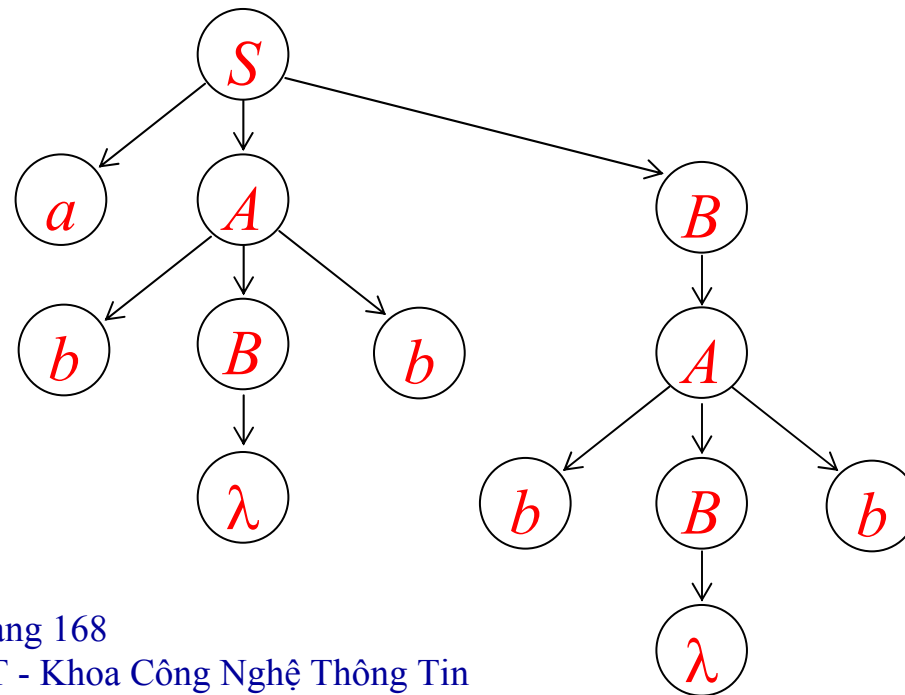
$$A \rightarrow bBb,$$

$$B \rightarrow A \mid \lambda,$$

CDX riêng phần



CDX cho chuỗi $abbbb$





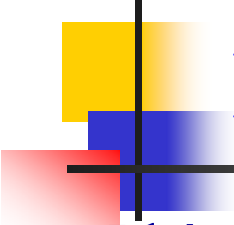
Mối quan hệ giữa dạng câu và CDX

■ Nhận xét

- CDX đưa ra một mô tả của dẫn xuất rất tường minh và dễ hiểu. Giống như ĐTCTT cho ô tô máy hữu hạn, sự tường minh là một sự giúp đỡ lớn trong việc thực hiện lý luận. Tuy vậy, đầu tiên chúng ta phải thiết lập một quan hệ giữa dẫn xuất và CDX.

■ Định lý 5.1

- Cho $G = (V, T, S, P)$ là một VPPNC, thì $\forall w \in L(G)$, tồn tại một CDX của G mà kết quả của nó là w . Ngược lại, kết quả của một CDX bất kỳ là thuộc $L(G)$. Tương tự, nếu t_G là một CDX riêng phần bất kỳ của G mà gốc của nó được gán nhãn là S thì kết quả của t_G là một dạng câu của G .



Phân tích cú pháp và tính nhập nhằng

- Phân tích cú pháp (Syntax analysis hay parsing)
 - Phân tích cú pháp (PTCP) là quá trình xác định một chuỗi có được sinh ra bởi một văn phạm nào đó không, cụ thể là quá trình tìm CDX cho chuỗi đó.
 - Kết quả của quá trình PTCP rơi vào một trong hai khả năng “**yes**” hoặc “**no**”. “Yes” có nghĩa là chuỗi được sinh ra bởi văn phạm và kèm theo một hay một số dẫn xuất sinh ra chuỗi. “No” có nghĩa là chuỗi không được sinh ra bởi văn phạm hay còn gọi là chuỗi không đúng cú pháp, có lỗi (error).
 - Các giải thuật phân tích cú pháp thường có dạng như sau:
Input: $G = (V, T, S, P)$ và chuỗi w cần phân tích
Output: “yes” hay “no”. Trong trường hợp “yes” thường có kèm theo DXTN hay DXPN của chuỗi.



Các trường phái phân tích cú pháp

- Có hai trường phái PTCP cơ bản

1. **PTCP từ trên xuống (Top-down parsing)**: xây dựng CDX từ gốc xuống lá.
2. **PTCP từ dưới lên (Bottom-up parsing)**: xây dựng CDX từ lá lên gốc.

- Ví dụ

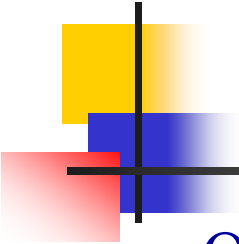
- Cho văn phạm G sau:

$$S \rightarrow aAbS \mid bBS \mid \lambda \quad (1, 2, 3)$$

$$A \rightarrow aAA \mid aS \mid b \quad (4, 5, 6)$$

$$B \rightarrow bBB \mid bS \mid a \quad (7, 8, 9)$$

Hãy PTCP từ trên xuống cho chuỗi sau: $w = aabbbba$.



Ví dụ về PTCP từ trên xuống

- Quá trình phân tích bắt đầu từ kí hiệu mục tiêu S . Là quá trình thay thế biến trong dạng câu để đi từ dạng này sang dạng câu khác chi tiết hơn cho đến khi hoặc đến được chuỗi cần phân tích hoặc không (còn được gọi là gặp lỗi).
- Việc PTCP từ trên xuống bao gồm hai đầu đọc, một đọc trên chuỗi kí hiệu nhập, di chuyển từ trái sang phải, một đọc trên các dạng câu, cũng di chuyển từ trái sang phải. Vào thời điểm khởi đầu, đầu đọc 1 nằm ở vị trí khởi đầu của chuỗi nhập, đầu đọc 2 nằm ở vị trí khởi đầu của dạng câu thứ nhất chính là kí hiệu mục tiêu S . Ta thể hiện mỗi đầu đọc bằng một dấu chấm •.
- Vấn đề cốt lõi của PTCP từ trên xuống là ***quyết định chọn về phải nào*** trong các về phải của biến cần thay thế mà ***có khả năng nhất*** sinh ra được chuỗi nhập.

Ví dụ về PTCP từ trên xuống (tt)

$$S \rightarrow aAbS \mid bBS \mid \lambda \quad (1, 2, 3)$$

$$A \rightarrow aAA \mid aS \mid b \quad (4, 5, 6)$$

$$B \rightarrow bBB \mid bS \mid a \quad (7, 8, 9)$$

DXTN: 1.4.6.6.2.9.3

	Khởi đầu	1		4	
Chuỗi nhập	$\bullet aabbbbba$	$\bullet aabbbbba$	$a\bullet abbbbba$	$a\bullet abbbbba$	$aa\bullet bbbbba$
Dạng câu	$\bullet S$	$\bullet aAbS$	$a\bullet AbS$	$a\bullet aAAbS$	$aa\bullet AAbS$
	6		6		
Chuỗi nhập	$aa\bullet bbbb$	$aab\bullet bbba$	$aab\bullet bbba$	$aabb\bullet bba$	$aabbb\bullet ba$
Dạng câu	$aaa\bullet bAbS$	$aab\bullet AbS$	$aab\bullet bbS$	$aabb\bullet bS$	$aabbb\bullet S$
	2		9		3
Chuỗi nhập	$aabbb\bullet ba$	$aabbbb\bullet a$	$aabbbb\bullet a$	$aabbbbba\bullet$	$aabbbbba\bullet$
Dạng câu	$aabbb\bullet bBS$	$aabbbb\bullet BS$	$aabbbb\bullet aS$	$aabbbbba\bullet S$	$aabbbbba\bullet$

Ví dụ về PTCP từ dưới lên

- Hãy PTCP từ dưới lên cho $w = abbcde$ trên văn phạm G sau:

$$S \rightarrow aABe \quad (1)$$

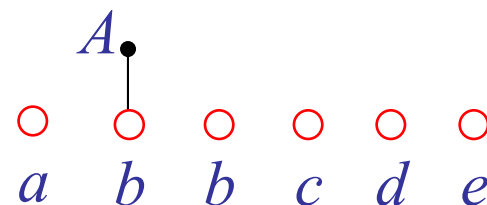
$$A \rightarrow Abc \mid b \quad (2, 3)$$

$$B \rightarrow d \quad (4)$$

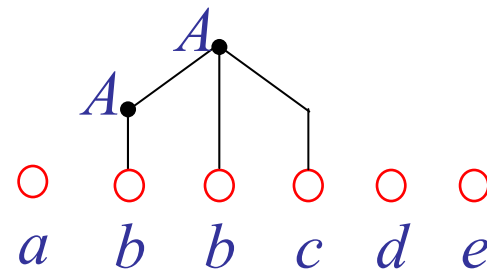
B1. Các lá của cây dẫn xuất

$a \quad b \quad b \quad c \quad d \quad e$

B2. Thu giảm bằng $A \rightarrow b$



B3. Thu giảm bằng $A \rightarrow Abc$



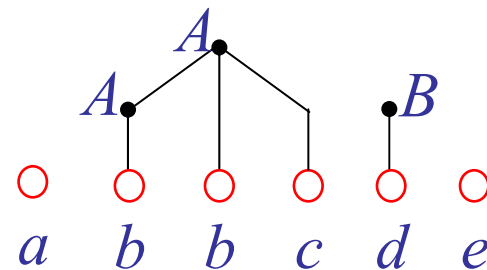
Ví dụ về PTCP từ dưới lên (tt)

$$S \rightarrow aABe \quad (1)$$

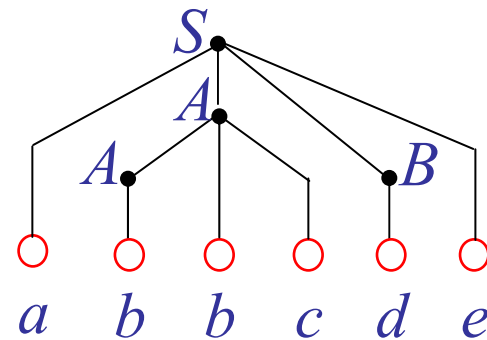
$$A \rightarrow Abc \mid b \quad (2, 3)$$

$$B \rightarrow d \quad (4)$$

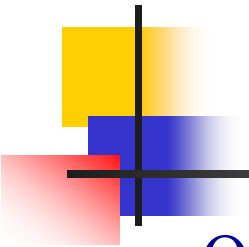
B4. Thu giảm bằng $B \rightarrow d$



B5. Thu giảm bằng $S \rightarrow aABe$

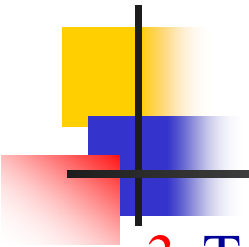


- Kết quả: $abbcd e \xleftarrow{3} aAbcd e \xleftarrow{2} aAde \xleftarrow{4} aABe \xleftarrow{1} S$
- Hay $S \xrightarrow{1} aABe \xrightarrow{4} aAde \xrightarrow{2} aAbcd e \xrightarrow{3} abbcde \text{ (DXPN)}$



Phương pháp PTCP vét cạn

- Qua ví dụ trên ta thấy, vấn đề cốt lõi của PTCP từ dưới lên là là *quyết định chọn chuỗi thành phần nào* của dạng câu để thu gọn mà *có khả năng nhất thu gọn* được về thành biến mục tiêu.
- Phương pháp phân tích cú pháp vét cạn (PPPTCPVC - exhaustive search parsing)
 1. Ở *lượt (round)* thứ nhất xem xét tất cả các luật sinh có dạng
$$S \rightarrow x,$$
tìm tất cả các x mà có thể được dẫn xuất từ S bởi một bước.
 2. Nếu không có kết quả nào trong số này trùng với w , chúng ta sẽ đi tiếp đến lượt tiếp theo, trong đó chúng ta áp dụng tất cả các luật sinh có thể tới biến trái nhất của mỗi x .



Phương pháp PTCP vét cạn (tt)

3. Trong mỗi lượt kế tiếp, chúng ta lại lấy tất cả các biến trái nhất và áp dụng tất cả các luật sinh có thể, rồi lặp lại bước 2.

■ Nhận xét

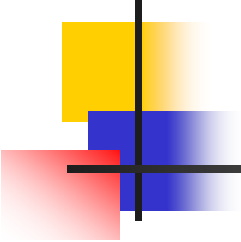
- Sau lượt thứ n chúng ta có các dạng câu mà có thể được dẫn xuất từ S với n luật sinh.
- Nếu $w \in L(G)$, thì nó phải có một DXTN có độ dài hữu hạn. Vì vậy phương pháp này cuối cùng sẽ tìm được một DXTN của w .

■ Ví dụ

- Xét văn phạm

$$S \rightarrow SS \mid aSb \mid bSa \mid \lambda \quad 1, 2, 3, 4$$

và chuỗi $w = aabb$.



Ví dụ

$$S \rightarrow SS \mid aSb \mid bSa \mid \lambda \quad 1, 2, 3, 4$$

$$w = aabb.$$

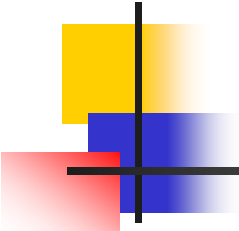
Lượt 1

1. $S \Rightarrow SS$
2. $S \Rightarrow aSb$
3. $S \Rightarrow \text{~~bSa~~}$
4. $S \Rightarrow \text{~~\lambda~~}$

Lượt 2

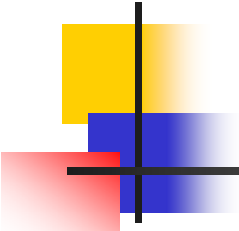
- 1.1 $S \Rightarrow SS \Rightarrow SSS$
- 1.2 $S \Rightarrow SS \Rightarrow aSbS$
- 1.3 $S \Rightarrow SS \Rightarrow \text{~~bSaS~~}$
- 1.4 $S \Rightarrow SS \Rightarrow \text{~~S~~}$
- 2.1 $S \Rightarrow aSb \Rightarrow aSSb$
- 2.2 $S \Rightarrow aSb \Rightarrow aaSbb$
- 2.3 $S \Rightarrow aSb \Rightarrow \text{~~abSab~~}$
- 2.4 $S \Rightarrow aSb \Rightarrow \text{~~ab~~}$

- Đến **Lượt 3** ta tìm thấy **2.2.4** $S \Rightarrow aSb \Rightarrow abSab \Rightarrow abab$
- Vậy chuỗi $aabb$ thuộc ngôn ngữ của văn phạm đang xét.



Nhận xét

- PPPTCPVC có các nhược điểm nghiêm trọng sau.
 1. Không hiệu quả. Bị **bùng nổ tổ hợp**.
 2. Có khả năng **không bao giờ kết thúc** đối với các chuỗi $\notin L(G)$.
Chẳng hạn với $w = abb$, phương pháp này sẽ đi đến việc sinh ra vô hạn các dạng câu mà không dừng lại, trừ phi chúng ta bổ sung thêm vào cách để cho nó dừng lại.
- Nhược điểm 2 có thể khắc phục được nếu chúng ta giới hạn văn phạm **không được phép chứa các luật sinh rỗng** ($A \rightarrow \lambda$) và đơn vị ($A \rightarrow B$).



Định lý

■ Định lý 5.2

- Giả sử rằng $G = (V, T, S, P)$ là một VPPNC mà không có bất kỳ luật sinh nào có dạng

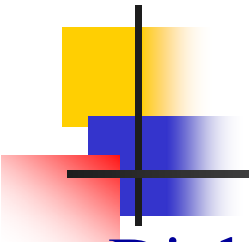
$$A \rightarrow \lambda, \text{ hay}$$

$$A \rightarrow B,$$

trong đó $A, B \in V$, thì PPPTCPVC có thể được hiện thực thành một giải thuật mà $\forall w \in T^*$, hoặc tạo ra được sự PTCP của w , hoặc biết rằng không có sự PTCP nào là có thể cho nó.

■ Chứng minh

- Ở mỗi bước dẫn xuất hoặc **chiều dài** hoặc **số kí hiệu kết thúc** của dạng câu tăng ít nhất 1 đơn vị. Vì vậy sau không quá **$(2|w| - 1)$** lượt, chúng ta sẽ xác định được w có $\in L(G)$ không.



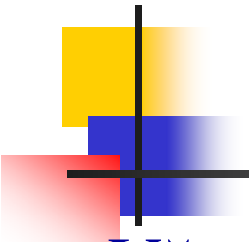
Định lý (tt)

■ Định lý 5.3

- Đối $\forall$ VPPNC $\exists$ giải thuật mà phân tích một chuỗi w bất kỳ có $\in L(G)$ không trong một số bước tỉ lệ với $|w|^3$.

■ Nhận xét

- Một PP mà thời gian tỉ lệ với $|w|^3$ là không hiệu quả. Nếu một trình biên dịch dựa trên đó sẽ cần một lượng thời gian khá lớn để PTCP cho thậm chí một chương trình có độ dài trung bình.
- Những gì mà chúng ta muốn là tỉ lệ với $|w|$. Chúng ta gọi những PP như vậy là PPPTCP *thời gian tuyến tính*.
- Tổng quát, chúng ta không biết một PPPTCP thời gian tuyến tính nào cho NNPNC, nhưng các PP như thế có thể được tìm thấy đối với một số lớp VP đặc biệt.



Văn phạm-s

- Văn phạm-s (simple grammar)

- Là một VPPNC trong đó các luật sinh có dạng

$$A \rightarrow ax$$

trong đó $A \in V$, $a \in T$, $x \in V^*$, và mỗi cặp (A, a) chỉ có thể xuất hiện tối đa trên một luật sinh. Nói cách khác, nếu hai luật sinh bất kỳ mà có vế trái giống nhau thì vế phải của chúng phải bắt đầu bằng các kí hiệu kết thúc khác nhau.

- Ví dụ

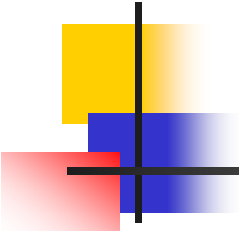
- Bên dưới là một ví dụ về văn phạm-s

$$S \rightarrow aS \mid bA \quad (1, 2)$$

$$A \rightarrow aAA \mid b \quad (3, 4)$$

Văn phạm-s (tt)

- Văn phạm-s cho phép PTCP một chuỗi w bất kỳ không quá $|w|$ bước.
- Với mỗi cặp (A, a) trong đó A là biến cần thay thế, a là kí hiệu đang được xét ở chuỗi nhập, có tối đa một vế phải của A có thể được áp dụng.
- Ví dụ với VP trên việc PTCP chuỗi *ababb*
 $S \rightarrow aS \mid bA \quad (1, 2)$
 $A \rightarrow aAA \mid b \quad (3, 4)$
chỉ tốn 5 bước và được kết quả như sau.
$$S \xRightarrow{1} aS \xRightarrow{2} abA \xRightarrow{3} abaAA \xRightarrow{4} ababA \xRightarrow{4} ababb$$
- Văn phạm-s có thể mở rộng ở x , bằng cách cho $x \in (V \cup T)^*$. Điều này không làm thay đổi khả năng và tính chất của văn phạm mà còn làm quá trình PTCP đơn giản hơn một chút.
- Ngôn ngữ Pascal có thể được biểu thị bằng văn phạm-s.



Tính nhập nhằng trong VP và NN

■ Định nghĩa 5.4

- Một VPPNC G được gọi là nhập nhằng nếu $\exists$ một $w \in L(G)$ mà có ít nhất hai CDX khác nhau. Nói cách khác, sự nhập nhằng suy ra tồn tại hai hay nhiều DXTN hay PN.

■ Ví dụ

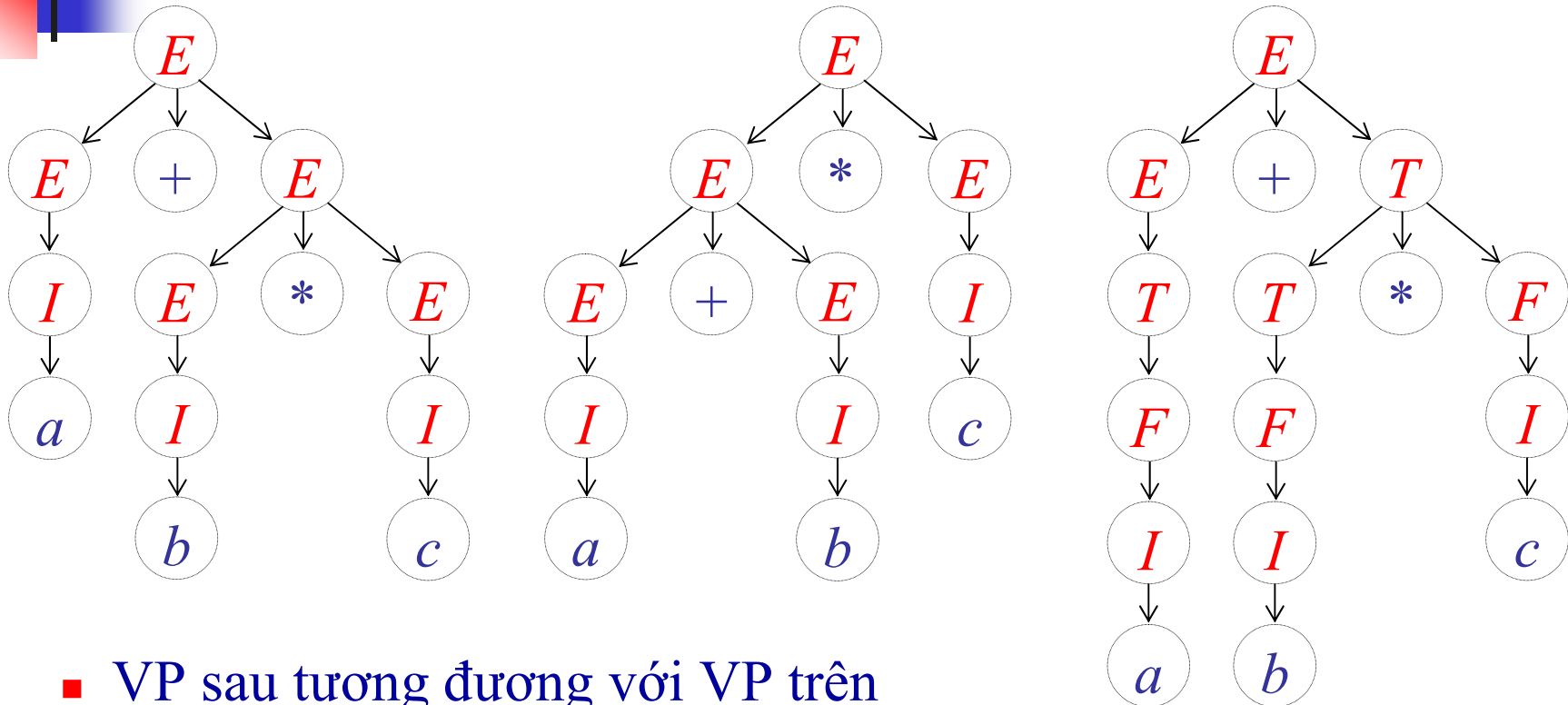
- Xét văn phạm sau $G = (V, T, E, P)$ với $V = \{E, I\}$, $T = \{a, b, c, +, *, (,)\}$ và các luật sinh

$$E \rightarrow I \mid E + E \mid E * E \mid (E)$$

$$I \rightarrow a \mid b \mid c$$

- Văn phạm này là nhập nhằng vì với chuỗi $a + b * c$ có hai CDX khác nhau trên G như sau.

Tính nhập nhằng trong VP và NN (tt)



- VP sau tương đương với VP trên nhưng không có nhập nhằng.
- Tập biến $V = \{E, T, F, I\}$

$$E \rightarrow T \mid E + T$$

$$T \rightarrow F \mid T * F$$

$$F \rightarrow I \mid (E)$$

$$I \rightarrow a \mid b \mid c$$

Tính nhập nhằng trong VP và NN (tt)

■ Định nghĩa 5.5

- Nếu L là một NNPNC mà đối với nó $\exists$ một VP không nhập nhằng, thì L được gọi là không nhập nhằng. Nếu mọi VP sinh ra L mà nhập nhằng, thì NN được gọi là **nhập nhằng cố hữu**.

■ Ví dụ

- Ngôn ngữ $L = \{a^n b^n c^m\} \cup \{a^n b^m c^m\}$ với n, m không âm là một NNPNC nhập nhằng cố hữu. (Chú ý $L = L_1 \cup L_2$).

$$G_1: S_1 \rightarrow X_1 C$$

$$X_1 \rightarrow aX_1 b \mid \lambda$$

$$C \rightarrow cC \mid \lambda$$

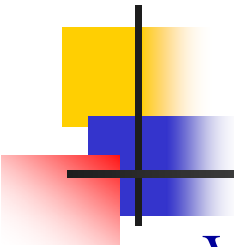
$$G_2: S_2 \rightarrow AX_2$$

$$X_2 \rightarrow bX_2 c \mid \lambda$$

$$A \rightarrow aA \mid \lambda$$

- Một VP cho L bằng cách kết hợp hai VP trên với luật sinh thêm vào là

$$S \rightarrow S_1 \mid S_2$$



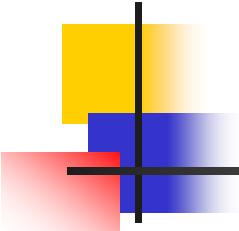
Tính nhập nhằng trong VP và NN (tt)

- Văn phạm này là nhập nhằng vì chuỗi $a^n b^n c^n$ thuộc cả L_1 lẫn L_2 nên nó có hai dẫn xuất riêng biệt một cái bắt đầu bằng $S \Rightarrow S_1$ và một cái bắt đầu bằng $S \Rightarrow S_2$.
- Điều này cũng gợi ý cho chúng ta chứng minh rằng mọi VP cho L đều sẽ nhập nhằng trên chuỗi $a^n b^n c^n$ tương tự như trường hợp trên.
- Một chứng minh chặt chẽ đã được thực hiện trong tài liệu của Harrison năm 1978. Ở đây nó được để lại như bài tập cho các bạn.



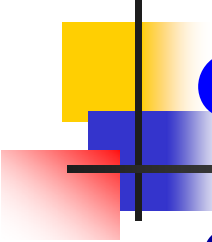
VPPNC và ngôn ngữ lập trình

- Một ứng dụng quan trọng của lý thuyết NNHT là định nghĩa các NNLT cũng như xây dựng các trình dịch cho chúng.
- Theo truyền thống người ta dùng dạng ký pháp *Backus-Naur* (viết tắt là BNF) để viết một NNLT. Chẳng hạn
$$\langle expression \rangle ::= \langle term \rangle \mid \langle expression \rangle + \langle term \rangle,$$
$$\langle term \rangle ::= \langle factor \rangle \mid \langle term \rangle * \langle factor \rangle,$$
$$\langle if_statement \rangle ::= if \langle expression \rangle \langle then_clause \rangle \langle else_clause \rangle$$
- Văn phạm-s không đủ sức để biểu diễn các NNLT.
- Có hai loại văn phạm là *LL* và *LR* có khả năng biểu diễn các NNLT, và còn cho phép PTCP trong thời gian tuyến tính.
- Không $\exists$ giải thuật loại bỏ sự nhập nhằng của VP.
- VPPNC không thể biểu diễn mặt ngữ nghĩa của các NNLT.



Chương 6 Đơn giản hóa VPPNC và các dạng chuẩn

- 6.1 Các phương pháp để biến đổi văn phạm
- 6.2 Hai dạng chuẩn quan trọng
- 6.3 Giải thuật thành viên cho văn phạm phi ngữ cảnh



Các phương pháp để biến đổi văn phạm

- Chuỗi trống đóng một vai trò khá đặc biệt trong nhiều định lý và chứng minh, và thường cần có một sự chú ý đặc biệt cho nó.
- Nếu $L \ni \lambda$ thì biểu diễn $L = L_1 \cup \{\lambda\}$ với $L_1 = L - \{\lambda\}$. Nếu

$$G_1 = (V_1, T, S_1, P_1)$$

là văn phạm biểu diễn cho L_1 thì

$$G = (V_1 \cup \{S\}, T, S, P_1 \cup \{S \rightarrow S_1 \mid \lambda\})$$

là văn phạm biểu diễn cho L .

- Trong chương này, chúng ta chỉ xem xét các NNPNC không chứa λ .
- Tuy nhiên những kết luận cho ngôn ngữ không chứa λ vẫn có thể áp dụng cho ngôn ngữ có chứa λ .



Một vài qui tắc thay thế hiệu quả

■ Định lý 6.1

- Cho $G = (V, T, S, P)$ là một VPPNC. Giả sử P có chứa luật sinh

$$A \rightarrow x_1 B x_2$$

trong đó A, B là các biến khác nhau và

$$B \rightarrow y_1 \mid y_2 \mid \dots \mid y_n$$

là tập tất cả các luật sinh trong P mà có B ở vế trái.

Cho $G_1 = (V, T, S, P_1)$ là VP được xây dựng bằng cách xóa đi

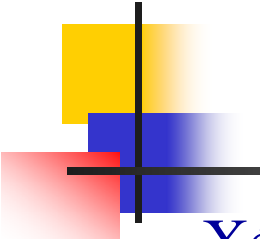
$$A \rightarrow x_1 B x_2$$

từ P , và thêm vào nó

$$A \rightarrow x_1 y_1 x_2 \mid x_1 y_2 x_2 \mid \dots \mid x_1 y_n x_2$$

Thì

$$L(G) = L(G_1)$$



Ví dụ

- Xét văn phạm $G = (\{A, B\}, \{a, b\}, A, P)$ với các luật sinh

$$A \rightarrow a \mid aA \mid bBc,$$

$$B \rightarrow abA \mid b.$$

Sau khi thay thế biến B ta nhận được VP tương đương như sau

$$A \rightarrow a \mid aA \mid babAc \mid bbc,$$

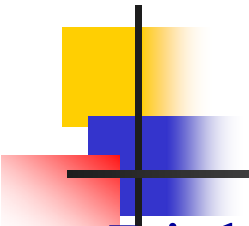
$$B \rightarrow abA \mid b$$

- Chuỗi $abbc$ có các dẫn xuất trong G và G_1 lần lượt như sau:

$$A \Rightarrow aA \Rightarrow abBc \Rightarrow abbc$$

$$A \Rightarrow aA \Rightarrow abbc$$

- Chú ý rằng, biến B và các luật sinh của nó vẫn còn ở trong VP mặc dù chúng không còn đóng vai trò gì trong bất kỳ dẫn xuất nào. Sau này chúng ta sẽ thấy rằng những luật sinh không cần thiết như vậy có thể bị loại bỏ ra khỏi văn phạm.



Loại bỏ đệ qui trái

■ Định lý 6.2 (Loại bỏ đệ qui trái)

- Cho $G = (V, T, S, P)$ là một VPPNC. Chia tập các luật sinh mà về trái của chúng là một biến đã cho nào đó (chẳng hạn là A), thành hai tập con riêng biệt

$$A \rightarrow Ax_1 \mid Ax_2 \mid \dots \mid Ax_n \quad (6.2)$$

$$A \rightarrow y_1 \mid y_2 \mid \dots \mid y_m \quad (6.3)$$

với $x_i, y_i \in (V \cup T)^*$, và A không là prefix của bất kỳ y_i nào.

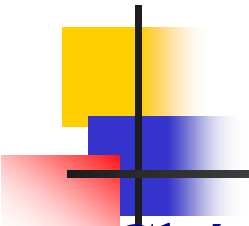
Xét $G_1 = (V \cup \{Z\}, T, S, P_1)$, trong đó $Z \notin V$ và P_1 nhận được bằng cách thay mọi luật sinh của P có dạng (6.2) và (6.3) bởi

$$A \rightarrow y_i \mid y_i Z, i = 1, 2, \dots, m,$$

$$Z \rightarrow x_i \mid x_i Z, i = 1, 2, \dots, n,$$

Thì

$$L(G) = L(G_1).$$



Loại bỏ đệ qui trái (tt)

■ Chứng minh

- Các dạng câu mà A sinh ra trong văn phạm G có dạng:

$$A \xRightarrow{*} A(x_1 + x_2 + \dots + x_n)^* \Rightarrow y_i(x_1 + x_2 + \dots + x_n)^*$$

Các dạng câu này cũng có thể được sinh ra trong G_1 bằng cách chú ý Z có thể sinh ra các dạng câu có dạng

$$Z \xRightarrow{*} (x_1 + x_2 + \dots + x_n)(x_1 + x_2 + \dots + x_n)^*$$

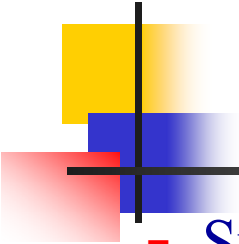
mà $A \rightarrow y_i \mid y_i Z$ nên

$$A \xRightarrow{*} y_i(x_1 + x_2 + \dots + x_n)^*$$

Vì vậy $L(G) = L(G_1)$.

■ Ghi chú

- Các luật sinh đệ qui-trái chỉ là một trường hợp đặc biệt của đệ qui-trái trong văn phạm như được phát biểu sau.
- Một văn phạm được gọi là đệ qui-trái nếu có một biến A nào đó mà đối với nó $A \xRightarrow{*} Ax$ là có thể.



Ví dụ

- Sử dụng Định lý 6.2 để loại bỏ các luật sinh đệ qui-trái khỏi VP

$$A \rightarrow Aa \mid aBc \mid \lambda$$

$$B \rightarrow Bb \mid ba$$

- Áp dụng định lý cho biến A ta được tập luật sinh mới như sau:

$$A \rightarrow aBc \mid \lambda \mid aBcZ \mid Z$$

$$B \rightarrow Bb \mid ba$$

$$Z \rightarrow a \mid aZ$$

- Áp dụng định lý một lần nữa lần này cho biến B ta được tập luật sinh kết quả cuối cùng như sau:

$$A \rightarrow aBc \mid aBcZ \mid Z \mid \lambda$$

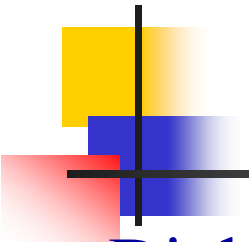
$$B \rightarrow ba \mid baY$$

$$Z \rightarrow a \mid aZ$$

$$Y \rightarrow b \mid bY$$

■ Nhận xét

- Việc loại bỏ các luật sinh đệ qui-trái đưa ra các biến mới. VP kết quả có thể là "đơn giản" hơn đáng kể so với VP gốc nhưng một cách tổng quát nó sẽ có nhiều biến và luật sinh hơn.



Luật sinh vô dụng

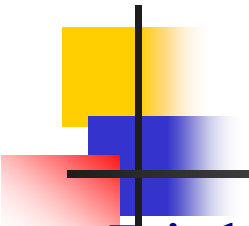
■ Định nghĩa 6.1:

- Cho $G = (V, T, S, P)$ là một VPPNC. Một biến $A \in V$ được gọi là **khả dụng** nếu và chỉ nếu có ít nhất một chuỗi $w \in L(G)$ sao cho
$$S \xRightarrow{*} xAy \xRightarrow{*} w,$$

với $x, y \in (V \cup T)^*$. Bằng lời, một biến là khả dụng nếu và chỉ nếu nó xuất hiện trong ít nhất một dẫn xuất. Một biến mà không khả dụng thì gọi là **vô dụng**. Một luật sinh được gọi là vô dụng nếu nó có chứa bất kỳ biến vô dụng nào.

■ Các dạng vô dụng

- Vô dụng loại 1: $A \not\xRightarrow{*} w \in T^*$
- Vô dụng loại 2: $S \not\xRightarrow{*} xAy$



Loại bỏ các luật sinh vô dụng

■ Định lý 6.3

- Cho $G = (V, T, S, P)$ là một VPPNC, $\exists$ một VP tương đương $G_0 = (V_0, T, S, P_0)$ mà không chứa bất kỳ biến vô dụng nào.

■ Chứng minh

- Loại bỏ các biến và luật sinh vô dụng loại 1

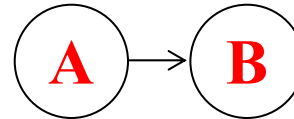
Tạo văn phạm $G_1 = (V_1, T, S, P_1)$ với V_1 là tập biến không vô dụng loại 1. Ta tìm V_1 như sau:

1. Khởi tạo $V_1 = \emptyset$.
2. Lặp lại bước sau cho đến khi không còn biến nào được thêm vào V_1 .
 - Đối với mỗi $A \in V$ mà có luật sinh $A \rightarrow x, x \in (V_1 \cup T)^*$, thì thêm A vào V_1 .
3. Loại khỏi P các luật sinh có chứa các biến $\notin V_1$, ta được P_1 .

Loại bỏ các luật sinh vô dụng (tt)

- Để loại tiếp các biến và các luật sinh vô dụng loại 2 ta dựa vào G_1 vừa có ở trên và vẽ đồ thị phụ thuộc cho nó, sau đó tìm tập các biến không đạt tới được từ S . Loại các biến này và các luật sinh liên quan đến nó ra khỏi G_1 ta được văn phạm kết quả G_0 .
- Đồ thị phụ thuộc (dependency graph)
 - Là một đồ thị có các đỉnh biểu diễn các biến, còn một cạnh nối hai đỉnh A và B khi và chỉ khi có luật sinh dạng

$$A \rightarrow xBy$$



■ Ví dụ

- Loại bỏ các biến và các luật sinh vô dụng ra khỏi văn phạm $G = (\{S, A, B, C\}, \{a, b\}, S, P)$, với tập luật sinh P là:

$$S \rightarrow aS \mid A \mid C$$

$$B \rightarrow aa$$

$$A \rightarrow a$$

$$C \rightarrow aCb$$

Ví dụ

- Loại bỏ các biến vô dụng loại 1 ta được $V_1 = \{S, A, B\}$ và tập luật sinh P_1

$$S \rightarrow aS \mid A \mid C$$

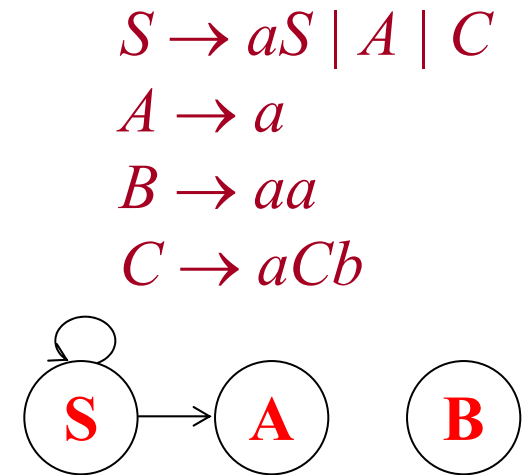
$$A \rightarrow a$$

$$B \rightarrow aa$$

- Loại bỏ các biến vô dụng loại 2 ta được văn phạm kết quả

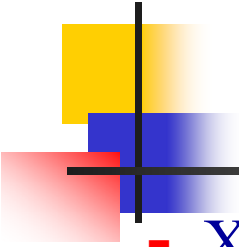
$$S \rightarrow aS \mid A$$

$$A \rightarrow a$$



■ Nhận xét

- Nếu thay đổi thứ tự loại bỏ (loại bỏ các biến và luật sinh vô dụng loại 2 trước) thì sẽ không loại bỏ được tất cả các biến và luật sinh vô dụng chỉ bằng một lần như ví dụ sau cho thấy.



Ví dụ (tt)

- Xét văn phạm sau

$$S \rightarrow aSb \mid ab \mid A$$

$$A \rightarrow aAB$$

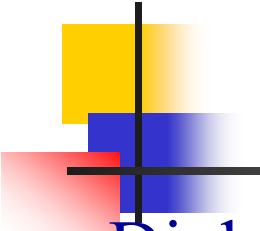
$$B \rightarrow b$$

- Nếu loại bỏ các biến và luật sinh vô dụng loại 2 trước ta thấy văn phạm vẫn không thay đổi vì tất cả các biến đều đạt tới được từ S . Sau đó loại bỏ tiếp các biến và luật sinh vô dụng loại 1 ta sẽ được văn phạm sau:

$$S \rightarrow aSb \mid ab \mid A$$

$$B \rightarrow b$$

- Rõ ràng văn phạm này còn biến B là vô dụng loại 2.



Loại bỏ luật sinh- λ

■ Định nghĩa 6.2

- Bất kỳ luật sinh nào của VPPNC có dạng

$$A \rightarrow \lambda$$

được gọi là luật sinh- λ . Bất kỳ biến A nào mà

$$A \xRightarrow{*} \lambda$$

là có thể thì được gọi là khả trống (nullable).

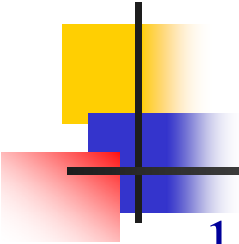
■ Định lý 6.4

- Cho G là một VPPNC bất kỳ mà $L(G)$ không chứa λ , thì tồn tại một văn phạm G_0 tương đương mà không có chứa luật sinh- λ .

■ Chứng minh:

Bước 1

- Tìm tập V_N tất cả các biến khả trống của G bằng các bước sau.

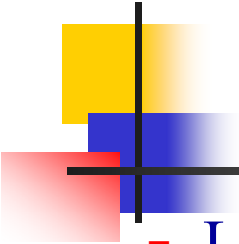


Loại bỏ luật sinh- λ

1. Đối với mọi luật sinh $A \rightarrow \lambda$, đưa A vào V_N .
2. Lặp lại bước sau cho đến khi không còn biến nào được thêm vào V_N .
 - Đối với mọi luật sinh $B \rightarrow A_1 A_2 \dots A_n$, mà $A_1, A_2, A_n \in V_N$ thì đặt B vào V_N .

Bước 2

- Sau khi có tập V_N ta xây dựng tập luật sinh như sau.
- Ứng với mỗi luật sinh có dạng $A \rightarrow x_1 x_2 \dots x_m$, $m \geq 1$, trong đó mỗi $x_i \in V \cup T$, đặt luật sinh này vào cùng với các luật sinh được sinh ra bằng cách thay thế các biến khả trống bằng λ trong **mọi tổ hợp có thể**, ngoại trừ nếu tất cả các x_i đều khả trống thì không đặt luật sinh $A \rightarrow \lambda$ vào P_0 của G_0



Ví dụ

- Loại bỏ các luật sinh- λ của văn phạm sau:

$$S \rightarrow ABaC \qquad C \rightarrow D \mid \lambda$$

$$A \rightarrow BC \qquad D \rightarrow d$$

$$B \rightarrow b \mid \lambda$$

- Vì $B \rightarrow \lambda$ và $C \rightarrow \lambda$ suy ra B và C là các biến khả trống.
- Vì $A \rightarrow BC$ nên suy ra A cũng là biến khả trống. Ngoài ra không còn biến nào khác là khả trống.
- Theo Bước 2 ta xây dựng được tập luật sinh mới tương đương như sau:

$$S \rightarrow ABaC \mid BaC \mid AaC \mid ABa \mid aC \mid Aa \mid Ba \mid a$$

$$A \rightarrow BC \mid B \mid C$$

$$B \rightarrow b$$

$$C \rightarrow D$$

$$D \rightarrow d$$



Loại bỏ luật sinh đơn vị

■ Định nghĩa 6.3

- Bất kỳ luật sinh nào của VPPNC có dạng

$$A \rightarrow B$$

trong đó $A, B \in V$ được gọi là luật sinh-đơn vị.

■ Định lý 6.5

- Cho $G = (V, T, S, P)$ là một VPPNC bất kỳ không có luật sinh- λ , thì tồn tại một VPPNC $G_1 = (V_1, T, S, P_1)$ mà không có bất kỳ luật sinh đơn vị nào và tương đương với G_1 .

■ Chứng minh

1. Đặt vào trong P_1 tất cả các luật sinh không đơn vị của P .
2. Đối với mỗi biến A tìm tất cả các biến B mà $A \xRightarrow{*} B$ (*)
 - Điều này thực hiện bằng cách vẽ đồ thị phụ thuộc cho G nhưng một cạnh nối 2 đỉnh A và B khi và chỉ khi có luật sinh-đơn vị $A \rightarrow B$. Hai biến A và B thỏa (*) khi và chỉ khi có một con đường trong đồ thị đi từ A đến B .

Ví dụ

3. Đối với mỗi A, B thỏa (*) thêm vào trong P_1 các luật sinh

$$A \rightarrow y_1 \mid y_2 \mid \dots \mid y_n$$

với $B \rightarrow y_1 \mid y_2 \mid \dots \mid y_n$ là các luật sinh không đơn vị của B .

■ Ví dụ

- Loại bỏ các luật sinh đơn vị cho VP sau

$$\begin{aligned} S &\rightarrow Aa \mid B \\ B &\rightarrow A \mid bb \\ A &\rightarrow a \mid bc \mid B \end{aligned}$$

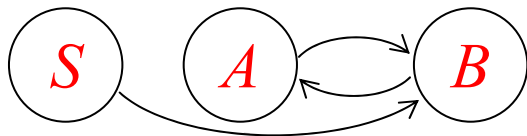


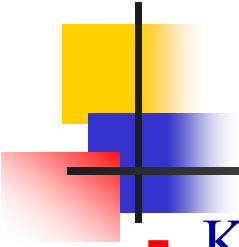
Trước hết, đặt các luật sinh không đơn vị vào trong P_1

$$\begin{aligned} S &\rightarrow Aa \\ A &\rightarrow a \mid bc \\ B &\rightarrow bb \end{aligned}$$

Từ ĐTPT ta đưa được thêm các luật sinh sau vào

$$\begin{aligned} S &\rightarrow a \mid bc \mid bb \\ A &\rightarrow bb \\ B &\rightarrow a \mid bc \end{aligned}$$





Ví dụ (tt)

- Kết quả ta có văn phạm tương đương sau không có luật sinh đơn vị

$$S \rightarrow Aa \mid a \mid bc \mid bb$$

$$A \rightarrow a \mid bc \mid bb$$

$$B \rightarrow bb \mid a \mid bc$$

■ Định lý 6.6

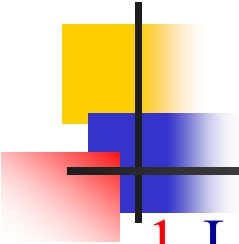
- Cho L là một NNPNC không chứa λ , tồn tại một VPPNC sinh ra L mà không chứa bất kỳ luật sinh vô dụng, luật sinh- λ , hay luật sinh-đơn vị nào.

■ Chứng minh:

B1. Loại bỏ luật sinh- λ

B2. Loại bỏ luật sinh đơn vị

B3. Loại bỏ luật sinh vô dụng loại 1, rồi vô dụng loại 2



Một số nhận xét

1. Loại bỏ biến vô dụng loại 1 có thể sinh ra biến vô dụng loại 2.
2. Việc loại bỏ biến vô dụng loại 2 không sinh ra biến vô dụng loại 1.
3. Văn phạm không có luật sinh đơn vị thì việc loại bỏ luật sinh- λ có thể sinh ra luật sinh-đơn vị.
4. Văn phạm không có luật sinh- λ thì việc loại bỏ luật sinh-đơn vị không thể sinh ra luật sinh- λ mới.
5. Loại bỏ luật sinh- λ có thể sinh ra biến vô dụng loại 1.
6. Loại bỏ luật sinh-đơn vị có thể sinh ra biến vô dụng loại 2.
7. Văn phạm không có luật sinh- λ , luật sinh-đơn vị thì việc loại bỏ các luật sinh vô dụng loại 1, loại 2 không sinh ra thêm bất kỳ luật sinh- λ và luật sinh-đơn vị nào mới.



Dạng chuẩn Chomsky

■ Định nghĩa 6.4

- Một VPPNC là thuộc dạng chuẩn Chomsky nếu mọi luật sinh có dạng

$$A \rightarrow BC, \text{ hoặc}$$

$$A \rightarrow a$$

trong đó $A, B, C \in V$, còn $a \in T$.

■ Định lý 6.7

- Bất kỳ VPPNC $G = (V, T, S, P)$ nào với $\lambda \notin L(G)$ đều có một văn phạm tương đương $G_1 = (V_1, T, S, P_1)$ có dạng chuẩn Chomsky.

■ Chứng minh

- Không mất tổng quát giả sử G không có luật sinh-vô dụng, luật sinh-đơn vị và luật sinh- λ . Ta xây dựng văn phạm G_1 có dạng chuẩn Chomsky bằng thủ tục sau:

Thủ tục: G -to- G_{Chomsky}

- **Input:** $G = (V, T, S, P)$ với $\lambda \notin L(G)$
- **Output:** $G_1 = (V_1, T, S, P_1)$ có dạng chuẩn Chomsky.
 1. Đặt các luật sinh $A \rightarrow a$ vào P_1 .
 2. Đối với các luật sinh $A \rightarrow x_1 x_2 \dots x_n$ với $n \geq 2, x_i \in (V \cup T)$ thì thay các kí hiệu kết thúc, chẳng hạn $x_k = a$, bằng các biến đại diện mới B_a , tạo thành các luật sinh trung gian $A \rightarrow C_1 C_2 \dots C_n$.
 3. Ứng với mỗi biến đại diện B_a đặt vào P_1 các luật sinh $B_a \rightarrow a$.
 4. Sau khi thực hiện bước 2, ứng với mỗi luật sinh $A \rightarrow C_1 C_2 \dots C_n$ mà $n = 2$ đặt nó vào P_1 . Ngược lại ứng với $n > 2$ ta giới thiệu các biến mới $D_1, D_2, \dots$ và đưa vào các luật sinh sau:

$$\begin{array}{lcl} A & \rightarrow & C_1 D_1 \\ D_1 & \rightarrow & D_1 D_2 \\ & \vdots & \\ D_{n-2} & \rightarrow & C_{n-1} C_n \end{array}$$

Ví dụ

- Hãy biến đổi VP sau thành VP có dạng chuẩn Chomsky.

$$\begin{aligned} S &\rightarrow a \mid ABa \\ A &\rightarrow aab \\ B &\rightarrow b \mid Ac \end{aligned}$$

Bước 1

$$\begin{aligned} S &\rightarrow a \\ B &\rightarrow b \end{aligned}$$

Bước 2

$$\begin{aligned} S &\rightarrow ABX_a \\ A &\rightarrow X_aX_aX_b \\ B &\rightarrow AX_c \end{aligned}$$

Bước 3

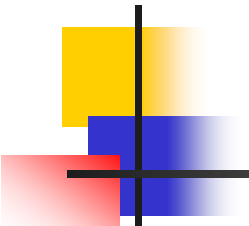
$$\begin{aligned} X_a &\rightarrow a \\ X_b &\rightarrow b \\ X_c &\rightarrow c \end{aligned}$$

Bước 4

$$\begin{aligned} S &\rightarrow AD_1 \\ D_1 &\rightarrow BX_a \\ A &\rightarrow X_aD_2 \\ D_2 &\rightarrow X_aX_b \end{aligned}$$

Kết quả

$$\begin{aligned} S &\rightarrow a \mid AD_1 \\ D_1 &\rightarrow BX_a \\ A &\rightarrow X_aD_2 \\ D_2 &\rightarrow X_aX_b \\ B &\rightarrow b \mid AX_c \\ X_a &\rightarrow a \\ X_b &\rightarrow b \\ X_c &\rightarrow c \end{aligned}$$



Dạng chuẩn Greibach

■ Định nghĩa 6.5

- Một VPPNC là thuộc dạng chuẩn Greibach nếu mọi luật sinh có dạng

$$A \rightarrow ax$$

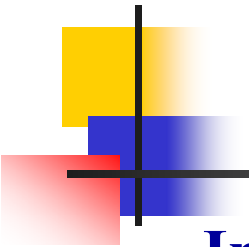
trong đó $a \in T$ còn $x \in V^*$.

■ Định lý 6.8

- Đối với mọi VPPNC G với $\lambda \notin L(G)$, thì tồn tại một văn phạm tương đương trong dạng chuẩn *Greibach*.

■ Chứng minh

- Không mất tính tổng quát giả sử G không có luật sinh-vô dụng, luật sinh-đơn vị và luật sinh- λ . Ta xây dựng văn phạm có dạng chuẩn Greibach bằng thủ tục sau.



Thủ tục: G -to- G_{Greibach}

- **Input:** $G = (V, T, S, P)$ với $\lambda \notin L(G)$
- **Output:** $G_1 = (V_1, T, S, P_1)$ có dạng chuẩn Greibach.
 1. Đánh số thứ tự cho các biến chẳng hạn là $A_1, A_2, \dots, A_n$.
 2. Dùng Định lý 6.1 và 6.2 để viết lại VP sao cho các luật sinh có một trong ba dạng sau
$$\begin{aligned} A_i &\rightarrow A_j x_j, i < j \\ Z_i &\rightarrow A_j x_j, j \leq n \\ A_i &\rightarrow a x_i \end{aligned} \quad \begin{array}{l} a \in T \text{ và } x_i \in (V \cup T)^* \\ Z_i \text{ là các biến mới} \end{array}$$
- Điều này thực hiện được bằng cách sử dụng Định lý 6.1 và 6.2 cho các biến A_i theo thứ tự i đi từ 1, 2, ... đến n như sau.
- Giả sử xét luật sinh của biến A_i . Nếu có luật sinh $A_i \rightarrow A_j x$ mà $i > j$ thì *thay A_j đi đầu* bằng các vế phải của nó, và làm cho đến khi các luật sinh của A_i có dạng $A_i \rightarrow A_j x, i \leq j$. Đến đây loại đệ qui trái cho A_i thì các luật sinh của nó sẽ có dạng như đã nêu.

Thủ tục: $G\text{-to-}G_{\text{Greibach}}$ (tt)

3. Sau khi thực hiện bước 2, tất cả các luật sinh của A_n phải có dạng

$$A_n \rightarrow ax_n$$

- Thay A_n đi đầu về phải của các luật sinh bằng các vế phải của nó. Kết quả các luật sinh của A_{n-1} có dạng

$$A_{n-1} \rightarrow ax_{n-1}$$

- Tương tự thay thế A_{n-1} đi đầu về phải của các luật sinh bằng các vế phải của nó. Và thực hiện lần lượt cho đến A_1 .

4. Thay các kí hiệu kết thúc, chẳng hạn a , không đi đầu về phải bằng các biến đại diện, chẳng hạn X_a , đồng thời thêm vào các luật sinh mới $X_a \rightarrow a$.

■ Ví dụ

- Biến đổi VP sau thành VP có dạng chuẩn Greibach

$$S \rightarrow SBb \mid Ab$$

$$A \rightarrow Sb \mid Ba$$

$$B \rightarrow Sa \mid b$$

Ví dụ

$$\begin{aligned} S &\rightarrow SBb \mid Ab \\ A &\rightarrow Sb \mid Ba \\ B &\rightarrow Sa \mid b \end{aligned}$$

$$S_0 \rightarrow S_0B_2b \mid A_1b$$

Loại đệ qui trái

$$\begin{aligned} S_0 &\rightarrow A_1b \mid A_1bZ_0 \quad (1) \\ Z_0 &\rightarrow B_2b \mid B_2bZ_0 \quad (2) \end{aligned}$$

$$A_1 \rightarrow S_0b \mid B_2a$$

Thay thế

$$A_1 \rightarrow A_1bb \mid A_1bZ_0b \mid B_2a$$

Loại đệ qui trái

$$\begin{aligned} A_1 &\rightarrow B_2a \mid B_2aZ_1 \quad (3) \\ Z_1 &\rightarrow bb \mid bZ_0b \mid bbZ_1 \mid bZ_0bZ_1 \quad (4) \end{aligned}$$

$$B_2 \rightarrow S_0a \mid b$$

Thay thế

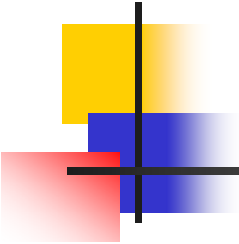
$$B_2 \rightarrow A_1ba \mid A_1bZ_0a \mid b$$

Thay thế

$$\begin{aligned} B_2 &\rightarrow b \mid bZ_2 \quad (5) \\ Z_2 &\rightarrow aba \mid aZ_1ba \mid abZ_0a \mid \\ &\quad aZ_1bZ_0a \mid abaZ_2 \mid aZ_1baZ_2 \mid \\ &\quad abZ_0aZ_2 \mid aZ_1bZ_0aZ_2 \quad (6) \end{aligned}$$

$$\begin{aligned} B_2 &\rightarrow B_2aba \mid B_2aZ_1ba \mid B_2abZ_0a \mid \\ &\quad B_2aZ_1bZ_0a \mid b \end{aligned}$$

Loại đệ qui trái



Ví dụ (tt)

$$B_2 \rightarrow b \mid bZ_3 \quad (5)$$

$$A_1 \rightarrow B_2a \mid B_2aZ_1 \quad (3) \xrightarrow{\text{Thay thế}} A_1 \rightarrow ba \mid bZ_2a \mid baZ_1 \mid bZ_2aZ_1 \quad (7)$$

$$S_0 \rightarrow A_1b \mid A_1bZ_0 \quad (1) \xrightarrow{\text{Thay thế}} S_0 \rightarrow bab \mid bZ_2ab \mid baZ_1b \mid \\ bZ_2aZ_1b \mid babZ_0 \mid bZ_2abZ_0 \mid \\ baZ_1bZ_0 \mid bZ_2aZ_1bZ_0 \quad (8)$$

$$Z_0 \rightarrow B_2b \mid B_2bZ_0 \quad (2) \xrightarrow{\text{Thay thế}} Z_0 \rightarrow bb \mid bZ_2b \mid bbZ_0 \mid bZ_2bZ_0 \quad (9)$$

Ví dụ (tt)

Văn phạm gần-
Greibach

$$S_0 \rightarrow bab \mid bZ_2ab \mid baZ_1b \mid bZ_2aZ_1b \mid babZ_0 \mid bZ_2abZ_0 \mid baZ_1bZ_0 \mid bZ_2aZ_1bZ_0 \quad (8)$$

$$A_1 \rightarrow ba \mid bZ_2a \mid baZ_1 \mid bZ_2aZ_1 \quad (7)$$

$$B_2 \rightarrow b \mid bZ_3 \quad (5)$$

$$Z_0 \rightarrow bb \mid bZ_2b \mid bbZ_0 \mid bZ_2bZ_0 \quad (9)$$

$$Z_1 \rightarrow bb \mid bZ_0b \mid bbZ_1 \mid bZ_0bZ_1 \quad (4)$$

$$Z_2 \rightarrow aba \mid aZ_1ba \mid abZ_0a \mid aZ_1bZ_0a \mid abaZ_2 \mid aZ_1baZ_2 \mid abZ_0aZ_2 \mid aZ_1bZ_0aZ_2 \quad (6)$$

Thay kí hiệu kết thúc không đi đầu bằng biến đại diện

$$S \rightarrow bXY \mid bZ_2XY \mid bXZ_1Y \mid bZ_2XZ_1Y \mid bXYZ_0 \mid bZ_2XYZ_0 \mid bXZ_1YZ_0 \mid bZ_2XZ_1YZ_0$$

$$A \rightarrow bX \mid bZ_2X \mid bXZ_1 \mid bZ_2XZ_1$$

$$B \rightarrow b \mid bZ_3$$

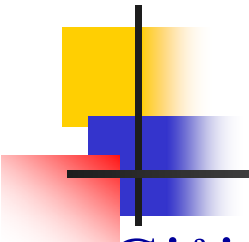
$$Z_0 \rightarrow bY \mid bZ_2Y \mid bYZ_0 \mid bZ_2YZ_0$$

$$Z_1 \rightarrow bY \mid bZ_0Y \mid bYZ_1 \mid bZ_0YZ_1$$

$$Z_2 \rightarrow aYX \mid aZ_1YX \mid aYZ_0X \mid aZ_1YZ_0X \mid aYXZ_2 \mid aZ_1YXZ_2 \mid aYZ_0XZ_2 \mid aZ_1YZ_0XZ_2$$

$$X \rightarrow a$$

$$Y \rightarrow b$$



Giải thuật thành viên cho VPPNC

- Giải thuật CYK (J.Cocke, D.H.Younger, T.Kasami)

- **Input**: Văn phạm Chomsky $G = (V, T, S, P)$

Chuỗi $w = a_1 a_2 \dots a_n$

- **Output**: “Yes” + DXTN hoặc “No”.

- Chúng ta định nghĩa các chuỗi con

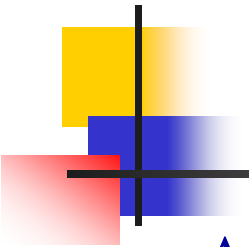
$$w_{ij} = a_i \dots a_j,$$

- Và các tập con của V

$$V_{ij} = \{A \in V : A \xRightarrow{*} w_{ij}\},$$

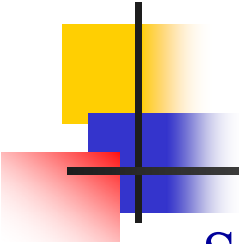
- Để ý $w = w_{1n}$, vậy $w \in L(G)$ khi và chỉ khi $S \in V_{1n}$.

- *Vậy để biết $w \in L(G)$ hay không chúng ta tính V_{1n} và xem $S \in V_{1n}$ hay không.*



Giải thuật CYK

- $A \in V_{ii}$ nếu và chỉ nếu A có luật sinh $A \rightarrow a_i$.
- Vậy, V_{ii} có thể được tính $\forall i, 1 \leq i \leq n$.
- Nếu $B \xRightarrow{*} w_{ik} (\Leftrightarrow B \in V_{ik})$, $C \xRightarrow{*} w_{(k+1)j} (\Leftrightarrow C \in V_{(k+1)j})$ và đồng thời $A \rightarrow BC$ thì $A \xRightarrow{*} w_{ij} (\Leftrightarrow A \in V_{ij}) \forall i \leq k, k < j$.
- $V_{ij} = \cup_{k \in \{i, i+1, \dots, j-1\}} \{A: A \rightarrow BC, \text{ với } B \in V_{ik}, C \in V_{(k+1)j}\}$
- Quá trình tính các tập V_{ij}
- $V_{11}, V_{22}, \dots, V_{nn}$
- $V_{12}, V_{23}, \dots, V_{(n-1)n}$
- $V_{13}, V_{24}, \dots, V_{(n-2)n}$
- ...
- V_{1n}



Ví dụ

- Sử dụng giải thuật CYK để PTCP chuỗi $w = aabbb$ trên G sau

$$S \rightarrow AB \quad (1)$$

$$A \rightarrow BB \mid a \quad (2, 3)$$

$$B \rightarrow AB \mid b \quad (4, 5)$$

- Ta có $w = a \ a \ b \ b \ b$

1 2 3 4 5

- $V_{11} = \{A\}, V_{22} = \{A\}, V_{33} = \{B\}, V_{44} = \{B\}, V_{55} = \{B\},$
- $V_{12} = \emptyset, V_{23} = \{S, B\}, V_{34} = \{A\}, V_{45} = \{A\},$
- $V_{13} = \{S, B\}, V_{24} = \{A\}, V_{35} = \{S, B\},$
- $V_{14} = \{A\}, V_{25} = \{S, B\},$
- $V_{15} = \{S, B\}.$ $S \in V_{15} \Rightarrow w \in L(G).$

Ví dụ (tt)

$$S \rightarrow AB \quad (1)$$

$$A \rightarrow BB \mid a \quad (2, 3)$$

$$B \rightarrow AB \mid b \quad (4, 5)$$

$$w = a \ a \ b \ b \ b$$

1 2 3 4 5

- Để tìm dẫn xuất cho w , chúng ta phải tìm cách “lưu vết”
- $V_{11} = \{A(\overset{3}{\rightarrow} a)\}$, $V_{22} = \{A(\overset{3}{\rightarrow} a)\}$, $V_{33} = \{B(\overset{5}{\rightarrow} b)\}$,
 $V_{44} = \{B(\overset{5}{\rightarrow} b)\}$, $V_{55} = \{B(\overset{5}{\rightarrow} b)\}$,
- $V_{12} = \emptyset$, $V_{23} = \{S(\overset{1}{\rightarrow} A_{22}B_{33}), B(\overset{4}{\rightarrow} A_{22}B_{33})\}$,
 $V_{34} = \{A(\overset{2}{\rightarrow} B_{33}B_{44})\}$, $V_{45} = \{A(\overset{2}{\rightarrow} B_{44}B_{55})\}$,
- $V_{13} = \{S(\overset{1}{\rightarrow} A_{11}B_{23}), B(\overset{4}{\rightarrow} A_{11}B_{23})\}$, $V_{24} = \{A(\overset{2}{\rightarrow} B_{23}B_{44})\}$,
 $V_{35} = \{S(\overset{1}{\rightarrow} A_{34}B_{55}), B(\overset{4}{\rightarrow} A_{34}B_{55})\}$,
- $V_{14} = \{A(\overset{2}{\rightarrow} B_{13}B_{44})\}$, $V_{25} = \{S(\overset{1}{\rightarrow} A_{22}B_{35}, \overset{1}{\rightarrow} A_{24}B_{55}),$
 $B(\overset{4}{\rightarrow} A_{22}B_{35}, \overset{4}{\rightarrow} A_{24}B_{55})\}$,
- $V_{15} = \{S(\overset{1}{\rightarrow} A_{11}B_{25}, \overset{1}{\rightarrow} A_{14}B_{55}), B(\overset{4}{\rightarrow} A_{11}B_{25}, \overset{4}{\rightarrow} A_{14}B_{55})\}$.

Ví dụ (tt)

$$S \rightarrow AB \quad (1)$$

$$A \rightarrow BB \mid a \quad (2, 3)$$

$$B \rightarrow AB \mid b \quad (4, 5)$$

$$w = a a b b b$$

1 2 3 4 5

- Kết quả có 3 DXTN như sau:

$$(1) S \xRightarrow{1} A_{11}B_{25} \xRightarrow{3} aB_{25} \xRightarrow{4} aA_{22}B_{35} \xRightarrow{3} aaB_{35} \xRightarrow{4} aaA_{34}B_{55} \xRightarrow{2} aaB_{33}B_{44}B_{55} \xRightarrow{5,5,5} aabbb \text{ (DXTN: 134342555)}$$

$$(2) S \xRightarrow{1} A_{11}B_{25} \xRightarrow{3} aB_{25} \xRightarrow{4} aA_{24}B_{55} \xRightarrow{2} aB_{23}B_{44}B_{55} \xRightarrow{4} aA_{22}B_{33}B_{44}B_{55} \xRightarrow{3,5,5,5} aabbb \text{ (DXTN: 134243555)}$$

$$(3) S \xRightarrow{1} A_{14}B_{55} \xRightarrow{2} B_{13}B_{44}B_{55} \xRightarrow{4} A_{11}B_{23}B_{44}B_{55} \xRightarrow{3} aB_{23}B_{44}B_{55} \xRightarrow{4} aA_{22}B_{33}B_{44}B_{55} \xRightarrow{3,5,5,5} aabbb \text{ (DXTN: 124343555)}$$

Ví dụ (tt)

$$S \rightarrow AB \quad (1)$$

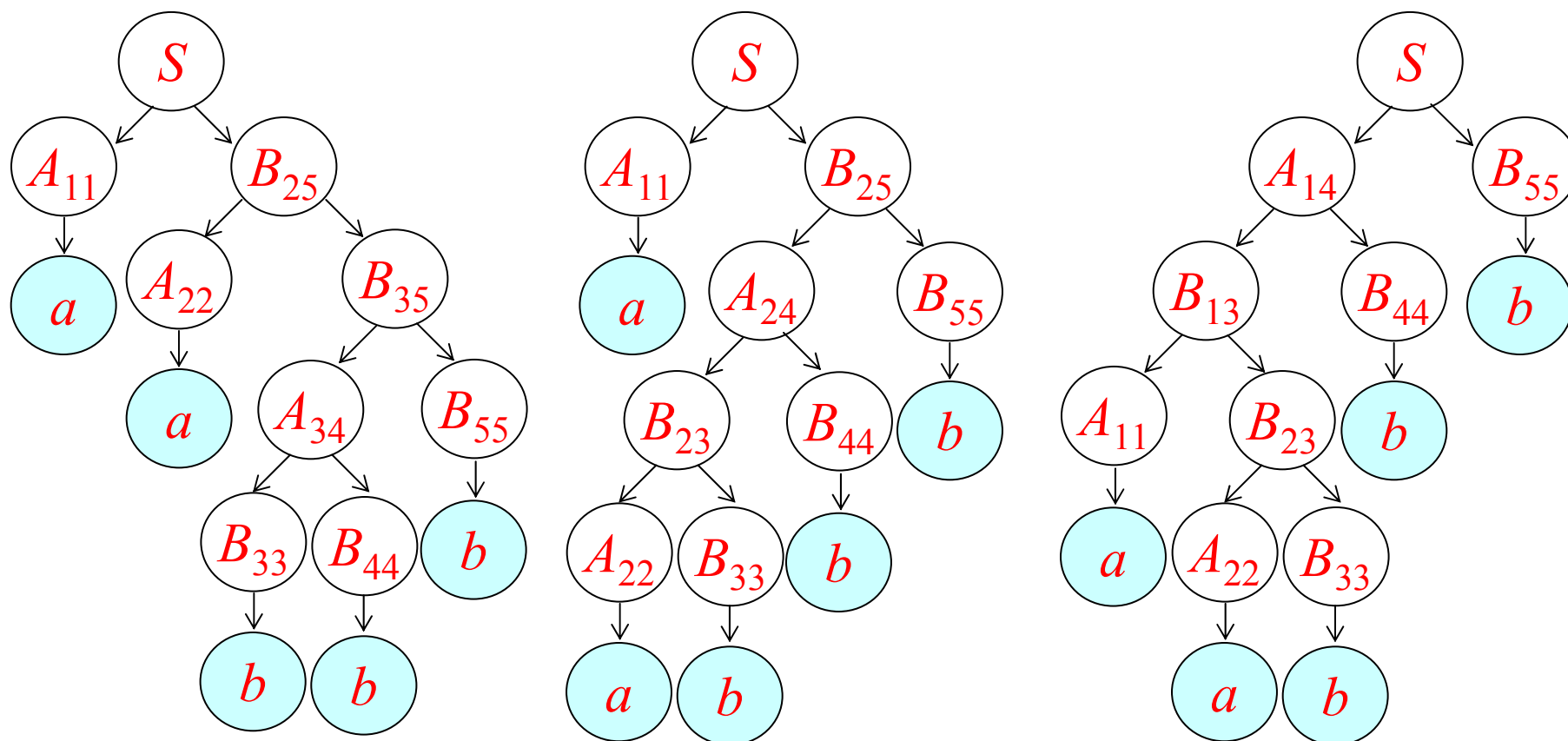
$$A \rightarrow BB \mid a \quad (2, 3)$$

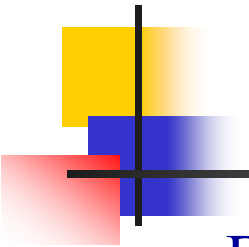
$$B \rightarrow AB \mid b \quad (4, 5)$$

$$w = a a b b b$$

1 2 3 4 5

■ 3 CDX tương ứng:





Bài tập

- Dùng giải thuật CYK PTCP các chuỗi sau $w_1 = abab$, $w_2 = abaa$ trên các VP G_1 , G_2 tương ứng.

G_1

$S \rightarrow AB \mid BB \quad (1, 2)$

$A \rightarrow BA \mid a \quad (3, 4)$

$B \rightarrow AA \mid a \mid b \quad (5, 6, 7)$

G_2

$S \rightarrow AB \quad (1)$

$A \rightarrow BB \mid a \quad (2, 3)$

$B \rightarrow BA \mid b \quad (4, 5)$



Chương 7 Ôtômát đẩy xuống

- Có hay không lớp ôtômát tương ứng với lớp NNPNC?
- Như đã biết, ôtômát hữu hạn không thể nhận biết tất cả NNPNC, chẳng hạn $L = \{a^n b^n : n \geq 0\}$, vì nó có một bộ nhớ hữu hạn. Vì vậy chúng ta muốn có một máy mà **đếm không giới hạn**.
- Từ ví dụ ngôn ngữ $\{ww^R\}$, chúng ta cần thêm khả năng **lưu và so trùng một dãy kí hiệu trong thứ tự ngược lại**.
- Điều này đề nghị chúng ta thử một **stack** như một cơ chế lưu trữ. Đó chính là lớp **ôtômát đẩy xuống (PushDown Automata - PDA)**



Chương 7 Ôtômat đẩy xuống

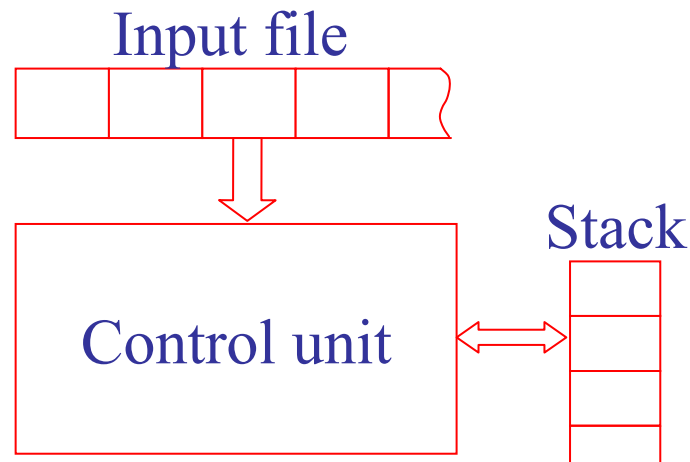
7.1 PDA không đơn định

7.2 NPDA và NNPNC

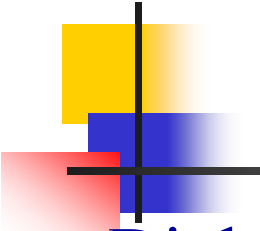
7.3 PDA đơn định và NNPNC đơn định

7.4 Văn phạm cho NNPNC đơn định

Ôtômat đẩy xuống không đơn định



- Mỗi di chuyển của đơn vị điều khiển đọc một kí hiệu nhập, trong cùng thời điểm đó thay đổi nội dung của stack.
- Mỗi di chuyển được xác định bằng **kí hiệu nhập hiện tại, kí hiệu hiện tại trên đỉnh của stack**. Kết quả là một trạng thái mới của đơn vị điều khiển và một **sự thay đổi trên đỉnh của stack**.
- Chúng ta sẽ chỉ nghiên cứu các PDA thuộc loại accepter.



Định nghĩa ôtômát đẩy xuống

■ Định nghĩa 7.1

Một accepter đẩy xuống không đơn định (npda) được định nghĩa bằng bộ bảy $M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$, trong đó

- Q là tập hữu hạn các trạng thái nội của đơn vị điều khiển,
- Σ là bảng chữ cái ngõ nhập (input alphabet),
- Γ là bảng chữ cái stack (stack alphabet),
- $q_0 \in Q$ là trạng thái khởi đầu của đơn vị điều khiển,
- $z \in \Gamma$ là kí hiệu khởi đầu stack (stack start symbol),
- $F \subseteq Q$ là tập các trạng thái kết thúc.
- Hàm chuyển trạng thái δ là một ánh xạ

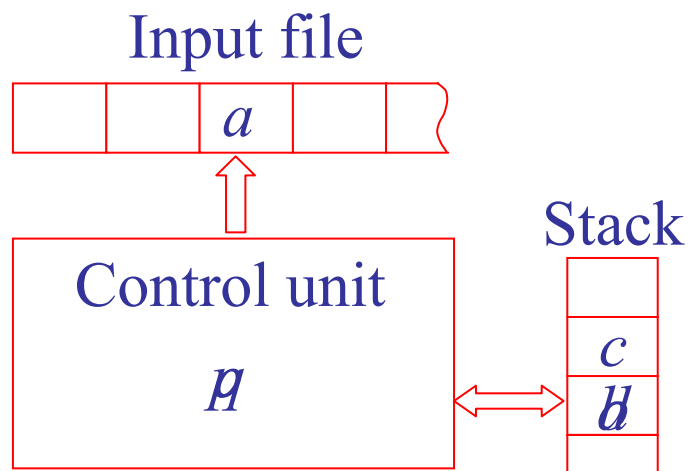
$$\delta : Q \times (\Sigma \cup \{\lambda\}) \times \Gamma \rightarrow \text{tập con hữu hạn của } Q \times \Gamma^*,$$

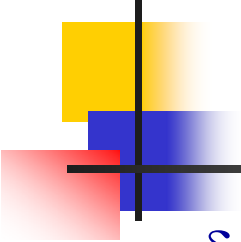
Ví dụ

- $\delta(q, a, b) = \{(p, cd)\}$

- Ví dụ

- Xét một npda với
- $Q = \{q_0, q_1, q_2, q_f\}$,
- $\Sigma = \{a, b\}$,
- $\Gamma = \{0, 1, z\}$,
- $F = \{q_f\}$,





Nhận xét

- $\delta(q_0, a, z) = \{(q_1, 1z), (q_f, \lambda)\}, \quad \delta(q_0, \lambda, z) = \{(q_f, \lambda)\},$
- $\delta(q_1, a, 1) = \{(q_1, 11)\}, \quad \delta(q_1, b, 1) = \{(q_2, \lambda)\},$
- $\delta(q_2, b, 1) = \{(q_2, \lambda)\}, \quad \delta(q_2, \lambda, z) = \{(q_f, \lambda)\}.$
- $\delta(q_0, b, 0)$ không được định nghĩa tương đương với cấu hình chết mà ta đã học.
- $\delta(q_1, a, 1) = \{(q_1, 11)\}$ thêm một kí hiệu 1 vào stack khi a được đọc.
- $\delta(q_2, b, 1) = \{(q_2, \lambda)\}$ xóa một kí hiệu 1 khỏi stack khi b được đọc.
- $L(M) = \{a^n b^n : n \geq 0\} \cup \{a\}$



Một số khái niệm

■ Hình trạng tức thời

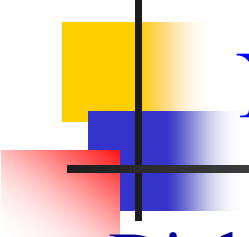
- Là bộ ba (q, w, u) , trong đó q là trạng thái của đơn vị điều khiển, w là phần chưa đọc của chuỗi nhập, còn u là nội dung của stack (với kí hiệu trái nhất là kí hiệu đỉnh của stack).

■ Di chuyển, $\vdash$

- Một di chuyển từ một hình trạng tức thời này đến một hình trạng tức thời khác sẽ được kí hiệu bằng $\vdash$.
- $(q_1, aw, bx) \vdash (q_2, w, yx)$ là có khả năng $\Leftrightarrow (q_2, y) \in \delta(q_1, a, b)$.

■ $\vdash^*$, $\vdash^+$, $\vdash^M$

- Dấu $*$ chỉ ra có ≥ 0 bước di chuyển được thực hiện còn dấu $+$ chỉ ra ≥ 1 bước di chuyển. Chữ M chỉ ra di chuyển của ô tô măt nào.



Ngôn ngữ được chấp nhận bởi một npda

■ Định nghĩa 7.2

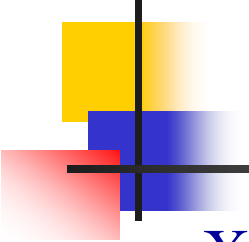
- Cho $M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$ là một npda. Ngôn ngữ được chấp nhận bởi M là tập

$$L(M) = \{w \in \Sigma^*: (q_0, w, z) \vdash^* (q_f, \lambda, u), q_f \in F, u \in \Gamma^*\}.$$

■ Ví dụ

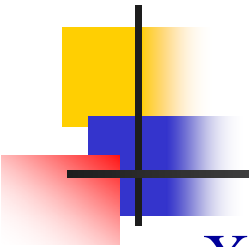
- Xây dựng một npda cho ngôn ngữ

$$L = \{w \in \{a, b\}^*: n_a(w) = n_b(w)\}$$



Ví dụ

- Xây dựng npda cho ngôn ngữ này như sau.
- $M = (\{q_0, q_f\}, \{a, b\}, \{0, 1, z\}, \delta, q_0, z, \{q_f\})$.
 $\delta(q_0, \lambda, z) = \{(q_f, z)\},$
 $\delta(q_0, a, z) = \{(q_0, 0z)\}, \quad \delta(q_0, b, z) = \{(q_0, 1z)\},$
 $\delta(q_0, a, 0) = \{(q_0, 00)\}, \quad \delta(q_0, b, 0) = \{(q_0, \lambda)\},$
 $\delta(q_0, a, 1) = \{(q_0, \lambda)\}, \quad \delta(q_0, b, 1) = \{(q_0, 11)\},$
- Trong quá trình xử lý chuỗi *baab*, npda thực hiện các di chuyển sau.
- $(q_0, baab, z) \vdash (q_0, aab, 1z) \vdash (q_0, ab, z) \vdash (q_0, b, 0z) \vdash (q_0, \lambda, z) \vdash (q_f, \lambda, z)$



Ví dụ (tt)

- Xây dựng npda cho ngôn ngữ

$$L = \{ww^R: w \in \{a, b\}^+\}$$

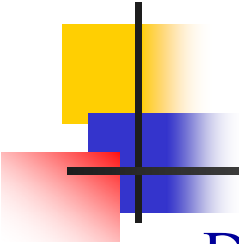
- $M = (\{q_0, q_1, q_f\}, \{a, b\}, \{a, b, z\}, \delta, q_0, z, \{q_f\})$.

$$\begin{aligned}\delta(q_0, a, z) &= \{(q_0, az)\}, \\ \delta(q_0, b, z) &= \{(q_0, bz)\}, \\ \delta(q_0, a, a) &= \{(q_0, aa)\}, \\ \delta(q_0, b, a) &= \{(q_0, ba)\}, \\ \delta(q_0, a, b) &= \{(q_0, ab)\}, \\ \delta(q_0, b, b) &= \{(q_0, bb)\}.\end{aligned}$$


$$\begin{aligned}\delta(q_0, \lambda, a) &= \{(q_1, a)\}, \\ \delta(q_0, \lambda, b) &= \{(q_1, b)\},\end{aligned}$$


$$\begin{aligned}\delta(q_1, a, a) &= \{(q_1, \lambda)\}, \\ \delta(q_1, b, b) &= \{(q_1, \lambda)\},\end{aligned}$$


$$\delta(q_1, \lambda, z) = \{(q_f, z)\}.$$



Ví dụ (tt)

- Dãy chuyển hình trạng để chấp nhận chuỗi *abba* là.

$$(q_0, abba, z) \vdash (q_0, bba, az) \vdash (q_0, ba, baz) \vdash (q_1, ba, baz) \\ \vdash (q_1, a, az) \vdash (q_1, \lambda, z) \vdash (q_f, \lambda, z).$$

- Npda cải tiến

$$\delta(q_0, a, z) = \{(q_0, aa)\},$$

$$\delta(q_1, a, a) = \{(q_1, \lambda)\},$$

$$\delta(q_0, b, z) = \{(q_0, bz)\},$$

$$\delta(q_1, b, b) = \{(q_1, \lambda)\},$$

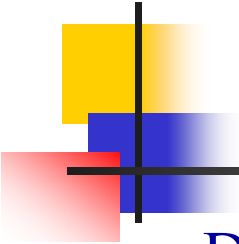
$$\delta(q_0, a, a) = \{(q_0, aa), (q_1, \lambda)\},$$

$$\delta(q_1, \lambda, z) = \{(q_f, z)\}.$$

$$\delta(q_0, b, a) = \{(q_0, ba)\},$$

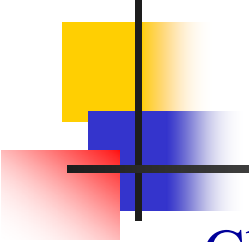
$$\delta(q_0, a, b) = \{(q_0, ab)\},$$

$$\delta(q_0, b, b) = \{(q_0, bb), (q_1, \lambda)\}.$$



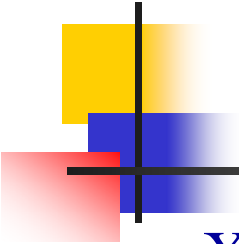
Bài tập

- Dãy chuyển hình trạng để chấp nhận chuỗi *abba* là.
 $(q_0, abba, z) \vdash (q_0, bba, az) \vdash (q_0, ba, baz) \vdash (q_1, a, az)$
 $\vdash (q_1, \lambda, z) \vdash (q_f, \lambda, z).$
- Xây dựng npda cho các ngôn ngữ sau
 - $L_1 = \{a^n b^m c^{n+m} : n, m \geq 0\}$
 - $L_2 = \{a^n b^{n+m} c^m : n, m \geq 1\}$
 - $L_3 = \{a^n b^m : 2n \leq m \leq 3n\}$
 - $L_4 = \{w : n_a(w) = n_b(w) + 2\}$
 - $L_5 = \{w : n_a(w) = 2n_b(w)\}$
 - $L_6 = \{w : 2n_b(w) \leq n_a(w) \leq 3n_b(w)\}$



Ôtômát đẩy xuống cho NNPNC

- Chúng ta xây dựng một npda mà có thể thực hiện được (**mô phỏng**) một **DXTN** của một chuỗi bất kỳ trong ngôn ngữ.
- Giả thiết ngôn ngữ được sinh ra bởi một văn phạm có dạng chuẩn Greibach.
- Pda sắp xây dựng sẽ biểu diễn sự dẫn xuất bằng cách như sau.
- Giữ *các biến trong phần bên phải của dạng câu* trên stack của nó, *còn phần bên trái, chuỗi chứa các kí hiệu kết thúc*, là giống với phần chuỗi đã được đọc ở ngõ nhập.
- Chúng ta bắt đầu bằng việc đặt kí hiệu khởi đầu lên stack.
- Để mô phỏng việc áp dụng luật sinh $A \rightarrow ax$, chúng ta phải có *biến A trên đỉnh stack* và *kí hiệu kết thúc a là kí hiệu nhập*.
- Biến trên đỉnh stack *được loại bỏ và thay thế bằng chuỗi biến x* .



Ví dụ

- Xây dựng npda cho ngôn ngữ được sinh ra bởi văn phạm sau.

$$S \rightarrow aSbb \mid a.$$

- Đầu tiên ta biến đổi văn phạm này sang dạng chuẩn Greibach, thành văn phạm là:

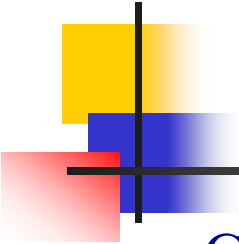
$$S \rightarrow aSA \mid a,$$

$$A \rightarrow bB,$$

$$B \rightarrow b.$$

- Automat tương ứng sẽ có ba trạng thái $\{q_0, q_1, q_f\}$, với trạng thái khởi đầu là q_0 và trạng thái kết thúc là q_f .
- Đầu tiên, ở trạng thái khởi đầu biến S được đặt trên stack bằng

$$\delta(q_0, \lambda, z) = \{(q_1, Sz)\}$$



Ví dụ (tt)

- Các luật sinh $S \rightarrow aSA \mid a$ được mô phỏng thành

$$\delta(q_1, a, S) = \{(q_1, SA), (q_1, \lambda)\}$$

- Bảng kiểu tương tự, các luật sinh khác được mô phỏng bằng

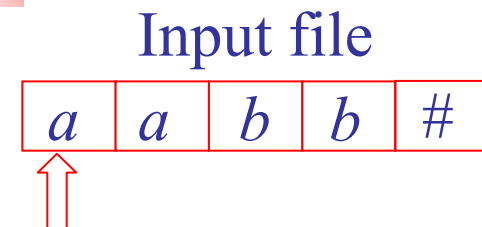
$$\delta(q_1, b, A) = \{(q_1, B)\},$$

$$\delta(q_1, b, B) = \{(q_1, \lambda)\}$$

- Sự xuất hiện kí hiệu khởi đầu stack trên đỉnh stack báo hiệu sự hoàn tất của dẫn xuất và PDA sẽ được đặt vào trong trạng thái kết thúc của nó bằng chuyển trạng thái

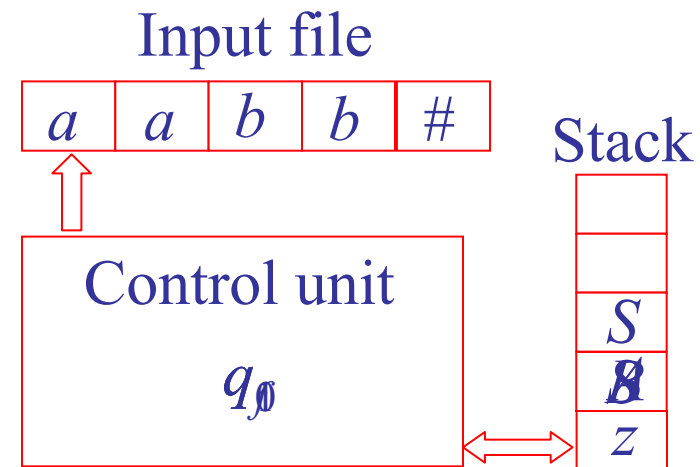
$$\delta(q_1, \lambda, z) = \{(q_f, \lambda)\}.$$

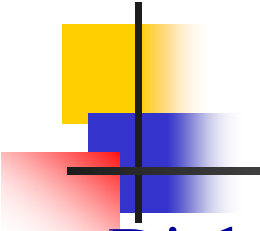
Ví dụ (tt)



$\bullet S \Rightarrow a \bullet SA$
 $\Rightarrow aa \bullet A$
 $\Rightarrow aab \bullet B$
 $\Rightarrow aabb \bullet$

$S \rightarrow aSA \mid a,$
 $A \rightarrow bB,$
 $B \rightarrow b.$





Định lý

■ Định lý 7.1

- Đối với một NNPNC bất kỳ không chứa λ , tồn tại một npda M sao cho

$$L = L(M).$$

■ Thủ tục: G_{Greibach} -to-npda

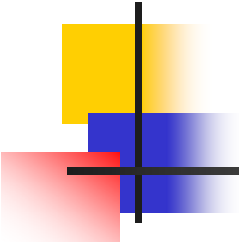
- Input: $G = (V, T, S, P)$ có dạng chuẩn Greibach
- Output: npda $M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$ sao cho $L(M) = L(G)$.

B1 $M = (\{q_0, q_1, q_f\}, T, V \cup \{z\}, \delta, q_0, z, \{q_f\}), z \notin V.$

B2 $\delta(q_0, \lambda, z) = \{(q_1, Sz)\}$

B3 $\delta(q_1, a, A) \ni \{(q_1, u)\} \Leftrightarrow P \text{ có luật sinh } A \rightarrow au$

B4 $\delta(q_1, \lambda, z) = \{(q_f, z)\}$



Ví dụ

$S \rightarrow aA,$
 $A \rightarrow aABC \mid bB \mid a,$
 $B \rightarrow b,$
 $C \rightarrow c.$

$w = aaabc$

$\bullet S$

$\Rightarrow a \bullet A$

$\Rightarrow aa \bullet ABC$

$\Rightarrow aaa \bullet BC$

$\Rightarrow aaab \bullet C$

$\Rightarrow aaabc \bullet$

$\delta(q_0, \lambda, z) = \{(q_1, Sz)\},$

$\delta(q_1, a, S) = \{(q_1, A)\},$

$\delta(q_1, a, A) = \{(q_1, ABC), (q_1, \lambda)\},$

$\delta(q_1, b, A) = \{(q_1, B)\},$

$\delta(q_1, b, B) = \{(q_1, \lambda)\},$

$\delta(q_1, c, C) = \{(q_1, \lambda)\},$

$\delta(q_1, \lambda, z) = \{(q_f, z)\}.$

$(q_0, aaabc, z) \vdash (q_1, aaabc, Sz)$

$\vdash (q_1, aabc, Az)$

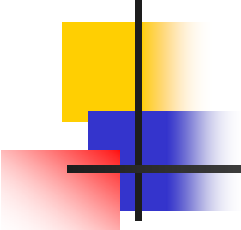
$\vdash (q_1, abc, ABCz)$

$\vdash (q_1, bc, BCz)$

$\vdash (q_1, c, Cz)$

$\vdash (q_1, \lambda, z)$

$\vdash (q_f, \lambda, z).$



Thủ tục *G-to-npda* cải tiến

■ Thủ tục: *G-to-npda*

- Input: $G = (V, T, S, P)$ có dạng tùy ý
- Output: npda $M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$ sao cho $L(M) = L(G)$.
- B1 $M = (\{q_0, q_1, q_f\}, T, V \cup T \cup \{z\}, \delta, q_0, z, \{q_f\}), z \notin V$.
- B2 $\delta(q_0, \lambda, z) = \{(q_1, Sz)\}$
- B3 $\delta(q_1, a, A) \ni \{(q_1, u)\} \Leftrightarrow P$ có luật sinh $A \rightarrow au, a \in T$
- B4 $\delta(q_1, \lambda, A) \ni \{(q_1, u)\} \Leftrightarrow P$ có luật sinh $A \rightarrow u$ và u không có kí hiệu kết thúc đi đầu.
- B5 $\delta(q_1, a, a) = (q_1, \lambda)$ với $a \in T$ và a xuất hiện trong một vế phải luật sinh nào đó mà không phải ở vị trí khởi đầu.
- B6 $\delta(q_1, \lambda, z) = \{(q_f, z)\}$

Ví dụ

$$\begin{aligned} S &\rightarrow aA \mid bBbS \mid AB, \\ A &\rightarrow aB \mid bBaA, \\ B &\rightarrow aBbB \mid SB \mid \lambda \end{aligned}$$

$$\begin{aligned} \delta(q_0, \lambda, z) &= \{(q_1, Sz)\}, \\ \delta(q_1, a, S) &= \{(q_1, A)\}, \\ \delta(q_1, b, S) &= \{(q_1, BbS)\}, \\ \delta(q_1, \lambda, S) &= \{(q_1, AB)\}, \\ \delta(q_1, a, A) &= \{(q_1, B)\}, \\ \delta(q_1, b, A) &= \{(q_1, BaA)\}, \\ \delta(q_1, a, B) &= \{(q_1, BbB)\}, \\ \delta(q_1, \lambda, B) &= \{(q_1, SB), (q_1, \lambda)\}, \\ \delta(q_1, a, a) &= \{(q_1, \lambda)\}, \\ \delta(q_1, b, b) &= \{(q_1, \lambda)\}, \\ \delta(q_1, \lambda, z) &= \{(q_f, z)\}. \end{aligned}$$

$$\begin{aligned} w &= baabaa \\ \bullet S & \\ \Rightarrow b \bullet BbS & \\ \Rightarrow b \bullet SBbS & \\ \Rightarrow ba \bullet ABbS & \\ \Rightarrow & \\ baa \bullet BBbS & \\ \Rightarrow baa \bullet BbS & \\ \Rightarrow baab \bullet S & \\ \Rightarrow baaba \bullet A & \\ \Rightarrow baabaa \bullet B & \\ \Rightarrow baabaa \bullet & \end{aligned}$$

$$\begin{aligned} (q_0, baabaa, z) & \\ \vdash (q_1, baabaa, Sz) & \\ \vdash (q_1, aabaa, BbSz) & \\ \vdash (q_1, aabaa, SBbSz) & \\ \vdash (q_1, abaa, ABbSz) & \\ \vdash (q_1, baa, BBbSz) & \\ \vdash (q_1, baa, BbSz) & \\ \vdash (q_1, baa, bSz) & \\ \vdash (q_1, aa, Sz) & \\ \vdash (q_1, a, Az) & \\ \vdash (q_1, \lambda, Bz) & \\ \vdash (q_1, \lambda, z) & \\ \vdash (q_f, \lambda, z). & \end{aligned}$$

VPPNC cho ô tô măt đẩỵ xuống

- Quá trình này ngược với quá trình trong Định lý 7.1, tức là xây dựng văn phạm mô phỏng sự di chuyển của pda.
- Phần biến của dạng câu phản ánh nội dung stack, phần chuỗi nhập đã được xử lý chính là phần chuỗi kí hiệu kết thúc làm tiếp đầu ngữ của dạng câu.

■ Bô đề

- $\forall$ npda $\exists$ npda tương đương thỏa 2 điều kiện
- (1) Chỉ có một trạng thái kết thúc và npda chỉ ở trong trạng thái này $\Leftrightarrow$ **stack là trống**.
- (2) Mọi chuyển trạng thái có dạng $\delta(q_i, a, A) = \{c_1, c_2, \dots, c_n\}$, trong đó

$$c_i = (q_j, \lambda), \quad (7.5) \text{ hoặc}$$

$$c_i = (q_j, BC) \quad (7.6)$$



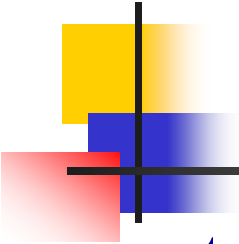
Tức là, một di chuyển hoặc tăng hoặc giảm nội dung stack một kí hiệu.

VPPNC cho ô tô máy đẩy xuống (tt)

- Chúng ta muốn dạng câu chỉ ra nội dung của stack.
- Cấu hình của một npda còn liên quan đến trạng thái nội của ô tô máy nên nó **phải được ghi nhớ trong dạng câu**.
- Lấy $(q_i A q_j)$ làm các biến cho văn phạm, với diễn dịch $(q_i A q_j) \xRightarrow{*} w$ nếu và chỉ nếu npda “xóa” A khỏi stack và đi từ trạng thái q_i đến q_j trong khi đọc ngõ nhập chuỗi w .
- “Xóa” ở đây có nghĩa là A và các kết quả sau nó biến mất khỏi stack, và kí hiệu ngay bên dưới A sẽ trở thành đỉnh stack.

■ Ví dụ

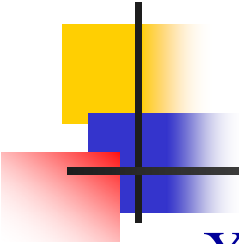
- $\delta(q_i, a, A) = (q_j, \lambda)$ $\longrightarrow$ $(q_i A q_j) \rightarrow a$
- $\delta(q_i, a, A) = (q_j, BC)$ $\longrightarrow$ $(q_i A q_k) \rightarrow a(q_j B q_l)(q_l C q_k)$



VPPNC cho ô tô máy đẩy xuống (tt)

trong đó q_k và q_l lấy mọi giá trị có thể trong Q .

- Khi vết cặn có thể có một vài q_k không thể đạt tới được từ q_i trong khi xóa A cũng như có thể có một vài q_l không thể đạt tới được từ q_j trong khi xóa B .
- Trong trường hợp đó các biến $(q_i A q_k)$ và $(q_j B q_l)$ sẽ là vô dụng.
- Những biến này sẽ được loại bỏ bằng giải thuật loại bỏ các biến vô dụng đã học.
- Biến $(q_0 z q_f)$ sẽ là biến khởi đầu, trong đó q_f là trạng thái kết thúc đơn của npda.



Ví dụ

- Xây dựng VPPNC cho npda sau (q_0 khởi đầu, q_2 kết thúc)

$$\delta(q_0, a, z) = \{(q_0, Az)\},$$

$$\delta(q_0, a, A) = \{(q_0, A)\},$$

$$\delta(q_0, b, A) = \{(q_1, \lambda)\},$$

$$\delta(q_1, \lambda, z) = \{(q_2, \lambda)\}.$$

$$L = \{a^n b : n \geq 1\}$$

- Biến đổi nó thành npda tương đương thỏa 2 điều kiện.

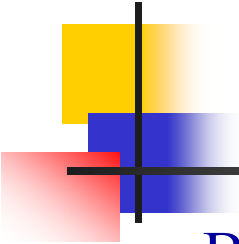
$$\delta(q_0, a, z) = \{(q_0, Az)\}, \quad (1)$$

$$\delta(q_0, a, A) = \{(q_3, \lambda)\}, \quad (2)$$

$$\delta(q_3, \lambda, z) = \{(q_0, Az)\}, \quad (3)$$

$$\delta(q_0, b, A) = \{(q_1, \lambda)\}, \quad (4)$$

$$\delta(q_1, \lambda, z) = \{(q_2, \lambda)\}. \quad (5)$$



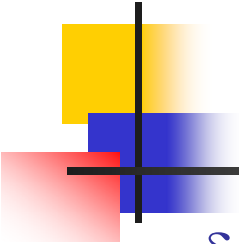
Ví dụ (tt)

- Ba chuyển trạng thái (2), (4), (5) có dạng (7.5) nên có các luật sinh tương ứng với nó là

$$\delta(q_0, a, A) = \{(q_3, \lambda)\}, \quad (2) \quad \longrightarrow \quad (q_0 A q_3) \rightarrow a \quad (6)$$

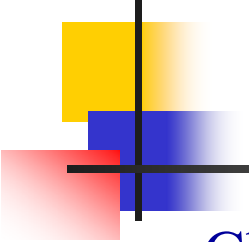
$$\delta(q_0, b, A) = \{(q_1, \lambda)\}, \quad (4) \quad \longrightarrow \quad (q_0 A q_1) \rightarrow b \quad (7)$$

$$\delta(q_1, \lambda, z) = \{(q_2, \lambda)\}. \quad (5) \quad \longrightarrow \quad (q_1 z q_2) \rightarrow \lambda \quad (8).$$



Ví dụ (tt)

- $\delta(q_0, a, z) = \{(q_0, Az)\}$ được mô phỏng thành
- $(q_0zq_0) \rightarrow a(q_0Aq_0)(q_0zq_0) \mid a(q_0Aq_1)(q_1zq_0) \mid a(q_0Aq_2)(q_2zq_0) \mid a(q_0Aq_3)(q_3zq_0),$
- $(q_0zq_1) \rightarrow a(q_0Aq_0)(q_0zq_1) \mid a(q_0Aq_1)(q_1zq_1) \mid a(q_0Aq_2)(q_2zq_1) \mid a(q_0Aq_3)(q_3zq_1),$
- $(q_0zq_2) \rightarrow a(q_0Aq_0)(q_0zq_2) \mid a(q_0Aq_1)(q_1zq_2) \mid a(q_0Aq_2)(q_2zq_2) \mid a(q_0Aq_3)(q_3zq_2),$
- $(q_0zq_3) \rightarrow a(q_0Aq_0)(q_0zq_3) \mid a(q_0Aq_1)(q_1zq_3) \mid a(q_0Aq_2)(q_2zq_3) \mid a(q_0Aq_3)(q_3zq_3),$



Ví dụ (tt)

- Chuyển trạng thái $\delta(q_3, \lambda, z) = \{(q_0, Az)\}$ có thể được mô phỏng bằng tập luật sinh sau
- $(q_3zq_0) \rightarrow (q_0Aq_0)(q_0zq_0) \mid (q_0Aq_1)(q_1zq_0) \mid (q_0Aq_2)(q_2zq_0) \mid (q_0Aq_3)(q_3zq_0),$
- $(q_3zq_1) \rightarrow (q_0Aq_0)(q_0zq_1) \mid (q_0Aq_1)(q_1zq_1) \mid (q_0Aq_2)(q_2zq_1) \mid (q_0Aq_3)(q_3zq_1),$
- $(q_3zq_2) \rightarrow (q_0Aq_0)(q_0zq_2) \mid (q_0Aq_1)(q_1zq_2) \mid (q_0Aq_2)(q_2zq_2) \mid (q_0Aq_3)(q_3zq_2),$
- $(q_3zq_3) \rightarrow (q_0Aq_0)(q_0zq_3) \mid (q_0Aq_1)(q_1zq_3) \mid (q_0Aq_2)(q_2zq_3) \mid (q_0Aq_3)(q_3zq_3),$

Ví dụ (tt)

- Biến khởi đầu của văn phạm là (q_0zq_2) .

$$(q_0Aq_3) \rightarrow a \quad (6)$$

- Loại bỏ biến vô dụng

$$(q_0Aq_1) \rightarrow b \quad (7)$$

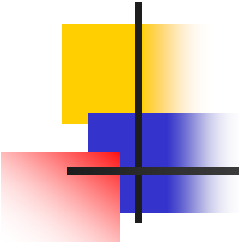
$$(q_1zq_2) \rightarrow \lambda \quad (8)$$

- $(q_0zq_0) \rightarrow a(q_0Aq_0)(q_0zq_0) \mid a(q_0Aq_1)(q_1zq_0) \mid a(q_0Aq_2)(q_2zq_0) \mid a(q_0Aq_3)(q_3zq_0),$

- $(q_0zq_1) \rightarrow a(q_0Aq_0)(q_0zq_1) \mid a(q_0Aq_1)(q_1zq_1) \mid a(q_0Aq_2)(q_2zq_1) \mid a(q_0Aq_3)(q_3zq_1),$

- $(q_0zq_2) \rightarrow a(q_0Aq_0)(q_0zq_2) \mid a(q_0Aq_1)(q_1zq_2) \mid a(q_0Aq_2)(q_2zq_2) \mid a(q_0Aq_3)(q_3zq_2),$

- $(q_0zq_3) \rightarrow a(q_0Aq_0)(q_0zq_3) \mid a(q_0Aq_1)(q_1zq_3) \mid a(q_0Aq_2)(q_2zq_3) \mid a(q_0Aq_3)(q_3zq_3),$



Ví dụ (tt)

$$(q_0 A q_3) \rightarrow a \quad (6)$$

$$(q_0 A q_1) \rightarrow b \quad (7)$$

$$(q_1 z q_2) \rightarrow \lambda \quad (8)$$

- $(q_3 z q_0) \rightarrow (q_0 A q_0)(q_0 z q_0) \mid (q_0 A q_1)(q_1 z q_0) \mid (q_0 A q_2)(q_2 z q_0) \mid (q_0 A q_3)(q_3 z q_0),$
- $(q_3 z q_1) \rightarrow (q_0 A q_0)(q_0 z q_1) \mid (q_0 A q_1)(q_1 z q_1) \mid (q_0 A q_2)(q_2 z q_1) \mid (q_0 A q_3)(q_3 z q_1),$
- $(q_3 z q_2) \rightarrow (q_0 A q_0)(q_0 z q_2) \mid (q_0 A q_1)(q_1 z q_2) \mid (q_0 A q_2)(q_2 z q_2) \mid (q_0 A q_3)(q_3 z q_2),$
- $(q_3 z q_3) \rightarrow (q_0 A q_0)(q_0 z q_3) \mid (q_0 A q_1)(q_1 z q_3) \mid (q_0 A q_2)(q_2 z q_3) \mid (q_0 A q_3)(q_3 z q_3),$

Ví dụ (tt)

Văn phạm kết quả

$(q_0zq_2) \rightarrow a(q_0Aq_1)(q_1zq_2) \mid a(q_0Aq_3)(q_3zq_2),$
 $(q_3zq_2) \rightarrow (q_0Aq_1)(q_1zq_2) \mid (q_0Aq_3)(q_3zq_2),$
 $(q_0Aq_3) \rightarrow a,$
 $(q_0Aq_1) \rightarrow b,$
 $(q_1zq_2) \rightarrow \lambda,$

Npda

$\delta(q_0, a, z) = \{(q_0, Az)\},$
 $\delta(q_0, a, A) = \{(q_3, \lambda)\},$
 $\delta(q_3, \lambda, z) = \{(q_0, Az)\},$
 $\delta(q_0, b, A) = \{(q_1, \lambda)\},$
 $\delta(q_1, \lambda, z) = \{(q_2, \lambda)\}.$

Phân tích chuỗi *aab*

$(q_0, aab, z) \vdash$	(q_0, ab, Az)	$(q_0zq_2) \Rightarrow a(q_0Aq_3)(q_3zq_2)$
$\vdash$	(q_3, b, z)	$\Rightarrow aa(q_3zq_2)$
$\vdash$	(q_0, b, Az)	$\Rightarrow aa(q_0Aq_1)(q_1zq_2)$
$\vdash$	(q_1, λ, z)	$\Rightarrow aab(q_1zq_2)$
$\vdash$	(q_2, λ, λ)	$\Rightarrow aab$

Ví dụ (tt)

Rút gọn văn phạm

$$(q_0zq_2) \rightarrow a(q_0Aq_1)(q_1zq_2) \mid a(q_0Aq_3)(q_3zq_2),$$

$$(q_3zq_2) \rightarrow (q_0Aq_1)(q_1zq_2) \mid (q_0Aq_3)(q_3zq_2),$$

$$(q_0Aq_3) \rightarrow a,$$

$$(q_0Aq_1) \rightarrow b,$$

$$(q_1zq_2) \rightarrow \lambda,$$

$$S \rightarrow aBC \mid aAX,$$

$$X \rightarrow BC \mid AX,$$

$$A \rightarrow a,$$

$$B \rightarrow b,$$

$$C \rightarrow \lambda,$$

$$S \rightarrow ab \mid aaX,$$

$$X \rightarrow b \mid aX,$$

$$L = \{a^n b : n \geq 1\}$$

■ Định lý 7.2

- Nếu $L = L(M)$ đối với một npda M nào đó, thì L là NNPNC.



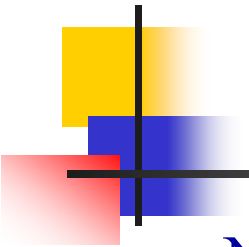
PDA đơn định và NNPNC đơn định

■ Định nghĩa 7.3

- Một ô tô máy đẩy xuống $M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$ được gọi là **đơn định** nếu nó là một ô tô máy được định nghĩa như trong Định nghĩa 7.1, nhưng phải chịu sự giới hạn rằng, đối $\forall$ trạng thái $q \in Q$, kí hiệu $a \in \Sigma \cup \{\lambda\}$, và $b \in \Gamma$,
 - (1) $\delta(q, a, b)$ chứa tối đa một phần tử,
 - (2) Nếu $\delta(q, \lambda, b) \neq \emptyset$, thì $\delta(q, c, b) = \emptyset \forall c \in \Sigma$.

■ Định nghĩa 7.4

- Một ngôn ngữ L được gọi là NNPNC **đơn định** nếu và chỉ nếu tồn tại một dpda M sao cho $L = L(M)$.



Ví dụ

- Ngôn ngữ $L = \{a^n b^n: n \geq 0\}$ là PNCĐĐ. Vì nó được chấp nhận bởi dpda sau

$M = (\{q_0f, q_1, q_2\}, \{a, b\}, \{z, a\}, \delta, q_0f, z, \{q_0f\})$ với

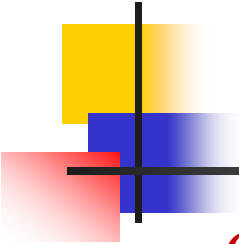
$$\delta(q_0f, a, z) = \{(q_1, az)\},$$

$$\delta(q_1, a, a) = \{(q_1, aa)\},$$

$$\delta(q_1, b, a) = \{(q_2, \lambda)\},$$

$$\delta(q_2, b, a) = \{(q_2, \lambda)\},$$

$$\delta(q_2, \lambda, z) = \{(q_0f, \lambda)\}.$$



Ví dụ (tt)

- **Cách 2** (với # là kí hiệu kết thúc chuỗi – eof)

$M = (\{q_0, q_1, q_f\}, \{a, b\}, \{z, 1\}, \delta, q_0, z, \{q_f\})$ với

$$\delta(q_0, \#, z) = \{(q_f, \lambda)\},$$

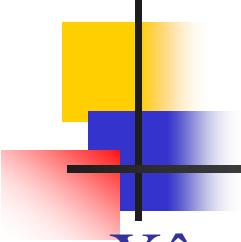
$$\delta(q_0, a, z) = \{(q_0, 1z)\},$$

$$\delta(q_0, a, 1) = \{(q_0, 11)\},$$

$$\delta(q_0, b, 1) = \{(q_1, \lambda)\},$$

$$\delta(q_1, b, 1) = \{(q_1, \lambda)\},$$

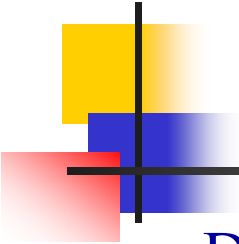
$$\delta(q_1, \#, z) = \{(q_f, \lambda)\}.$$



Bài tập

- Xây dựng dpda cho các ngôn ngữ sau

- $L_1 = \{a^n b^m c^{n+m} : n, m \geq 0\}$
- $L_2 = \{a^n b^{n+m} c^m : n, m \geq 1\}$
- $L_3 = \{a^n b^m : 2n \leq m \leq 3n\}$
- $L_4 = \{w : n_a(w) = n_b(w) + 2\}$
- $L_5 = \{w : n_a(w) = 2n_b(w)\}$
- $L_6 = \{w : 2n_b(w) \leq n_a(w) \leq 3n_b(w)\}$



Dpda và Npda

- Dpda là một lớp con thực sự của npda
- $L_1 = \{a^n b^n : n \geq 0\}$ và $L_2 = \{a^n b^{2n} : n \geq 0\}$ là các NNPNC và $L = L_1 \cup L_2$ cũng là NNPNC.
- L sẽ được chứng minh không phải là NNPNC đơn định.
- Chứng minh
 - Trước hết chúng ta sử dụng một kết quả trong Chương 8 là rằng ngôn ngữ $L_0 = \{a^n b^n c^n : n \geq 0\}$ là không phi ngữ cảnh.
 - Giả sử L là NNPNC đơn định, gọi $M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$ là dpda của L với $Q = \{q_0, q_1, \dots, q_n\}$

Dpda và Npda (tt)

■ Xét

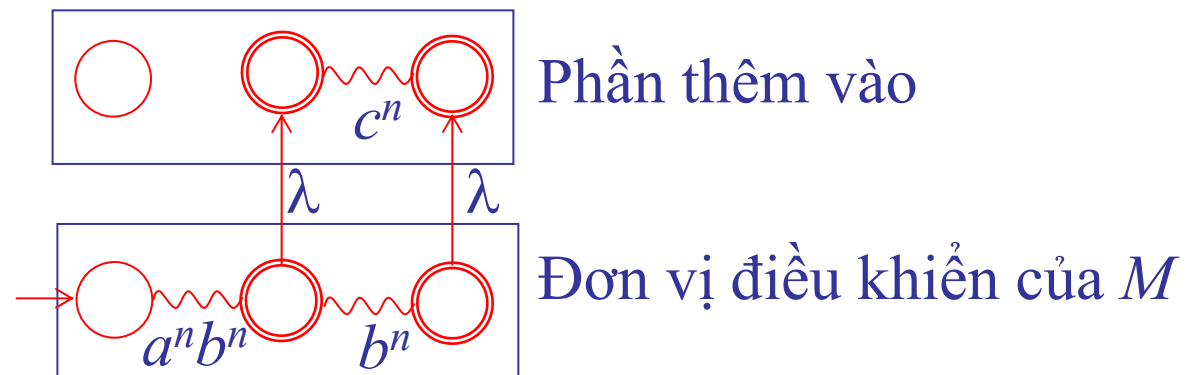
$$M' = (Q', \Sigma, \Gamma, \delta \cup \delta', q_0, z, F')$$

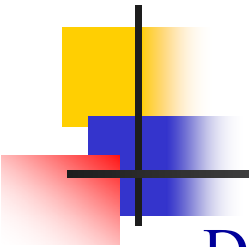
$$Q' = Q \cup \{q_0', q_1', \dots, q_n'\},$$

$$F' = F \cup \{q_i' : q_i \in F\},$$

$$\delta'(q_f, \lambda, s) = \{(q_f', s)\} \quad \forall q_f \in F, s \in \Gamma, \text{ và}$$

$$\delta'(q_i', c, s) = \{(q_j', u)\} \quad \forall \delta(q_i, b, s) = \{(q_j, u)\},$$





Dpda và Npda (tt)

- Để M chấp nhận $a^n b^n$ chúng ta phải có

$$(q_0, a^n b^n, z) \vdash_M^* (q_i, \lambda, u), \text{ với } q_i \in F.$$

Bởi M là đơn định, nó cũng phải đúng rằng

$$(q_0, a^n b^{2n}, z) \vdash_M^* (q_i, b^n, u),$$

vậy để nó chấp nhận $a^n b^{2n}$ phải có

$$(q_i, b^n, u) \vdash_M^* (q_j, \lambda, u_1),$$

với một q_j nào đó $\in F$. Nhưng theo cách xây dựng ta có

$$(q_i', c^n, u) \vdash_M^* (q_j', \lambda, u_1),$$

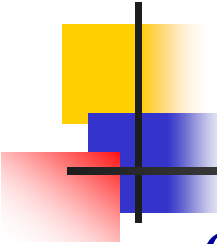
như vậy M' sẽ chấp nhận $a^n b^n c^n$.

- Không có chuỗi nào khác hơn những chuỗi trong L' là được chấp nhận bởi M' .
- Suy ra $\{a^n b^n c^n\}$ là PNC ($><$). Vậy L PNC không đơn định.

Văn phạm cho các NNPNC đơn định

- NNPNCĐĐ cho phép PTCP một cách hiệu quả, bằng cách xem dpda như là một **thiết bị phân tích**.
- **Tính đơn định** suy ra việc xử lý chuỗi nhập trong **thời gian tuyến tính** với chiều dài chuỗi nhập.
- Những loại văn phạm nào thích hợp cho việc mô tả các NNPNCĐĐ và cho phép PTCP thời gian tuyến tính.
- Giả sử chúng ta đang phân tích từ trên xuống, đang thử tìm DXTN của một câu cụ thể.

Chuỗi nhập w	a_1	a_2	a_3		a_4	.	.	.	a_n
Dạng câu	a_1	a_2	a_3		A	.	.	.	
	←				→				
	Phần đã được so trùng				Phần còn chưa được so trùng				



Văn phạm cho các NNPNC đơn định (tt)

- Quét chuỗi nhập w từ trái sang phải, trong khi phát triển dạng câu mà chuỗi kí hiệu kết thúc tiếp đầu ngữ của nó so trùng với tiếp đầu ngữ của chuỗi w cho đến kí hiệu được quét hiện tại.
- Để tiếp tục so trùng các kí hiệu kế tiếp, chúng ta muốn biết chính xác luật sinh nào là được áp dụng tại mỗi bước để tránh backtracking và cho phép PTCP hiệu quả.
- Có hay không loại văn phạm cho phép làm điều này?
- Với VPPNC tổng quát, điều này là không thể, nhưng nếu dạng của văn phạm được hạn chế hơn, có thể thực hiện được mục đích của chúng ta.
- Văn phạm-s là một ví dụ nhưng khả năng biểu diễn ngôn ngữ của nó còn hạn chế.

Văn phạm $LL(k)$

■ Định nghĩa 7.5

- Cho $G = (V, T, S, P)$ là một VPPNC. Nếu đối với mọi cặp dẫn xuất trái nhất

$$S \xRightarrow{*} w_1 A x_1 \Rightarrow w_1 y_1 x_1 \xRightarrow{*} w_1 w_2,$$

$$S \xRightarrow{*} w_1 A x_2 \Rightarrow w_1 y_2 x_2 \xRightarrow{*} w_1 w_3,$$

với $w_1, w_2, w_3 \in T^*$, sự bằng nhau của k kí hiệu trái nhất của w_2 và w_3 (nếu có) $\Rightarrow y_1 = y_2$, thì G được gọi là một $VP LL(k)$.

- Điều này có nghĩa là nếu nhìn trước k kí hiệu ngõ nhập thì chỉ có tối đa một luật sinh là đúng dẫn nhất.

■ Ví dụ

- Văn phạm bên là thuộc loại $LL(1)$

$$S \rightarrow aX \mid bS, \quad (1, 2)$$

$$X \rightarrow b \mid aS, \quad (3, 4)$$

Văn phạm $LL(k)$

■ Bảng PTCP

	a	b	$\#$
S	1	2	
X	4	3	

$$S \rightarrow aX \mid bS, \quad (1, 2)$$

$$X \rightarrow b \mid aS, \quad (3, 4)$$

■ Ví dụ

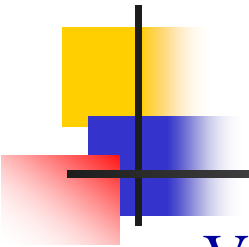
- Các văn phạm sau đây thuộc họ $LL(k)$ không? Nếu có $k = ?$

$$S \rightarrow aSbS \mid bSaS \mid \lambda, \quad (1, 2, 3)$$

$$S \rightarrow aS \mid AB, \quad (1, 2)$$

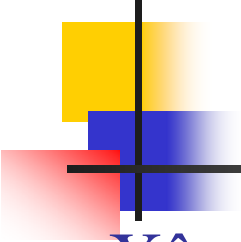
$$A \rightarrow bA \mid b, \quad (3, 4)$$

$$B \rightarrow aS \mid b, \quad (5, 6)$$



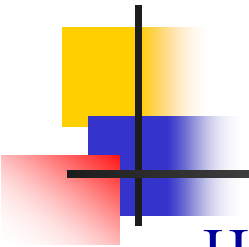
Văn phạm $LL(k)$

- Văn phạm $LL(k)$ cho phép PTCP đơn định nếu nhìn trước k kí hiệu.
- Văn phạm LL là một chủ đề quan trọng trong việc nghiên cứu các trình biên dịch.
- Một số NNLT có thể được định nghĩa bằng các văn phạm LL , và nhiều trình biên dịch đã được viết bằng cách sử dụng các bộ PTCP LL .
- Nhưng văn phạm LL là không đủ tổng quát để giải quyết các NNPNCDĐ. Vì vậy, có một mối quan tâm đến các loại văn phạm khác, văn phạm đơn định tổng quát hơn.
- Đó là văn phạm LR , cái mà cho phép PTCP hiệu quả, và xây dựng cây dẫn xuất từ dưới lên.



Bài tập

- Xây dựng văn phạm $LL(k)$ cho các ngôn ngữ sau
 - $L_1 = \{a^n b^n : n \geq 0\}$
 - $L_2 = \{w \in \{a, b\}^* : n_a(w) = n_b(w)\}$
 - $L_3 = \{w \in \{a, b\}^* : n_a(w) = n_b(w), n_a(v) \geq n_b(v) \text{ với } v \text{ là một tiếp đầu ngữ bất kỳ của } w\}$



Chương 8 Các tính chất của NNPNC

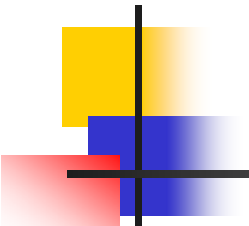
- Họ NNPNC chiếm một vị trí trung tâm trong hệ thống phân cấp các ngôn ngữ hình thức.
- Một mặt, NNPNC bao gồm các họ ngôn ngữ quan trọng nhưng bị giới hạn chẳng hạn như các NNPNC và PNCĐĐ.
- Mặt khác, có các họ ngôn ngữ khác rộng lớn hơn mà NNPNC chỉ là một trường hợp đặc biệt.
- Để nghiên cứu mối quan hệ giữa các họ ngôn ngữ và trình bày những cái giống nhau và khác nhau của chúng, chúng ta nghiên cứu các tính chất đặc trưng của các họ khác nhau.
- Như trong Chương 4, chúng ta xem xét tính đóng dưới nhiều phép toán khác nhau, các giải thuật để xác định tính thành viên, và cuối cùng là bộ đề bơm.



Chương 8 Các tính chất của NNPNC

8.1 Hai bộ đề bơm

8.2 Tính đóng và các giải thuật quyết định cho NNPNC



Bổ đề bơm cho NNPNC

■ Định lý 8.1

- Cho L là một NNPNC vô hạn, tồn tại một số nguyên dương m sao cho bất kỳ chuỗi w nào $\in L$ với $|w| \geq m$, w có thể được phân hoạch thành

$$w = uvxyz \quad (8.1) \text{ với}$$

$$|vxy| \leq m \quad (8.2) \text{ và}$$

$$|vy| \geq 1 \quad (8.3) \text{ sao cho}$$

$$uv^i xy^i z \in L \quad (8.4) \quad \forall i = 0, 1, 2, \dots$$

- Định lý này được gọi là bổ đề bơm cho NNPNC.

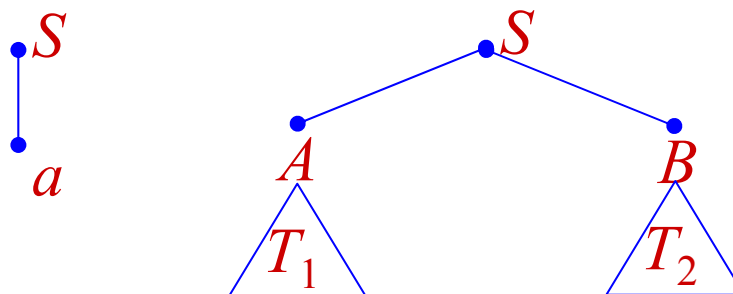
■ Chứng minh

- Xét ngôn ngữ $L - \{\lambda\}$. Đây là NNPNC $\Rightarrow \exists$ văn phạm có dạng chuẩn Chomsky G chấp nhận nó.

Chứng minh

■ Bổ đề

- Nếu cây dẫn xuất của một chuỗi w được sinh ra bởi một văn phạm Chomsky mà có chiều dài mọi con đường đi từ gốc tới lá nhỏ hơn hay bằng h thì $|w| \leq 2^{h-1}$.
- Bổ đề này có thể chứng minh bằng qui nạp dựa trên h .

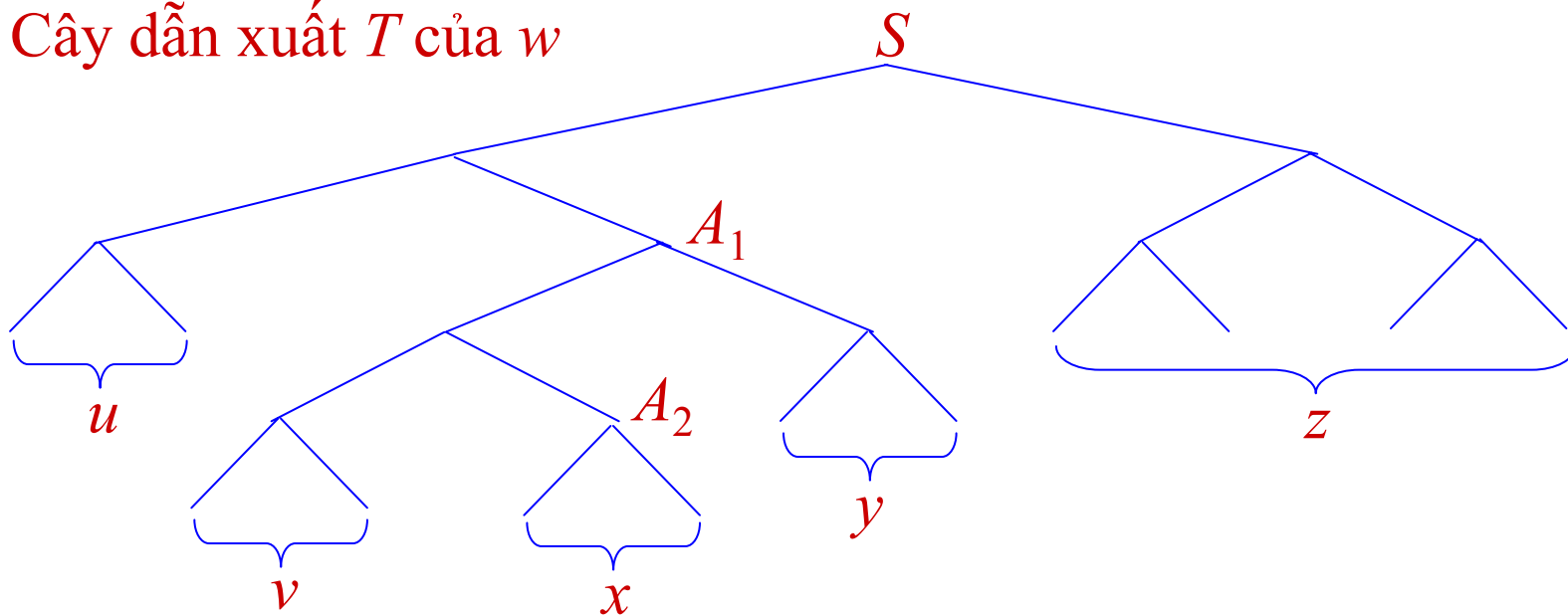


- Trở lại chứng minh của định lý. Giả sử G có k biến ($|V| = k$). Chọn $m = 2^k$. Lấy w bất kỳ $\in L$ sao cho $|w| \geq m$. Xét cây dẫn xuất T của w .
- Theo bổ đề trên suy ra T phải có ít nhất một con đường đi từ gốc tới lá có chiều dài $\geq k+1$.

Chứng minh (tt)

- Xét một đường như vậy. Trên đường này có $\geq k+2$ phần tử. Nếu không tính nốt lá là kí hiệu kết thúc thì có $\geq k+1$ nốt là biến.
- Vì tập biến chỉ có k biến $\Rightarrow \exists$ hai nốt trùng vào một biến. Giả sử đó là biến A (hai lần xuất hiện kí hiệu là A_1 và A_2)

Cây dẫn xuất T của w



Chứng minh (tt)

- Trong cây trên, gọi u, v, x, y, z là các chuỗi có tính chất sau:

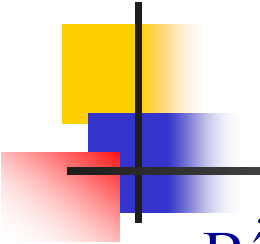
$$S \xRightarrow{*} uA_1z \quad (1)$$

$$A_1 \xRightarrow{*} vA_2y \quad (2)$$

$$A_2 \xRightarrow{*} x \quad (3)$$

Và $w = uvxyz$.

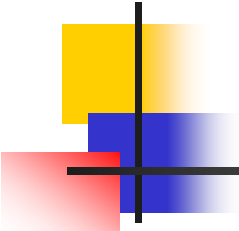
- vxy là kết quả của cây có gốc là A_1 mà mọi con đường của cây này có chiều dài $\leq (k+1) \Rightarrow$ theo bổ đề trên $|vxy| \leq 2^k = m$.
Mặt khác vì văn phạm có dạng chuẩn Chomsky tức là không có luật sinh-đơn vị và luật sinh- λ nên từ (2) suy ra $|vy| \geq 1$.
- Từ (1), (2), (3) chúng ta có:
$$S \xRightarrow{*} uAz \xRightarrow{*} uvAyz \xRightarrow{*} uv^iAy^iz \xRightarrow{*} uv^ixy^iz$$
hay $uv^ixy^iz \in L \quad \forall i = 0, 1, 2, \dots$
- Điều này kết thúc chứng minh.



Ví dụ

- Bổ đề bơm này được dùng để chứng minh một ngôn ngữ là không PNC tương tự như ở Chương 4.
- Ví dụ
 - Chứng minh ngôn ngữ $L = \{a^n b^n c^n : n \geq 0\}$ là không PNC.
- Chứng minh
 - Giả sử L là PNC $\Rightarrow \exists$ số nguyên dương m .
Chọn $w = a^m b^m c^m \in L$. $\exists$ một phân hoạch của w thành bộ 5
$$w = uvxyz$$

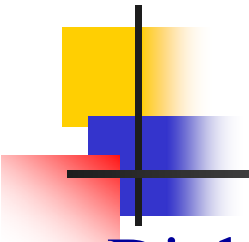
Vì $|vxy| \leq m$ nên vxy không chứa đồng thời cả 3 kí hiệu a, b, c .
Chọn $i = 2 \Rightarrow w_2 = uv^2xy^2z$ sẽ chứa a, b, c với số lượng không bằng nhau $\Rightarrow w_2 \notin L (><)$.



Bài tập

■ Ngôn ngữ nào sau đây PNC? Chứng minh.

- $L_1 = \{a^n b^j c^k : k = jn\}$
- $L_2 = \{a^n b^j c^k : k > n, k > j\}$
- $L_3 = \{a^n b^j c^k : n < j, n \leq k \leq j\}$
- $L_5 = \{a^n b^j a^n b^j : n \geq 0, j \geq 0\}$
- $L_4 = \{w : n_a(w) < n_b(w) < n_c(w)\}$
- $L_6 = \{a^n b^j a^k b^l : n + j \leq k + l\}$
- $L_7 = \{a^n b^j a^k b^l : n \leq k, j \leq l\}$
- $L_8 = \{a^n b^n c^j : n \leq j\}$



Bổ đề bơm cho ngôn ngữ tuyến tính

■ Định nghĩa 8.1

- Một NNPNC L được gọi là tuyến tính nếu $\exists$ một VPPNC tuyến tính G sao cho $L = L(G)$.

■ Định lý 8.2

- Cho L là một NN tuyến tính vô hạn, tồn tại một số nguyên dương m sao cho bất kỳ chuỗi w nào $\in L$ với $|w| \geq m$, w có thể được phân hoạch thành $w = uvxyz$ với

$$|uvyz| \leq m \quad (8.7) \text{ và}$$

$$|vy| \geq 1 \quad (8.8) \text{ sao cho}$$

$$uv^i xy^i z \in L \quad (8.9) \quad \forall i = 0, 1, 2, \dots$$

Chứng minh

- Gọi G là văn phạm tuyến tính mà không chứa luật sinh-đơn vị và luật sinh- λ .
- Gọi $k = \max \{\text{các chiều dài vế phải}\} \Rightarrow$ mỗi bước dẫn xuất chiều dài dạng câu tăng tối đa $(k-1)$ kí hiệu $\Rightarrow$ một chuỗi w dẫn xuất dài p bước thì $|w| \leq 1 + p(k-1)$ (1).
- Đặt $|V| = n$. Chọn $m = 2 + n(k-1)$. Xét w bất kỳ $\in L$, $|w| \geq m$. (1) $\Rightarrow$ dẫn xuất của w có $\geq (n+1)$ bước $\Rightarrow$ dẫn xuất có $\geq (n+1)$ dạng câu mà không phải là câu. Chú ý mỗi dạng câu có đúng một biến.
- Xét $(n+1)$ dạng câu đầu tiên của dẫn xuất trên $\Rightarrow \exists$ hai biến của hai dạng câu nào đó trùng nhau, giả sử là biến A . Như vậy dẫn xuất của w phải có dạng:

$$S \xRightarrow{*} uAz \xRightarrow{*} uvAyz \xRightarrow{*} uvxyz, \quad (2)$$

với $u, v, x, y, z \in T^*$.

Chứng minh (tt)

- Xét dẫn xuất riêng phần

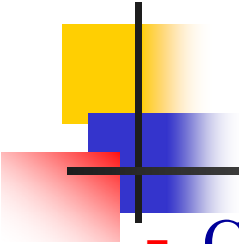
$$S \xRightarrow{*} uAz \xRightarrow{*} uvAyz$$

vì A được lặp lại trong $(n + 1)$ dạng câu đầu tiên nên dãy này có $\leq n$ bước dẫn xuất $\Rightarrow |uvAyz| \leq 1 + n(k-1)$, $\Rightarrow |uvyz| \leq n(k-1) < m$.
Mặt khác vì G không có luật sinh-đơn vị và luật sinh- λ nên ta có $|vy| \geq 1$.

- Từ (2) cũng suy ra:

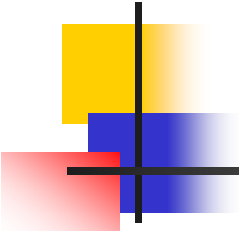
$$\begin{aligned} S &\xRightarrow{*} uAz \xRightarrow{*} uvAyz \xRightarrow{*} uv^iAy^iz \xRightarrow{*} uv^ixy^iz \\ &\Rightarrow uv^ixy^iz \in L \quad \forall i = 0, 1, 2, \dots \end{aligned}$$

- Chứng minh hoàn tất.



Ví dụ

- Chứng minh ngôn ngữ $L = \{w: n_a(w) = n_b(w)\}$ là không tuyến tính.
- Chứng minh
 - Giả sử L là tuyến tính. Chọn $w = a^m b^{2m} a^m$.
Từ (8.7) $\Rightarrow u, v, y, z$ phải chứa toàn a . Nếu bơm chuỗi này lên, chúng ta nhận được chuỗi $a^{m+k} b^{2m} a^{m+l}$, với $k \geq 1$ hoặc $l \geq 1$, mà chuỗi này $\notin L$ ($><$) $\Rightarrow L$ không phải là ngôn ngữ tuyến tính.



Bài tập

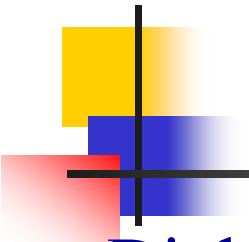
- Ngôn ngữ nào sau đây PNC tuyến tính? Chứng minh.
 - $L_1 = \{a^n b^n a^m b^m : n, m \geq 0\}$
 - $L_2 = \{w : n_a(w) \geq n_b(w)\}$
 - $L_3 = \{a^n b^j : j \leq n \leq 2j - 1\}$
 - $L_4 = L(G)$ với G được cho như sau:

$$E \rightarrow T \mid E + T$$

$$T \rightarrow F \mid T * F$$

$$F \rightarrow I \mid (E)$$

$$I \rightarrow a \mid b \mid c$$



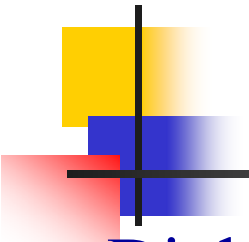
Tính đóng của NNPNC

■ Định lý 8.3

- Họ NNPNC là đóng dưới phép hội, kết nối, và bao đóng sao.

■ Chứng minh

- Giả sử $G_1 = (V_1, T_1, S_1, P_1)$, $G_2 = (V_2, T_2, S_2, P_2)$ là hai VPPNC.
Văn phạm $G_3 = (V_1 \cup V_2 \cup \{S_3\}, T_1 \cup T_2, S_3, P_1 \cup P_2 \cup \{S_3 \rightarrow S_1 \mid S_2\})$ sẽ có $L(G_3) = L(G_1) \cup L(G_2)$.
Văn phạm $G_4 = (V_1 \cup V_2 \cup \{S_4\}, T_1 \cup T_2, S_4, P_1 \cup P_2 \cup \{S_4 \rightarrow S_1 S_2\})$ sẽ có $L(G_4) = L(G_1)L(G_2)$.
Văn phạm $G_5 = (V_1 \cup \{S_5\}, T_1, S_5, P_1 \cup \{S_5 \rightarrow S_1 S_5 \mid \lambda\})$ sẽ có $L(G_5) = L(G_1)^*$.



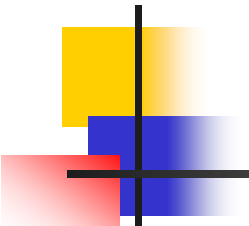
Tính đóng của NNPNC (tt)

■ Định lý 8.4

- Họ NNPNC không đóng dưới phép giao và bù.

■ Chứng minh

- Hai ngôn ngữ $\{a^n b^n c^m: n, m \geq 0\}$ và $\{a^n b^m c^m: n, m \geq 0\}$ là phi ngữ cảnh, tuy nhiên giao của chúng là ngôn ngữ $\{a^n b^n c^n: n \geq 0\}$ lại không phi ngữ cảnh, nên họ NNPNC không đóng dưới phép giao.
- Dựa vào luật Morgan suy ra họ NNPNC cũng không đóng dưới phép bù. Vì nếu đóng đối với phép bù thì dựa vào tính đóng đối với phép hội suy ra tính đóng dưới phép giao theo luật Morgan.



Tính đóng của NNPNC (tt)

■ Định lý 8.5

- Cho L_1 là một NNPNC và L_2 là một NNCQ, thì $L_1 \cap L_2$ là phi ngữ cảnh. Chúng ta nói rằng họ NNPNC là đóng dưới phép giao chính qui.

■ Chứng minh

- Cho $M_1 = (Q, \Sigma, \Gamma, \delta_1, q_0, z, F_1)$ là npda chấp nhận L_1 và $M_2 = (P, \Sigma, \delta_2, p_0, F_2)$ là dfa chấp nhận L_2 .
- Xây dựng một npda $M' = (Q', \Sigma, \Gamma, \delta', q'_0, z, F')$ mô phỏng hoạt động song song của M_1 và M_2

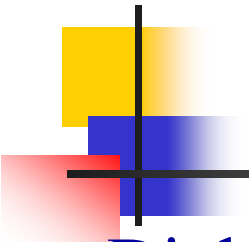
$$Q' = Q \times P, q'_0 = (q_0, p_0), F' = F_1 \times F_2,$$

$$((q_k, p_l), x) \in \delta'((q_i, p_j), a, b), \Leftrightarrow (q_k, x) \in \delta_1(q_i, a, b), \text{ và } \delta_2(p_j, a) = p_l,$$



Tính đóng của NNPNC (tt)

- Nếu $a = \lambda$, thì $p_j = p_l$.
 - Bằng qui nạp chứng minh rằng
$$\delta'^*((q_0, p_0), w, z) \mid -^*_M ((q_r, p_s), x), \text{ với } q_r \in F_1 \text{ và } p_s \in F_2 \Leftrightarrow$$
$$\delta_1^*(q_0, w, z) \mid -^*_{M1} (q_r, x), \text{ còn } \delta_2^*(p_0, w) = p_s.$$
 - Vì vậy $L(M') = L(M_1) \cap L(M_2)$ (điều phải chứng minh)
- Ví dụ
- Ngôn ngữ $L = \{ w \in \{a, b\}^*: n_a(w) = n_b(w), n_a(w) \text{ chẵn} \}$ là phi ngữ cảnh.



Một vài tính chất khả quyết định của NNPNC

■ Định lý 8.6

- Cho một VPPNC $G = (V, T, S, P)$, thì tồn tại một giải thuật để quyết định $L(G)$ có trống hay không.

■ Định lý 8.7

- Cho một VPPNC $G = (V, T, S, P)$, thì tồn tại một giải thuật để quyết định $L(G)$ có vô hạn hay không.



Chương 9 Máy Turing

- PDA về một mặt nào đó mạnh hơn rất nhiều FSA.
- NNPNC-PDA vẫn còn giới hạn. Bên ngoài nó là gì?
- FSA và PDA khác nhau ở bản chất của bộ lưu trữ tạm thời.
- Nếu PDA dùng hai, ba stack, một hàng (queue), hay một thiết bị lưu trữ khác nào đó thì sức mạnh sẽ thế nào?
- Mỗi thiết bị lưu trữ định nghĩa một loại ô tô-mát mới và thông qua nó một họ ngôn ngữ mới?
- Ô tô-mát có thể được mở rộng đến chừng nào? Khả năng mạnh nhất có thể của ô tô-mát? Những giới hạn của việc tính toán?
- Máy Turing ra đời và khái niệm về sự tính toán có tính máy móc hay giải thuật (mechanical or algorithmic computation).
- Máy Turing là khá thô sơ, nhưng đủ sức để bao trùm các quá trình rất phức tạp và **luận đề Turing (Turing thesis)** cho rằng bất kỳ quá trình tính toán nào thực hiện được bằng các máy tính ngày nay, đều có thể thực hiện được bằng máy Turing.



Chương 9 Máy Turing

9.1 Máy Turing chuẩn

9.2 Kết hợp các máy Turing cho các công việc phức tạp

9.3 Luận đề Turing

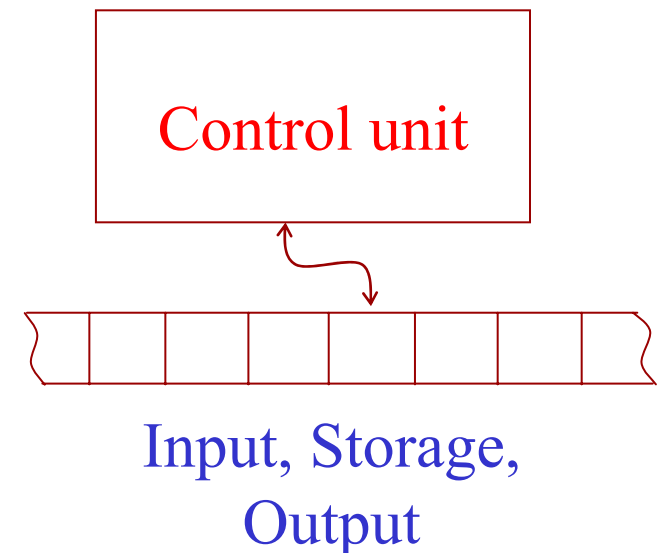
Máy Turing chuẩn

■ Định nghĩa 9.1

- Một máy Turing M được định nghĩa bằng bộ bảy

$$M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F),$$

- Q là tập hữu hạn các trạng thái nội,
- Σ là tập hữu hạn các kí hiệu được gọi là bảng chữ cái ngõ nhập,
- Γ là tập hữu hạn các kí hiệu được gọi là bảng chữ cái băng,
- δ là hàm chuyển trạng thái,
- $\square \in \Gamma$ là một kí hiệu đặc biệt, gọi là khoảng trắng (blank),
- $q_0 \in Q$ là trạng thái khởi đầu,
- $F \subseteq Q$ là tập các trạng thái kết thúc.



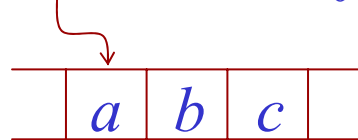
Máy Turing chuẩn (tt)

- Trong định nghĩa chúng ta giả thiết rằng $\Sigma \subseteq \Gamma - \{\square\}$.
- Hàm δ được định nghĩa như sau

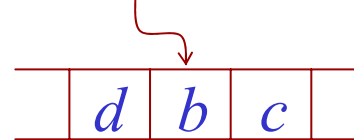
$$\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$$

- Nó được diễn dịch như sau: Các đối số của δ là trạng thái hiện hành của ô tômát và kí hiệu băng đang được đọc. Kết quả là một trạng thái mới của automat, một kí hiệu băng mới thay thế cho kí hiệu đang được đọc trên băng và một sự di chuyển đầu đọc sang phải hoặc sang trái.
- Ví dụ $\delta(q_0, a) = \{q_1, d, R\}$

Trạng thái nội q_0



Trạng thái nội q_1



Ví dụ

- Xét một máy Turing được định nghĩa như sau
- $Q = \{q_0, q_1\}$, $\Sigma = \{a, b\}$, $\Gamma = \{a, b, \square\}$, $F = \emptyset$, còn δ được định nghĩa

$$\delta(q_0, a) = (q_1, a, R)$$

$$\delta(q_1, a) = (q_0, a, L)$$

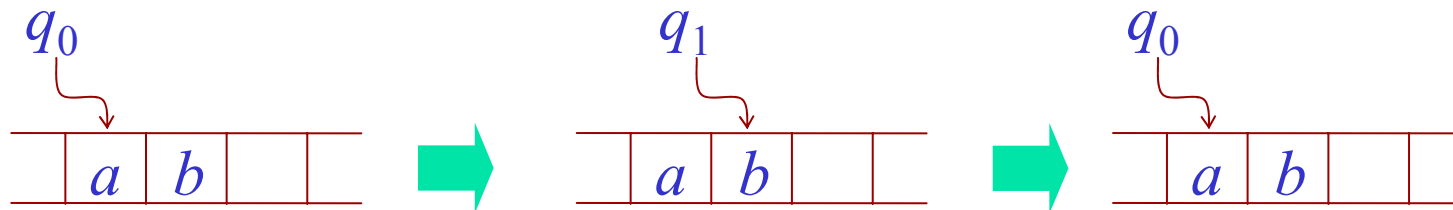
$$\delta(q_0, b) = (q_1, b, R)$$

$$\delta(q_1, b) = (q_0, b, L)$$

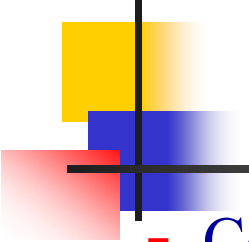
$$\delta(q_0, \square) = (q_1, \square, R)$$

$$\delta(q_1, \square) = (q_0, \square, L)$$

- Xét hoạt động của M trong trường hợp sau



- Trường hợp này máy không dừng lại và rơi vào một **vòng lặp vô tận (infinite loop)**



Các đặc điểm của máy Turing chuẩn

- Có nhiều mô hình khác nhau của máy Turing.
- Sau đây là một số đặc điểm của máy Turing *chuẩn*.
- ❖ Máy Turing có một băng không giới hạn cả hai đầu, cho phép di chuyển một số bước tùy ý về bên trái và phải.
- ❖ Máy Turing là **đơn định** trong ngữ cảnh là δ định nghĩa tối đa một chuyển trạng thái cho một cấu hình.
- ❖ Không có một băng nhập (input file) riêng biệt. Chúng ta giả thiết là vào thời điểm khởi đầu băng chứa một nội dung cụ thể. Một vài trong số này có thể được xem là chuỗi nhập (input). Tương tự không có một băng xuất (output file) riêng biệt. Bất kỳ khi nào máy dừng, một vài hay tất cả nội dung của băng có thể được xem là kết quả xuất (output).

Hình trạng tức thời

■ Định nghĩa 9.2

- Cho $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$ là một máy Turing, thì một chuỗi

$$a_1 a_2 \dots a_{k-1} q_1 a_k a_{k+1} \dots a_n$$

bất kỳ với $a_i \in \Sigma$ và $q_1 \in Q$, là một hình trạng tức thời của M (gọi tắt là hình trạng).

- Một di chuyển

$$a_1 a_2 \dots a_{k-1} q_1 a_k a_{k+1} \dots a_n \vdash a_1 a_2 \dots a_{k-1} b q_2 a_{k+1} \dots a_n$$

là có thể nếu và chỉ nếu

$$\delta(q_1, a_k) = (q_2, b, R).$$

- Một di chuyển

$$a_1 a_2 \dots a_{k-1} q_1 a_k a_{k+1} \dots a_n \vdash a_1 a_2 \dots q_2 a_{k-1} b a_{k+1} \dots a_n$$

là có thể nếu và chỉ nếu

$$\delta(q_1, a_k) = (q_2, b, L).$$

Hình trạng tức thời (tt)

- M được gọi là dừng sau khi bắt đầu từ một cấu hình khởi đầu nào đó $x_1 q_i x_2$ nếu

$$x_1 q_i x_2 \vdash^* y_1 q_j a y_2$$

với bất kỳ q_j và a , mà đối với nó $\delta(q_j, a)$ không được định nghĩa.

- Dây cấu hình dẫn tới một trạng thái dừng sẽ được gọi là **một sự tính toán** (computation).
- Ví dụ trong slide 290 trình bày khả năng rằng một máy Turing có thể không bao giờ dừng, thi hành trong một vòng lặp vô tận và từ đó nó không thể thoát.
- Trường hợp này đóng một vai trò cơ bản trong thảo luận về máy Turing, và được kí hiệu là

$$x_1 q x_2 \vdash^* \infty$$

để chỉ ra rằng, bắt đầu từ cấu hình khởi đầu $x_1 q x_2$, máy không bao giờ dừng.

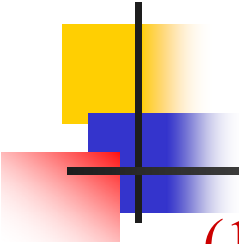
Máy Turing như một bộ chấp nhận ngôn ngữ

■ Định nghĩa 9.3

- Cho $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$ là một máy Turing, thì ngôn ngữ được chấp nhận bởi M là

$$L(M) = \{w \in \Sigma^+ : q_0 w \vdash^* x_1 q_f x_2 \text{ và dừng, đối với một } q_f \text{ nào đó} \\ \in F, x_1, x_2 \in \Gamma^*\}.$$

- Định nghĩa này chỉ ra rằng chuỗi nhập w được viết trên băng với các khoảng trắng chặn ở hai đầu. Đây cũng là lý do các khoảng trắng bị loại ra khỏi bảng chữ cái ngõ nhập Σ .
- Điều này đảm bảo chuỗi nhập được giới hạn trong một vùng rõ ràng của băng được bao bọc hai đầu bởi các kí hiệu trắng.
- Không có qui ước này, máy không thể giới hạn vùng trong đó nó tìm kiếm chuỗi nhập.
- Định nghĩa trên không nói rõ khi nào thì $w \notin L(M)$. Điều này đúng khi một trong hai trường hợp sau xảy ra



Ví dụ

- (1) Máy dừng lại ở một trạng thái không kết thúc.
- (2) Máy đi vào một vòng lặp vô tận và không bao giờ dừng.

■ Ví dụ

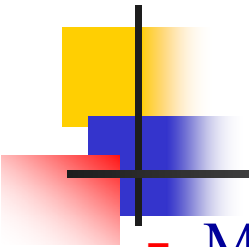
- Cho $\Sigma = \{a, b\}$, thiết kế máy Turing chấp nhận $L = \{a^n b^n : n \geq 1\}$.
- Ý tưởng thiết kế là đọc một a thay bằng một x , đi kiểm một b thay bằng một y . Cứ như vậy cho đến khi không còn đồng thời a và b để thay thì dừng và chấp nhận chuỗi, các trường hợp khác thì không chấp nhận. Máy Turing kết quả như sau.

$$\begin{aligned} Q &= \{q_0, q_1, q_2, q_3, q_f\}, F = \{q_f\}, \Sigma = \{a, b\}, \Gamma = \{a, b, x, y, \square\} \\ \delta(q_0, a) &= \{q_1, x, R\} & \delta(q_2, y) &= \{q_2, y, L\} & \delta(q_0, y) &= \{q_3, y, R\} \\ \delta(q_1, a) &= \{q_1, a, R\} & \delta(q_2, a) &= \{q_2, a, L\} & \delta(q_3, y) &= \{q_3, y, R\} \\ \delta(q_1, y) &= \{q_1, y, R\} & \delta(q_2, x) &= \{q_0, x, R\} & \delta(q_3, \square) &= \{q_f, \square, R\} \\ \delta(q_1, b) &= \{q_2, y, L\} \end{aligned}$$

Ví dụ

$q_0aaabbb$		xq_1aabbb		xaq_1abbb		$xaaq_1bbb$		xaq_2aybb
		xq_2aaybb		$q_2xaaybb$		xq_0aaybb		xxq_1aybb
		$xxaq_1ybb$		$xxayq_1bb$		$xxaq_2yyb$		xxq_2ayyb
		xq_2xayyb		xxq_0ayyb		$xxxq_1yyb$		$xxxq_1ybb$
		$xxxxyq_1b$		$xxxxyq_1b$		$xxxq_2yy$		$xxxq_2yyy$
		xxq_2xyyy		$xxxq_0yyy$		$xxxq_3yy$		$xxxq_3yy$
		$xxxxyyyq_3 \square$		$xxxxyyy \square q_f \square$		(chấp nhận)		

q_0aaabb		xq_1aabb		xaq_1abb		$xaaq_1bb$		xaq_2ayb
		xq_2aaybq_2		$xaayb$		xq_0aayb		xxq_1ayb
		$xxaq_1yb$		$xxayq_1b$		$xxaq_2yy$		xxq_2ayy
		xq_2xayy		xxq_0ayy		$xxxq_1yy$		$xxxq_1yy$
		$xxxxyq_1 \square$		(dừng)				



Máy Turing như là transducer

- Máy Turing không chỉ được quan tâm như là một bộ chấp nhận ngôn ngữ mà trong tổng quát còn cung cấp **một mô hình trừu tượng đơn giản của một máy tính số**.
- Vì mục đích chính của một máy tính là biến đổi ***input*** thành ***output***, nó hoạt động như một **transducer**.
- Hãy thử mô hình hóa máy tính bằng cách dùng máy Turing.
- ***Input*** của một sự tính toán là tất cả các kí hiệu không trắng trên băng tại thời điểm khởi đầu. Tại kết thúc của sự tính toán, ***output*** sẽ là bất kì cái gì có trên băng.
- Vậy có thể xem một máy Turing M như là một sự hiện thực của một hàm f được định nghĩa bởi

$$\hat{w} = f(w)$$

trong đó $q_0 w \vdash_M^* q_f \hat{w}$ với q_f là một trạng thái kết thúc nào đó.

Máy Turing như là transducer (tt)

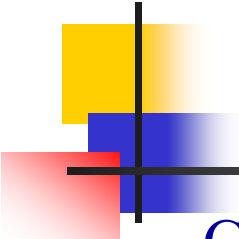
■ Định nghĩa 9.4

- Một hàm f với miền xác định D được gọi là khả tính toán-Turing hay đơn giản là khả tính toán nếu tồn tại một máy Turing nào đó $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$ sao cho

$$q_0 w \vdash_M^* q_f f(w), q_f \in F, \forall w \in D.$$

■ Ví dụ

- Cho x, y nguyên dương, thiết kế máy Turing tính $x + y$.
- Chúng ta đầu tiên chọn qui ước để biểu diễn số nguyên dương.
- Ta đã biết cách biểu diễn số nguyên dương bằng chuỗi nhị phân và cách cộng hai số nhị phân, tuy nhiên để ứng dụng điều đó vào trong trường hợp này thì hơi phức tạp một chút.
- Vậy để đơn giản hơn ta biểu diễn số nguyên dương x bằng chuỗi $w(x)$ các số 1 có chiều dài bằng x .



Ví dụ

- Chúng ta cũng phải quyết định các số x và y vào lúc ban đầu được đặt như thế nào trên băng và tổng của chúng xuất hiện như thế nào lúc kết thúc sự tính toán.
- Chúng ta giả thiết rằng $w(x)$ và $w(y)$ được phân cách bằng một kí hiệu 0 , với đầu đọc ở trên kí tự trái cùng của $w(x)$. Sau khi tính toán, $w(x+y)$ sẽ ở trên băng và được theo sau bởi một kí tự 0 , và đầu đọc sẽ được đặt trên kí tự trái cùng của kết quả.
- Chúng ta vì vậy muốn thiết kế một máy Turing để thực hiện sự tính toán (trong đó q_f là một trạng thái kết thúc)

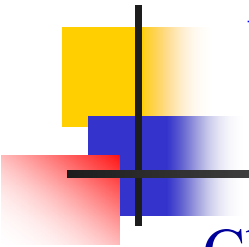
$$q_0 w(x) 0 w(y) \vdash^* q_f w(x+y) 0,$$

$$Q = \{q_0, q_1, q_2, q_3, q_f\}, F = \{q_f\}$$

$$\delta(q_0, 1) = (q_0, 1, R) \quad \delta(q_0, 0) = (q_1, 1, R) \quad \delta(q_1, 1) = (q_1, 1, R)$$

$$\delta(q_1, \square) = (q_2, \square, L) \quad \delta(q_2, 1) = (q_3, 0, L) \quad \delta(q_3, 1) = (q_3, 1, L)$$

$$\delta(q_3, \square) = (q_f, \square, R)$$



Kết hợp các máy Turing cho các công việc phức tạp

- Chúng ta đã thấy máy Turing có thể thực hiện được các phép toán cơ bản và quan trọng những cái mà có trong tất cả các máy tính.
- Vì trong các máy tính số, các phép toán cơ bản như vậy là các thành phần cơ bản cho các lệnh phức tạp hơn, vì vậy chúng ta ở đây cũng sẽ trình bày máy Turing có khả năng kết hợp các phép toán này lại với nhau.

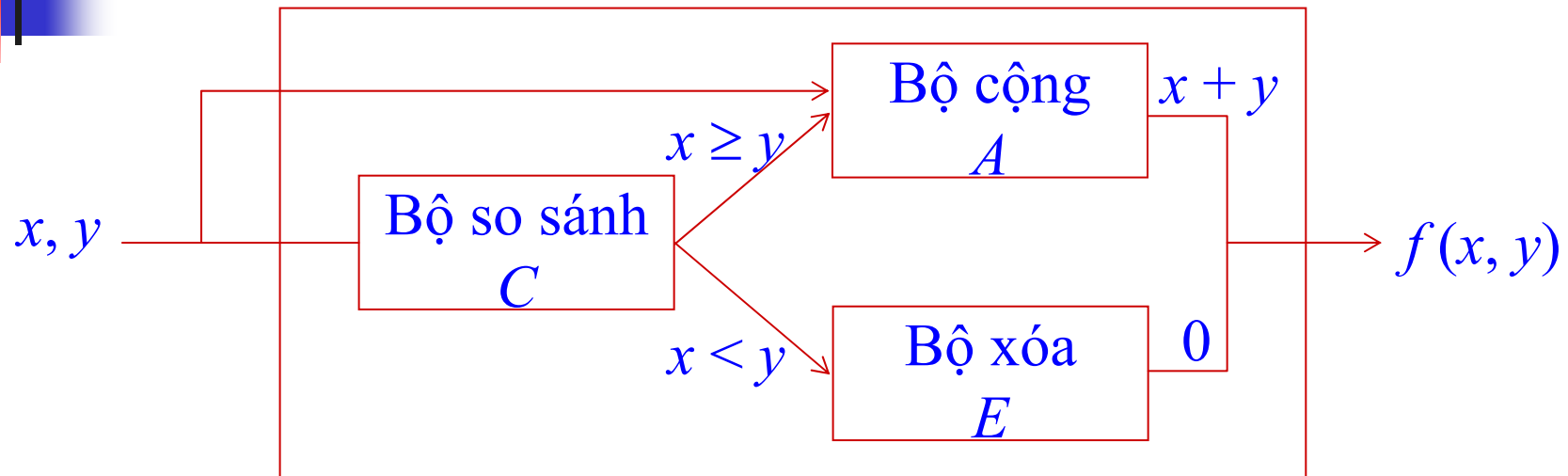
■ Ví dụ

- Thiết kế một máy Turing tính toán hàm sau

$$\begin{aligned} f(x, y) &= x + y \quad \text{nếu } x \geq y \\ &= 0 \quad \text{nếu } x < y \end{aligned}$$

- Ta xây dựng mô hình tính toán cho nó như sau

Kết hợp các máy Turing cho các công việc phức tạp (tt)

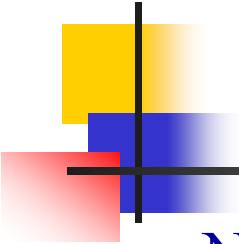


- Chúng ta sẽ xây dựng bộ so sánh C mà sau khi thực hiện xong có kết quả như sau:

$$q_{C,0}w(x)0w(y) \vdash^* q_{A,0}w(x)0w(y), \text{ nếu } x \geq y$$

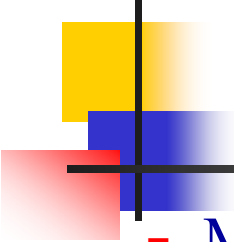
$$q_{C,0}w(x)0w(y) \vdash^* q_{E,0}w(x)0w(y), \text{ nếu } x < y$$

trong đó $q_{C,0}$, $q_{A,0}$ và $q_{E,0}$ lần lượt là trạng thái khởi đầu của bộ so sánh, bộ cộng và bộ xóa.



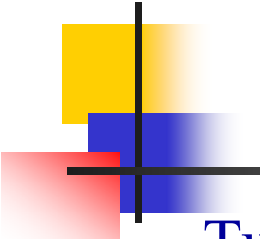
Bài tập

- Nếu chúng ta xây dựng được các bộ so sánh, bộ cộng và bộ xóa thì với mô hình kết hợp như trên chúng ta có thể xây dựng được hàm tính toán được yêu cầu.
- Xây dựng máy Turing thực hiện các phép toán sau
 - Hàm $f(x, y)$ trong slide trên
 - Phép AND, OR, XOR
 - Phép cộng hai số nhị phân



Luận đề Turing

- Máy Turing có thể được xây dựng từ các phần đơn giản hơn, tuy nhiên khá cồng kềnh cho dù phải thực hiện các phép toán đơn giản. Điều này là vì “tập lệnh” của một máy Turing là quá đơn giản và hạn chế.
- Vậy máy Turing có sức mạnh đến đâu và như thế nào trong sự so sánh với sức mạnh của máy tính ngày nay?
- Mặc dầu với cơ chế đơn giản nhưng máy Turing có thể giải quyết được các bài toán phức tạp mà máy tính ngày nay giải quyết được.
- Để chứng minh điều này người ta đã chọn ra một máy tính điển hình, sau đó xây dựng một máy Turing thực hiện được tất cả các lệnh trong tập lệnh của máy tính (tập lệnh của CPU).
- Tuy làm được điều này nhưng đó cũng chưa phải là một chứng minh chặt chẽ để chứng tỏ máy Turing có sức mạnh ngang bằng với các máy tính ngày nay.

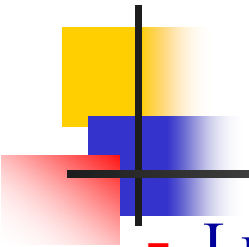


Luận đề Turing (tt)

- Tuy nhiên cũng không ai đưa ra được phản chứng chứng minh rằng máy Turing không mạnh bằng với máy tính ngày nay.
- Cuối cùng, với khá nhiều bằng chứng mạnh mẽ tuy chưa đủ là một chứng minh chặt chẽ, chúng ta chấp nhận luận đề Turing sau như là một định nghĩa của một “*sự tính toán cơ học*”

■ Luận đề Turing

- Bất kỳ cái gì có thể được thực hiện trên bất kỳ máy tính số đang tồn tại nào đều có thể được thực hiện bởi một máy Turing.
- Không ai có thể đưa ra một bài toán, có thể giải quyết được bằng những gì mà một cách trực quan chúng ta xem là một giải thuật, mà đối với nó không tồn tại máy Turing nào giải quyết được.
- Các mô hình thay thế khác có thể được đưa ra cho sự tính toán cơ học nhưng không có cái nào trong số chúng là mạnh hơn mô hình máy Turing.



Giải thuật

- Luận đề trên đóng một vai trò quan trọng trong khoa học máy tính cũng giống như vai trò của các định luật cơ bản trong vật lý và hóa học.
- Bằng việc chấp nhận luận đề Turing, chúng ta sẵn sàng để định nghĩa chính xác khái niệm giải thuật, cái mà là khá cơ bản trong khoa học máy tính.

■ Định nghĩa 9.5

- Một giải thuật cho một hàm $f: D \rightarrow R$ là một máy Turing M sao cho cho một chuỗi nhập $d \in D$ trên băng nhập, cuối cùng M dừng với kết quả $f(d) \in R$ trên băng. Một cách cụ thể là:

$$q_0 d \vdash_M^* q_f f(d), q_f \in F, \forall d \in D.$$



Chương 10 Phụ lục

10.1 Một số định nghĩa

10.2 Tổng kết các đối tượng đã học

10.3 Mối quan hệ giữa các đối tượng

10.4 Sự phân cấp các lớp ngôn ngữ hình thức theo Chomsky

10.5 Một số giải thuật quan trọng khác



Máy Turing không đơn định

- Định nghĩa 10.6

- Là máy Turing mà trong đó hàm δ được định nghĩa như sau:

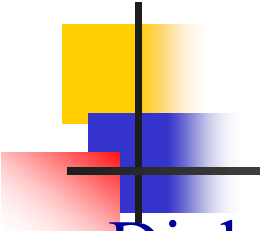
$$\delta: Q \times \Sigma \rightarrow 2^{Q \times \Sigma \times \{L, R\}}$$

- Định lý 10.5

- Lớp máy Turing không đơn định tương đương với lớp máy Turing chuẩn.

- Định lý 10.6

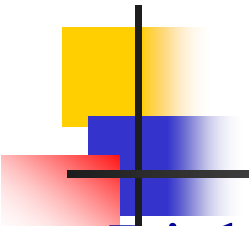
- Tập tất cả các máy Turing là vô hạn đếm được.



Ôtômát ràng buộc tuyến tính

■ Định nghĩa 10.7

- Một ôtômát ràng buộc tuyến tính (Linear Bounded Automat - LBA) là một máy Turing không đơn định $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$, như trong Định nghĩa 10.6, ngoại trừ bị giới hạn rằng Σ phải chứa hai kí tự đặc biệt $[$ và $]$, sao cho $\delta(q_i, [)$ có thể chứa chỉ một phân tử dạng $(q_j, [, R)$ và $\delta(q_i,])$ có thể chứa chỉ một phân tử dạng $(q_j,], L)$.
- Bằng lời, khi đầu đọc chạm đến dấu móc vuông ở một trong hai đầu nó phải giữ lại và đồng thời không thể vượt ra vùng nằm giữa hai dấu móc vuông.
- Trong trường hợp này chúng ta nói đầu đọc bị giới hạn giữa hai dấu móc vuông hai đầu.



Ôtômát ràng buộc tuyến tính (tt)

■ Định nghĩa 10.7

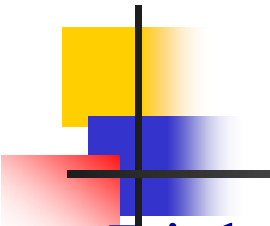
- Một chuỗi được chấp nhận bởi một ôtômát ràng buộc tuyến tính nếu có một dãy chuyển hình trạng có thể

$$q_0[w] \vdash^* [x_1 q_f x_2]$$

với một q_f nào đó $\in F$, $x_1, x_2 \in \Sigma^*$. Ngôn ngữ được chấp nhận bởi ***lba*** là tập tất cả các chuỗi được chấp nhận bởi ***lba***.

■ Ví dụ

- Ngôn ngữ $L = \{a^n b^n c^n : n \geq 0\}$ là một ngôn ngữ ràng buộc tuyến tính vì chúng ta có thể xây dựng được một ***lba*** chấp nhận đúng nó.



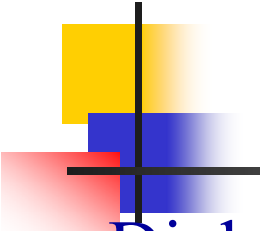
Ngôn ngữ khả liệt kê đệ qui, đệ qui

■ Định nghĩa 10.8

- Một ngôn ngữ L được gọi là khả liệt kê đệ qui nếu tồn tại một máy Turing M chấp nhận nó.
- Từ định nghĩa này cũng dễ dàng suy ra được mọi ngôn ngữ mà đối với nó tồn tại một thủ tục liệt kê (các phân tử của nó) thì khả liệt kê đệ qui.

■ Định nghĩa 10.9

- Một ngôn ngữ L trên Σ được gọi là đệ qui nếu tồn tại một máy Turing M chấp nhận nó và dừng đối với $w \in \Sigma^+$. Hay nói cách khác một ngôn ngữ là đệ qui nếu và chỉ nếu tồn tại một giải thuật thành viên cho nó.



Văn phạm

■ Định nghĩa 10

- Một văn phạm mà mọi luật sinh không cần thỏa bất kỳ ràng buộc nào tức là có dạng

$$\alpha \rightarrow \beta$$

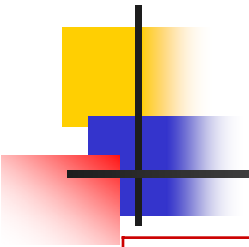
trong đó $\alpha \in (V \cup T)^* V (V \cup T)^*$, $\beta \in (V \cup T)^*$ thì được gọi là ***văn phạm loại 0*** hay là ***văn phạm không hạn chế***.

- Một văn phạm mà mọi luật sinh có dạng chiều dài vế trái nhỏ hơn hoặc bằng chiều dài vế phải tức là có dạng

$$\alpha \rightarrow \beta$$

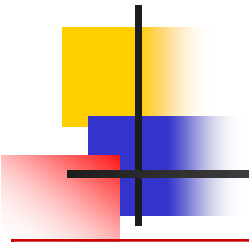
trong đó $\alpha \in (V \cup T)^* V (V \cup T)^*$, $\beta \in (V \cup T)^*$ và $|\alpha| \leq |\beta|$ thì được gọi là ***văn phạm loại 1*** hay ***văn phạm cảm ngữ cảnh***.

- Văn phạm phi ngữ cảnh còn được gọi là ***văn phạm loại 2***.
- Văn phạm chính qui còn được gọi là ***văn phạm loại 3***.



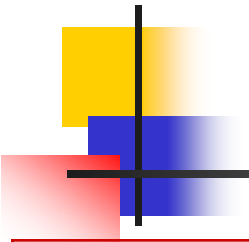
Tổng kết các lớp đối tượng

Các lớp ngôn ngữ		Kí hiệu
Chính qui	<u>R</u> egular	L_{REG}
Tuyến tính	<u>L</u> inear	L_{LIN}
Phi ngữ cảnh đơn định	<u>D</u> eterministic <u>C</u> ontext- <u>F</u> ree	L_{DCF}
Phi ngữ cảnh	<u>C</u> ontext- <u>F</u> ree	L_{CF}
Cảm ngữ cảnh	<u>C</u> ontext- <u>S</u> ensitive	L_{CS}
Đệ qui	<u>R</u> ecursive	L_{REC}
Khả liệt kê đệ qui	<u>R</u> ecursively <u>E</u> numerable	L_{RE}



Tổng kết các lớp đối tượng (tt)

Các lớp văn phạm		Kí hiệu
Chính qui $\equiv$ Tuyến tính-phải và tuyến tính-trái $\equiv$ Loại 3	<u>R</u> egular $\equiv$ <u>R</u> ight- <u>L</u> inear và <u>L</u> eft- <u>L</u> inear	$G_{\text{REG}} \equiv G_{\text{R-LIN}}$ và $G_{\text{L-LIN}}$
Tuyến tính	<u>L</u> inear	G_{LIN}
Phi ngữ cảnh đơn định: điển hình là LL(k) và LR(k)	LL(k) và LR(k)	G_{LL} và G_{LR}
Phi ngữ cảnh $\equiv$ Loại 2	<u>C</u> ontext- <u>F</u> ree	G_{CF}
Cảm ngữ cảnh $\equiv$ Loại 1	<u>C</u> ontext- <u>S</u> ensitive	G_{CS}
Không hạn chế $\equiv$ Loại 0	<u>U</u> n <u>R</u> estricted	G_{UR}



Tổng kết các lớp đối tượng (tt)

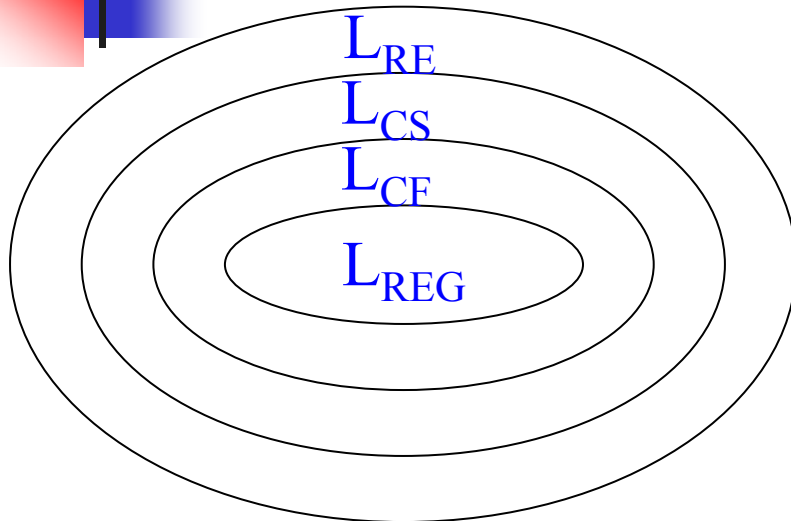
Các lớp ô tô măt		Kí hiệu
Hữu hạn	<u>F</u> inite <u>S</u> tate	FSA (nfa, dfa)
Đẩy xuống đơn định	<u>D</u> eterministic <u>P</u> ush <u>D</u> own	DPDA
Đẩy xuống không đơn định	<u>N</u> ondeterministic <u>P</u> ush <u>D</u> own	NPDA
Ràng buộc tuyến tính	<u>L</u> inear <u>B</u> ounded	LBA
Máy Turing	<u>T</u> uring <u>M</u> achine	TM

Mối quan hệ giữa các lớp đối tượng

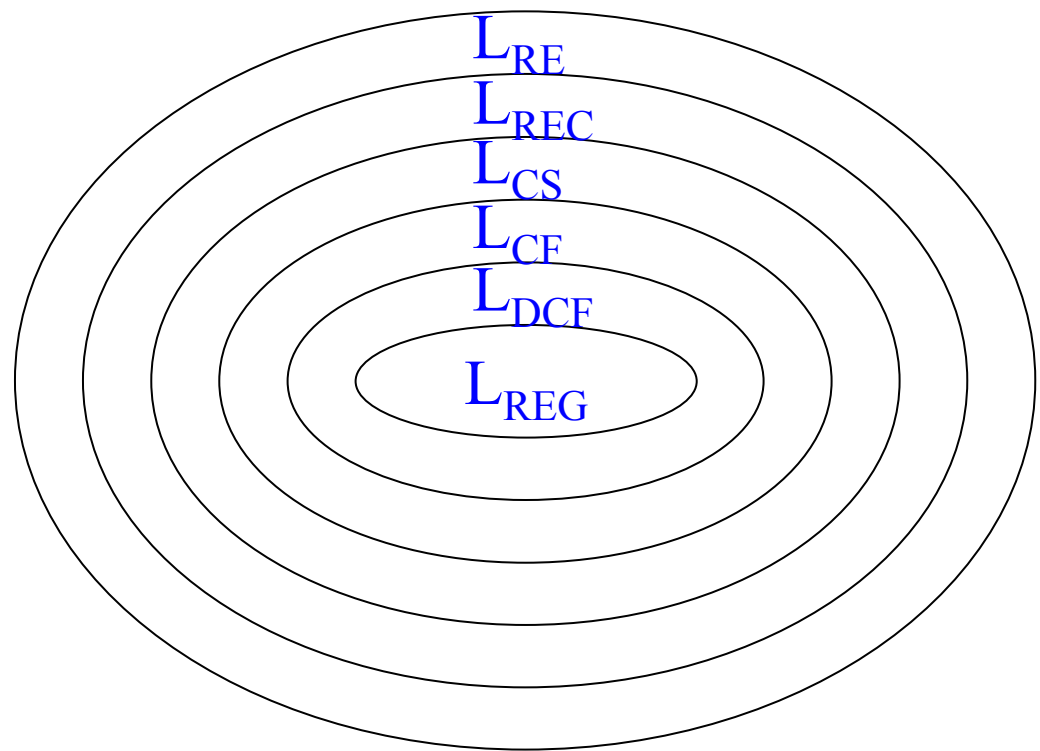
Ngôn ngữ	Văn phạm	Ôtômát
L_{REG}	$G_{\text{REC}} \equiv G_{\text{L-LIN}}$ và $G_{\text{R-LIN}}$	$\text{FSA} \equiv \text{DFA} = \text{NFA}$
L_{LIN}	G_{LIN}	$\subset \text{NPDA}$
L_{DCF}	$\supset \text{LL}(k)$ và $\text{LR}(k)$	DPDA
L_{CF}	G_{CF}	NPDA
L_{CS}	G_{CS}	LBA
L_{REC}	$\subset G_{\text{UR}}$	$\subset \text{TM}$
L_{RE}	G_{UR}	TM

- Dấu $\equiv$ có nghĩa là theo định nghĩa, còn dấu $=$ có nghĩa là tương đương, dấu $\supset$ có nghĩa là tập cha (không bằng), dấu $\subset$ có nghĩa là tập con (không bằng).

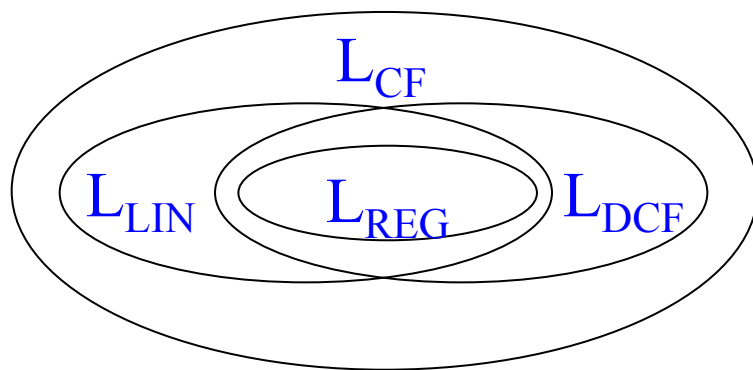
Phân cấp ngôn ngữ theo Chomsky



Sơ đồ phân cấp đơn giản



Sơ đồ phân cấp chi tiết



Sơ đồ phân cấp trong lớp PNC